

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 1_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 15

Section 1 : MCQ

1. What is the output of the following code?

```
class TestClass {  
    public static void main(String[] args) {  
        int a = 5;  
        int b = 10;  
  
        int sum = a + b;  
        int bitwiseAnd = a & b;  
        int bitwiseOr = a | b;  
  
        System.out.println(sum);  
        System.out.println(bitwiseAnd);  
        System.out.println(bitwiseOr);  
    }  
}
```

Answer

15015

Status : Correct

Marks : 1/1

2. Which of the following data types is used to store single characters?

Answer

char

Status : Correct

Marks : 1/1

3. What will be the output of the following program?

```
class DataTypesMCQ {  
    public static void main(String[] args) {  
        int a = 10;  
        double b = 5;  
        System.out.println(a / b);  
    }  
}
```

Answer

2.0

Status : Correct

Marks : 1/1

4. What will be the output of the following code snippet?

```
class DivisionExample {  
    public static void main(String[] args) {  
        double num1 = 10.5;  
        double num2 = 3;  
        int result = (int)(num1 / num2);  
        System.out.println(result);  
    }  
}
```

Answer

3

Status : Correct

Marks : 1/1

5. What is the output of the following program?

```
class Demo {  
    public static void main(String[] args) {  
        String text = "Hello, World!";  
        System.out.println(text);  
    }  
}
```

Answer

Hello, World!

Status : Correct

Marks : 1/1

6. What will be the output of the following code snippet?

```
import java.util.*;  
  
class OperatorPrecedenceExample {  
    public static void main(String[] args) {  
        int a = 5, b = 3, c = 2;  
        int result = a + b * c;  
  
        System.out.println(result);  
    }  
}
```

Answer

11

Status : Correct

Marks : 1/1

7. What is the output of the following code?

```
class TestClass {  
    public static void main(String[] args) {  
        int x = 5;  
        int X = 10;  
  
        int sum = x + X;  
        int bitwiseResult = x | X;  
  
        System.out.println(sum);  
        System.out.println(bitwiseResult);  
    }  
}
```

Answer

1515

Status : Correct

Marks : 1/1

8. What is the result of the following expression?

```
import java.util.*;
```

```
class ComplexExpressionExample {  
    public static void main(String[] args) {  
        int a = 5, b = 2, c = 3, d = 4;  
        int result = a + b * c / d - b;  
  
        System.out.println(result);  
    }  
}
```

Answer

4

Status : Correct

Marks : 1/1

9. What is the output of the following code?

```
import java.util.*;

class RelationalOperatorExample {
    public static void main(String[] args) {
        int x = 8, y = 4;
        boolean result = (x != y);

        System.out.println(result);
    }
}
```

Answer

true

Status : Correct

Marks : 1/1

10. Which of the following data types is used to store floating-point numbers with greater precision?

Answer

double

Status : Correct

Marks : 1/1

11. Which of the following is not a primitive data type?

Answer

string

Status : Correct

Marks : 1/1

12. What is the output of the following program?

```
class Arithmetic {
    public static void main(String[] args) {
        char ch = 'A';
        System.out.println(ch);
    }
}
```

}

Answer

A

Status : Correct

Marks : 1/1

13. What will be the output of the following code?

```
import java.util.*;
```

```
class TernaryOperatorExample {  
    public static void main(String[] args) {  
        int a = 5, b = 10;  
        int result = (a > b) ? a : b;  
        System.out.println(result);  
    }  
}
```

Answer

10

Status : Correct

Marks : 1/1

14. What is the output of the following code?

```
class TestClass {  
    public static void main(String[] args) {  
        int count = 8;  
        count = count ^ 1;  
  
        System.out.println(count);  
    }  
}
```

Answer

9

Status : Correct

Marks : 1/1

15. What is the output of the following code?

```
class TestClass {  
    public static void main(String[] args) {  
        int a = 10;  
        int b = 3;  
        System.out.println(a / b);  
    }  
}
```

Answer

3

Status : Correct

Marks : 1/1

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 1_Q1

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Gloria is responsible for monitoring the performance of two machines in a factory. She needs to determine which of the two machines is operating closest to the optimal temperature of 100 degrees Celsius using the relational operator.

Assist Gloria in displaying the machine's temperature, which is closer to 100, and the difference from 100.

Input Format

The first line of input consists of an integer N, representing the temperature of the first machine.

The second line consists of an integer M, representing the temperature of the second machine.

Output Format

The output prints "The integer closer to 100 is X with a difference of Y" where X is the temperature of the closer machine and Y is the difference from 100.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 90

80

Output: The integer closer to 100 is 90 with a difference of 10

Answer

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int N = sc.nextInt();
        int M = sc.nextInt();

        int diffN = Math.abs(N - 100);
        int diffM = Math.abs(M - 100);

        if (diffN < diffM) {
            System.out.println("The integer closer to 100 is " + N + " with a difference
of " + diffN);
        } else if (diffM < diffN) {
            System.out.println("The integer closer to 100 is " + M + " with a difference
of " + diffM);
        } else {
            System.out.println("The integer closer to 100 is " + N + " with a difference
of " + diffN);
        }
    }
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 1_Q2

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. PROBLEM STATEMENT:

Dave got two students who want help with their doubt. Each hands out an integer and wants to find if one integer is positive while the other is not divisible by 3. Write a program to achieve this and conclude for them.

Input Format

The first line of input represents the first integer.

The second line of input represents the second integer.

Output Format

The output should display as "One of the integers is positive while the other is not divisible by 3." or "Neither of the integers meets the condition."

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

3

Output: One of the integers is positive while the other is not divisible by 3.

Answer

```
import java.util.Scanner;
```

```
public class Main {  
    public static void main(String [] args) {  
        Scanner sc = new Scanner(System.in);  
        int a = sc.nextInt();  
        int b = sc.nextInt();  
  
        boolean case1 = (a > 0 && b % 3 != 0);  
        boolean case2 = (a > 0 && a % 3 != 0);  
  
        if (case1 || case2) {  
            System.out.println("One of the integers is positive while the other is not  
divisible by 3.");  
        } else {  
            System.out.println("Neither of the integers meets the condition.");  
        }  
    }  
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 1_Q3

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem statement

Manoj, a developer at MoneyMatters Inc., is working on improving the company's financial system. He needs to create a program that takes an integer input, converts it into a double, and displays both the original integer and the converted double value.

Input Format

The input consists of a single integer representing a monetary amount.

Output Format

The first line of the output displays the "Original Integer: ", followed by an integer representation of the input value.

The second line displays the "Converted Double: ", followed by a double value representing the input as a decimal value.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 20

Output: Original Integer: 20

Converted Double: 20.0

Answer

```
import java.util.Scanner;
public class Main {
    public static void main(String [] args) {
        Scanner sc = new Scanner(System.in);
        int num = sc.nextInt();
        double d = (double) num;
        System.out.println("Original Integer: " + num);
        System.out.println("Converted Double: "+ d);
    }
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 1_Q4

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Vishal and Arun are discussing the properties of numbers. Vishal gives Arun two integers. He asks Arun to check if the sum of these two numbers is a multiple of their product.

Can you assist Arun and determine whether the sum is a multiple of the product?

Input Format

The input consists of two space-separated integers.

Output Format

The output prints:

1. "Sum is Multiple of Product" if the sum of the two numbers is divisible by their product.
2. "Sum is Not Multiple of Product" otherwise.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1 2

Output: Sum is Not Multiple of Product

Answer

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int a = sc.nextInt();
        int b = sc.nextInt();
        int sum = a + b;
        int product = a * b;
        if (sum % product == 0) {
            System.out.println("Sum is Multiple of Product");
        } else {
            System.out.println("Sum is Not Multiple of Product");
        }
    }
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 1_Q5

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement:

Emily has a beautiful circular garden in her backyard. She's interested in calculating two important measurements for her garden: the circumference and the area. To do this, she needs a program that can take the radius of her circular garden as input and provide the calculated circumference and area as output. The formulas she should use are as follows:

To calculate the circumference (C) of a circle, you can use the formula:

$$C = 2 * \pi * r$$

$$A = \pi * r^2$$

Where:

C represents the circumference.

A represents the area.

π (pi) is approximately 3.14159.

r is the radius of the circle.

Emily is not a programmer, and she needs your help to create a program that will make these calculations for her garden.

Input Format

The first line of input contains a single double-point number radius, representing the radius of the circle.

Output Format

The output should consist of two lines:

The first line should print the circumference of the circle rounded to 2 decimal places, followed by the unit "meters".

The second line should print the area of the circle rounded to 2 decimal places, followed by the unit "square meters".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3.0

Output: Circumference: 18.85 meters

Area: 28.27 square meters

Answer

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        double r = sc.nextDouble();
        double pi = 3.14159;
        double circumference = 2 * pi * r;
```

```
double area = pi * r * r;  
System.out.printf("Circumference: %.2f meters%n", circumference);  
System.out.printf("Area: %.2f square meters%n", area);  
}  
  
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 1_Q6

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Joey is learning about bitwise operations and is working on a project that involves extracting specific bits from integers. He needs to write a program that takes an integer and the number of bits N as input and outputs the value of the lowest N bits of the integer.

Help Joey in his project to understand and visualize how bitwise operations work in practical scenarios.

Input Format

The first line of input consists of an integer X, representing the given integer.

The second line consists of an integer N, representing the number of bits to extract.

Output Format

The output displays "Result: " followed by an integer representing the value of the lowest N bits of the given integer.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 85

2

Output: Result: 1

Answer

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        int X = sc.nextInt();
        int N = sc.nextInt();

        int mask = (1 << N) - 1;
        int result = X & mask;

        System.out.println("Result: " + result);
    }
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 1_Q7

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement:

Miles is working on a program that involves analyzing two integers. He wants to check if either one of the integers is both:

Less than or equal to zero, and Odd. Can you help him create a program that identifies whether either of the integers meets these conditions?

Input Format

The input consists of two integers on separate lines, denoted as 'input1' and 'input2'.

Output Format

A single line with a boolean result (either 'true' or 'false') indicating whether either 'input1' or 'input2' is both less than or equal to zero and odd.

Refer to the sample output for format specifications

Sample Test Case

Input: -45

10

Output: true

Answer

```
import java.util.Scanner;
```

```
public class Main
```

```
{
```

```
    public static void main(String[] args)
```

```
{
```

```
    Scanner scanner = new Scanner(System.in);
```

```
    // Read inputs
```

```
    int input1 = scanner.nextInt();
```

```
    int input2 = scanner.nextInt();
```

```
    // Check if either number is both less than or equal to 0 AND odd
```

```
    boolean condition1 = (input1 <= 0) && (input1 % 2 != 0);
```

```
    boolean condition2 = (input2 <= 0) && (input2 % 2 != 0);
```

```
    // Final result
```

```
    boolean result = condition1 || condition2;
```

```
    // Output result
```

```
    System.out.println(result);
```

```
    scanner.close();
```

}

}

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 1_Q8

Attempt : 1
Total Mark : 10
Marks Obtained : 5

Section 1 : Coding

1. Problem Statement

In the Kingdom of Finance, the royal treasury is managed by the treasurer, Sir Cedric. Sir Cedric tracks the daily expenses of the kingdom using an expense report that lists three major categories: food, clothing, and utilities. However, the King wants to know if the average daily expense is greater than at least two of these categories to ensure the kingdom is spending wisely.

Your task is to help Sir Cedric determine if the average daily expense is greater than two of the categories. Specifically, you need to calculate the average of the three expenses and check if it is greater than any two categories.

Note: Use the ternary operator

Input Format

Three integers a, b, and c represent the daily expenses for food, clothing, and utilities. Each integer is provided on a single line.

Output Format

The average of the three expenses, rounded to two decimal places.

A message indicating whether the average is greater than at least two of the expense categories.

1. If the average is greater than the two smallest monthly expenses, print "Average is greater than both X and Y," where X and Y are the two smallest expenses.
2. Otherwise, display "Average is not greater than two smallest expenses".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4

6

10

Output: 6.67

Average is greater than both 4 and 6

Answer

```
import java.util.Scanner;  
import java.util.Arrays;
```

```
public class Main
```

```
{
```

```
    public static void main(String[] args)
```

```
{
```

```

Scanner sc = new Scanner(System.in);

int a = sc.nextInt(); // Food
int b = sc.nextInt(); // Clothing
int c = sc.nextInt(); // Utilities

int[] expenses =

{

a, b, c

};

Arrays.sort(expenses);
int smallest1 = expenses[0];
int smallest2 = expenses[1];

double average = (a + b + c) / 3.0;

// Print average rounded to 2 decimal places
System.out.printf("%.2f\n", average);

// Output must match exactly
String result = (average > smallest1 && average > smallest2)
    ? String.format("Average is greater than both %d and %d", smallest1,
smallest2)
    : "Average is not greater than two smallest expenses";

System.out.println(result);
sc.close();

}

}

```

Status : Partially correct

Marks : 5/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 1_Q9

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Phill is a quality control manager at a manufacturing plant. He needs to verify if a sensor reading at a midpoint station (S2) falls exactly halfway between the readings of the previous station (S1) and the next station (S3). Help him by developing a program that checks if the second sensor reading is the average (midpoint) of the first and third sensor readings.

Use the relational operator to solve the program.

Input Format

The first line of input consists of an integer S1, representing the sensor reading of the first station.

The second line consists of an integer S2, representing the sensor reading of the midpoint station.

The third line consists of an integer S3, representing the sensor reading of the next station.

Output Format

The first line of output displays a boolean value representing whether the sensor reading at the midpoint station is halfway between the readings of the first and the next stations.

The second line displays one of the following:

1. If the result is true, print "The second integer is halfway between the first and third integers."
2. Otherwise, print "The second integer is not halfway between the first and third integers."

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1

7

10

Output: false

The second integer is not halfway between the first and third integers.

Answer

```
import java.util.Scanner;
```

```
public class Main
```

```
{
```

```
    public static void main(String[] args)
```

```
{
```

```
    Scanner scanner = new Scanner(System.in);
```

```
// Input sensor readings
int S1 = scanner.nextInt();
int S2 = scanner.nextInt();
int S3 = scanner.nextInt();

// Check if S2 is halfway between S1 and S3
boolean isHalfway = S2 == (S1 + S3) / 2.0;

// Print boolean result
System.out.println(isHalfway);

// Print appropriate message
if (isHalfway)
{

    System.out.println("The second integer is halfway between the first and
third integers.");

} else
{

    System.out.println("The second integer is not halfway between the first
and third integers.");

}

scanner.close();

}

}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 1_Q10

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Aishu is supervising a construction project that needs to be completed with the help of three workers: A, B, and C.

She knows how many days each of them would take to complete the entire project individually:

A can complete it in x days, B in y days, C in z days.

Initially, all three workers (A, B, and C) work together for d1 days.

After that, C leaves, and only A and B continue for another d2 days.

Then B also leaves, and A works alone to finish the remaining work.

Your task is to help Aishu to implement this functionality using the class WorkDistribution and Method calculateWork(int x, int y, int z, int d1, int d2)

Calculate the total work completed in the first d_1 days by A, B, and C. Calculate the work completed in the next d_2 days by A and B. Determine the remaining work after these $d_1 + d_2$ days.

Input Format

The first line of input contains five space-separated integers: x y z d_1 d_2

where:

x represents the Days A takes to complete the work alone

y represents the Days B takes to complete the work alone

z represents the Days C takes to complete the work alone

d_1 represents the Days A, B, and C work together

d_2 represents the Days A and B work together (after C leaves)

Output Format

The first line of output prints "Work done in first d_1 days (A+B+C): " followed by a double value rounded to 2 decimal places.

The second line of output prints "Work done in next d_2 days (A+B): " followed by a double value rounded to 2 decimal places.

The third line prints "Remaining work: " followed by a double value rounded to 2 decimal places.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 10 20 30 2 2

Output: Work done in first d_1 days (A+B+C): 0.37

Work done in next d_2 days (A+B): 0.30

Remaining work: 0.33

Answer

```
import java.util.Scanner;
```

```
class Main
```

```
{
```

```
    public static void calculateWork(int x, int y, int z, int d1, int d2)
```

```
    {
```

```
        // Work done per day by each worker
```

```
        double rateA = 1.0 / x;
```

```
        double rateB = 1.0 / y;
```

```
        double rateC = 1.0 / z;
```

```
        // Work done in first d1 days by A, B, and C
```

```
        double work1 = d1 * (rateA + rateB + rateC);
```

```
        // Work done in next d2 days by A and B
```

```
        double work2 = d2 * (rateA + rateB);
```

```
        // Total work = 1. Remaining work = 1 - work1 - work2
```

```
        double remainingWork = 1.0 - (work1 + work2);
```

```
        // Rounding to 2 decimal places
```

```
        System.out.printf("Work done in first d1 days (A+B+C): %.2f\n", work1);
```

```
        System.out.printf("Work done in next d2 days (A+B): %.2f\n", work2);
```

```
        System.out.printf("Remaining work: %.2f\n", remainingWork);
```

```
    }
```

```
    public static void main(String[] args)
```

```
    {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        int x = sc.nextInt(); // Days A takes
```



```
int y = sc.nextInt(); // Days B takes  
int z = sc.nextInt(); // Days C takes  
int d1 = sc.nextInt(); // Days all three work together  
int d2 = sc.nextInt(); // Days A and B work together  
calculateWork(x, y, z, d1, d2);  
sc.close();
```

```
}
```

```
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 1_PAH

Attempt : 1
Total Mark : 40
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Mickey and Miney are walking through a magical forest. The forest is full of enchanted stones, each with a unique number. There is a legend that says the magic power of the stones can be revealed by using a special operation. To determine the magic power of a given stone, you need to perform a bitwise AND operation with the number 15.

Each stone's number is represented by an integer, and Mickey needs to find the magic power of each stone by applying this operation.

Your task is to help Mickey compute the result of the bitwise AND operation of the given stone number with 15, and print the result.

Input Format

The input consists of a single integer.

Output Format

The output should display a single integer, which is the result of the bitwise AND operation between input and 15.

Refer to the sample output for format specifications.

Sample Test Case

Input: 25

Output: 9

Answer

-

Status : Skipped

Marks : 0/10

2. PROBLEM STATEMENT:

Maria, a software developer, is working on a program to determine if two given integers which can be either positive or negative integers have the same parity (both even or both odd). She needs your help in writing this program.

Write a program that takes two integers as input and checks if both integers are either even or odd.

Input Format

The input consists of two lines:

The first line consists of an integer (input1) which can be either positive or negative.

The second line consists of an integer (input2) which can be either positive or negative.

Output Format

The output is displayed in the following format:

If both integers have the same parity (i.e., both even or both odd), print:

"Both integers are either even or odd"

Otherwise, print:

"The integers have different parities"

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 2

-4

Output: Both integers are either even or odd

Answer

-

Status : Skipped

Marks : 0/10

3. PROBLEM STATEMENT:

Maria, a software developer, is working on a project to create a simple program to determine which of two integers is closest to zero. The integers can be either positive or negative. The program needs to take two integer inputs and calculate which one is closer to zero. If both integers are equidistant from zero, the program should return 0.

Input Format

The input contains two lines:

The first line of the input contains an integer, which can be either a positive or a negative integer.

The second line of the input contains an integer, which can be either a positive or a negative integer.

Output Format

The output displays the integer that is closest to zero in the following format:

"The integer closest to zero is: [closest_integer]"

Here, [closest_integer] should be replaced with the integer that is closer to zero based on its absolute value.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5
8

Output: The integer closest to zero is: 5

Answer

```
import java.util.Scanner;
public class Main
{
    public static void main(String[] args)
    {
```

```
Scanner scanner = new Scanner(System.in);
int num1 = scanner.nextInt();
int num2 = scanner.nextInt();
int abs1 = Math.abs(num1);
int abs2 = Math.abs(num2);
if (abs1 < abs2)

{

    System.out.println("The integer closest to zero is: " + num1);

}
else if ( abs2 < abs1)
{

    System.out.println("The integer closest to zero is: " + num2);

}
else
{

    System.out.println("The integer closest to zero is: 0" );

}

}

}
```

Status : Correct

Marks : 10/10

4. Problem Statement

In the Kingdom of Delivery Logistics, there is a giant truck used for transporting packages across the kingdom. The truck has a maximum capacity represented by an integer, and each package also has a specific weight. The truck's efficiency and safety depend on whether the weight of the package is below a certain threshold.

The kingdom's delivery service has a rule: if the weight of a package is less than one-third of the truck's total capacity, the package is eligible for quick processing and dispatch. However, if the weight is too heavy, the package will require special handling.

As a logistics manager, you need to check whether the weight of the package is less than one-third of the truck's total capacity.

Write a program using a ternary operator that helps determine whether the package weight meets the requirement for quick processing or if it needs special handling.

Input Format

The first line of input consists of an integer p , representing the weight of the package.

The second line consists of an integer w , representing the total weight capacity of the truck.

Output Format

The first line of output prints "One-third of Truck: X," where X is one-third of the truck's total weight capacity as a double value with two decimal places.

The second line of output displays one of the following:

1. If p is less than one-third of the truck's total weight capacity, print "Package weight is less than one-third of the truck's capacity".
2. Otherwise, print "Package weight is not less than one-third of the truck's capacity".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 13

60

Output: One-third of Truck: 20.00

Package weight is less than one-third of truck's capacity

Answer

-

Status : -

Marks : 0/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 1_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement:

"Write a program that helps identify the type of a triangle based on the lengths of its three sides. The program prompts the user to input the lengths of sides 'a', 'b', and 'c', and then it classifies the triangle as 'Equilateral' if all sides are equal, 'Isosceles' if two sides are equal, or 'Scalene' if all sides are different. Can you provide the Java code for this task?"

Input Format

The first line of the input is an integer 'a' representing the length of side 'a.'

The second line of the input is an integer 'b' representing the length of side 'b.'

The third line of the input is an integer 'c' representing the length of side 'c.'

Output Format

The program outputs a single line that specifies the type of the triangle:
"Equilateral," "Isosceles," or "Scalene."

Sample Test Case

Input: 3

4

5

Output: The triangle is Scalene

Answer

```
import java.util.Scanner;
```

```
public class Main
```

```
{
```

```
    public static void main(String[] args)
```

```
{
```

```
    Scanner sc = new Scanner(System.in);
```

```
    int a = sc.nextInt();
```

```
    int b = sc.nextInt();
```

```
    int c = sc.nextInt();
```

```
    // Check if the sides can form a triangle first (optional but good practice)
```

```
    if (a + b <= c || a + c <= b || b + c <= a)
```

```
{
```

```
    System.out.println("Not a valid triangle");
```

```
    } else if (a == b && b == c)
```

```
{
```

```
        System.out.println("The triangle is Equilateral");

    } else if (a == b || b == c || a == c)
    {

        System.out.println("The triangle is Isosceles");

    } else
    {

        System.out.println("The triangle is Scalene");

    }

    sc.close();

}

}
```

Status : Correct

Marks : 10/10

2. Problem Statement

Mandy is a software engineer working on a program to analyze two integers based on specific conditions using a logical operator. She needs to determine if both integers are odd or if at least one of them is divisible by 7.

Depending on the result, she wants to print different messages.

If the condition is met, the program should identify and print the first number that is divisible by 7 or indicate that both numbers are odd. If the condition is not met, the program should print a message indicating the condition was not met, along with the input numbers.

Input Format

The first line of input consists of an integer representing the first input number.

The second line consists of an integer representing the second input number.

Output Format

The output displays "Condition met: " followed by an integer representing the first number divisible by 7, or prints "Both numbers are odd" if the two inputs are odd.

If the condition is not met, it displays "Conditions not met: " followed by the two input integers, separated by a space.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 7

14

Output: Condition met: 7

Answer

```
import java.util.Scanner;
```

```
public class Main
```

```
{
```

```
    public static void main(String[] args)
```

```
{
```

```
Scanner sc = new Scanner(System.in);
```

```
int num1 = sc.nextInt();
```

```
int num2 = sc.nextInt();
```

```
boolean bothOdd = (num1 % 2 != 0) && (num2 % 2 != 0);
```

```
boolean firstDivBy7 = (num1 % 7 == 0);
```

```
boolean secondDivBy7 = (num2 % 7 == 0);
```

```
if (bothOdd || firstDivBy7 || secondDivBy7)
```

```
{
```

```
    System.out.print("Condition met: ");
```

```
    if (firstDivBy7)
```

```
{
```

```
        System.out.println(num1);
```

```
    } else if (secondDivBy7)
```

```
{
```

```
        System.out.println(num2);
```

```
    } else
```

```
{
```

```
        System.out.println("Both numbers are odd");
```

```
}
```

```
} else
```

```
{
```

```
    System.out.println("Conditions not met: " + num1 + " " + num2);
```

```
}
```

```
    sc.close();
```

```
}
```

```
}
```

Status : Correct

Marks : 10/10

3. PROBLEM STATEMENT:

Julia a mathematician expert is given two integers to find if the second integer is above the average of the first and second integer. Write a program that achieves this using the ternary operator.

Input Format

The first line of input represents the first integer.

The second line of input represents the second integer.

Output Format

The output should be displayed as "Below Average" or "Above Average"

REFER THE SAMPLE TESTCASES FOR THE FORMAT SPECIFICATIONS.

Sample Test Case

Input: 1

1

Output: Below Average

Answer

```
import java.util.Scanner;
```

```
public class Main
```

```
{
```

```
    public static void main(String[] args)
```

```
{
```

```
    Scanner sc = new Scanner(System.in);
```

```
    int num1 = sc.nextInt();
```

```
    int num2 = sc.nextInt();
```

```
    double average = (num1 + num2) / 2.0;
```

```
    String result = (num2 > average) ? "Above Average" : "Below Average";
```

```
    System.out.println(result);
```

```
    sc.close();
```

```
}
```

```
}
```

Status : Correct

Marks : 10/10

4. Problem Statement:

Tom is tasked with writing a program that determines whether a given integer is the square of another integer. A perfect square is a number that can be expressed as the square of an integer. The program should take an integer as input and determine if it is a perfect square or not.

The task is to implement the logic to check if the provided integer is the square of an integer and return the result.

Input Format

The first line of the input contains an integer, "input", where |input| represents the absolute value of the integer.

Output Format

The output should display a boolean value, "result," which should be set to true if the input is a perfect square (the square of an integer), and false if it is not.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 16

Output: Is the integer a perfect square? true

Answer

```
import java.util.Scanner;

public class Main

{

    public static void main(String[] args)

    {

        Scanner sc = new Scanner(System.in);
```



```
int input = sc.nextInt();
boolean result = false;
if (input >= 0)
{

    int sqrt = (int) Math.sqrt(input);
    result = (sqrt * sqrt == input);

}
System.out.println("Is the integer a perfect square? " + result);

sc.close();
}
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 2_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 15

Section 1 : MCQ

1. What will be the output of the following code?

```
class Main {  
    public static void main(String[] args) {  
        for (int i = 5; i > 0; i--) {  
            System.out.print(i + " ");  
        }  
    }  
}
```

Answer

5 4 3 2 1

Status : Correct

Marks : 1/1

2. What will be the output of the following code?

```
public class Main {  
    public static void main(String[] args) {  
        int sum = 0;  
        for(int i = 1; i <= 5; i++) {  
            sum += i;  
        }  
        System.out.println(sum);  
    }  
}
```

Answer

15

Status : Correct

Marks : 1/1

3. What will be the output of the following code?

```
public class Main {  
    public static void main(String[] args) {  
        int i = 1;  
        while(i < 10) {  
            i += 2;  
        }  
        System.out.println(i);  
    }  
}
```

Answer

11

Status : Correct

Marks : 1/1

4. What will be the output of the following code?

```
class ConditionTest {  
    public static void main(String[] args) {  
        int x = 10;  
    }  
}
```

```
    if (x > 5)
        System.out.print("High");
    }
}
```

Answer

High

Status : Correct

Marks : 1/1

5. What will be the output of the following code?

```
class LoopTest {
    public static void main(String[] args) {
        int i = 1;
        while (i > 0) {
            System.out.print(i + " ");
            i++;
            if (i == 5) break;
        }
    }
}
```

Answer

1 2 3 4

Status : Correct

Marks : 1/1

6. What will be the output of the following code?

```
public class Main {
    public static void main(String[] args) {
        int i = 10;
        do {
            System.out.print(i + " ");
            i -= 3;
        } while(i > 0);
    }
}
```

Answer

10 7 4 1

Status : Correct

Marks : 1/1

7. What will be the output of the following code?

```
class LoopTest {  
    public static void main(String[] args) {  
        int i = 1;  
        do {  
            System.out.print(i + " ");  
            i *= 2;  
        } while (i <= 8);  
    }  
}
```

Answer

1 2 4 8

Status : Correct

Marks : 1/1

8. What will be the output of the following code?

```
class Test {  
    public static void main(String[] args) {  
        int x = 5, y = 2;  
        if (x + y == 10)  
            System.out.print("Ten");  
        else if (x - y == 3)  
            System.out.print("Three");  
        else  
            System.out.print("None");  
    }  
}
```

Answer

Three

Status : Correct

Marks : 1/1

9. What will be the output of the following code?

```
class Test {  
    public static void main(String[] args) {  
        int a = 4, b = 5;  
        if ((a + b) % 2 == 0)  
            System.out.print("Even");  
        else  
            System.out.print("Odd");  
    }  
}
```

Answer

Odd

Status : Correct

Marks : 1/1

10. What will be the output of the following code?

```
class Test {  
    public static void main(String[] args) {  
        int num = 15;  
        if (num > 10)  
            if (num % 3 == 0)  
                System.out.print("Divisible");  
            else  
                System.out.print("Not Divisible");  
    }  
}
```

Answer

Divisible

Status : Correct

Marks : 1/1

11. What will be the output of the following Java code snippet?

```
public class Main {  
    public static void main(String[] args) {  
        int score = 75;  
        if(score >= 90) {  
            System.out.println("Grade: A");  
        } else if(score >= 80) {  
            System.out.println("Grade: B");  
        } else if(score >= 70) {  
            System.out.println("Grade: C");  
        } else {  
            System.out.println("Grade: D");  
        }  
    }  
}
```

Answer

Grade: C

Status : Correct

Marks : 1/1

12. What will be the output of the following code?

```
public class Main {  
    public static void main(String[] args) {  
        for(int i = 1; i <= 20; i = i * 2) {  
            System.out.print(i + " ");  
        }  
    }  
}
```

Answer

1 2 4 8 16

Status : Correct

Marks : 1/1

13. What will be the output of the following Java code snippet?

```
public class Main {
```

```

public static void main(String[] args) {
    int day = 4;
    String result = "";
    switch(day) {
        case 1:
            result = "Monday";
            break;
        case 2:
            result = "Tuesday";
            break;
        case 3:
            result = "Wednesday";
            break;
        default:
            result = "Other Day";
    }
    System.out.println(result);
}

```

Answer

Other Day

Status : Correct

Marks : 1/1

14. What will be the output of the following code?

```

class Loop {
    public static void main(String[] args) {
        for (int i = 1; i <= 3; i++) {
            for (int j = 1; j <= 2; j++) {
                System.out.print(i + "" + j + " ");
            }
        }
    }
}

```

Answer

11 12 21 22 31 32

Status : Correct

Marks : 1/1

15. What will be the output of the following code?

```
class ConditionTest {  
    public static void main(String[] args) {  
        int a = 7;  
        if (a == 7)  
            System.out.print("Match");  
        else  
            System.out.print("No Match");  
    }  
}
```

Answer

Match

Status : Correct

Marks : 1/1

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 2_Q1

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Arun is working on a project to automate the process of determining whether a student has passed or failed based on their subject marks.

He aims to create a simple program that takes positive integers as marks for five subjects from the user. If the average of the marks is greater than or equal to 50, the student has passed the exam. Otherwise, the student has failed.

Help Arun to implement the project.

Input Format

The input consists of five space-separated integers, representing the marks in five subjects.

Output Format

The first line of output prints "Average score: " followed by an integer representing the average score.

The second line prints one of the following:

1. If the condition is satisfied, print "The student has passed".
2. Otherwise, the output prints "The student has failed".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 50 60 70 80 90

Output: Average score: 70

The student has passed

Answer

```
import java.util.Scanner;
public class Main
{
    public static void main(String[] args)

    {
        Scanner scanner = new Scanner(System.in);

        int mark1 = scanner.nextInt();
        int mark2 = scanner.nextInt();
        int mark3 = scanner.nextInt();
        int mark4 = scanner.nextInt();
        int mark5 = scanner.nextInt();
        int sum = mark1 + mark2 + mark3 + mark4 + mark5;
        int average = sum / 5;
        System.out.println("Average score: " + average);
        if (average >= 50)
        {
            System.out.println("The student has passed");
        }
    }
}
```

```
} else
```

```
{
```

```
    System.out.println("The student has failed");
```

```
}
```

```
    scanner.close();
```

```
}
```

```
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 2_Q2

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Samantha is a diligent math student who is exploring the world of programming. She is learning Java and has recently studied conditional statements. One day, her teacher gives her an interesting problem to solve, which takes a number as input and checks whether it is a multiple of 5 or 7.

Help her complete the task.

Input Format

The input consists of a single integer N, representing the number to be checked.

Output Format

If the number is a multiple of 5 but not 7, the output prints "N is a multiple of 5"

If the number is a multiple of 7, the output prints "N is a multiple of 7".

Otherwise the output prints "N is neither multiple of 5 nor 7" where N is an entered integer.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 10

Output: 10 is a multiple of 5

Answer

```
import java.util.Scanner;
public class Main
{
    public static void main(String[] args)

    {
        Scanner scanner = new Scanner(System.in);
        int N = scanner.nextInt();
        if (N % 5 == 0 && N % 7 != 0)
        {
            System.out.print(N + " is a multiple of 5");
        } else if (N % 7 == 0)
        {
            System.out.print(N + " is a multiple of 7");
        } else
        {
            System.out.print(N + " is neither multiple of 5 nor 7");
        }
        scanner.close();
    }
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 2_Q3

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

John is a fitness trainer, and he wants to use the BMI calculator to assess the body mass index of his clients. He has a list of clients based on their height and weight.

John plans to write a program to quickly determine the BMI and provide a classification for each client.

If BMI is less than 18.5, the program will classify it as "Underweight" If BMI is between 18.6 and 24.9, the program will classify it as "Normal Weight" If BMI is between 25.0 and 29.9, the program will classify it as "Overweight" If BMI is 30.0 or higher, the program will classify it as "Obese"

Note: Formula to calculate BMI = $\text{weight}/(\text{height} \times \text{height})$

Input Format

The first line of input consists of a double value, representing the height of the person in meters.

The second line consists of a double value, representing the weight of the person in kilograms.

Output Format

The first line of output prints "BMI: " followed by a double (rounded to two decimal places) representing the calculated BMI.

The second line prints "Classification: " followed by a string indicating the BMI category (Underweight, Normal Weight, Overweight, or Obese).

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1.2

45.2

Output: BMI: 31.39

Classification: Obese

Answer

```
import java.util.Scanner;
import java.text.DecimalFormat;
public class Main
{
    public static void main(String[] args)
    {
        Scanner scanner = new Scanner(System.in);
        double height = scanner.nextDouble();
        double weight = scanner.nextDouble();
        double bmi = weight / (height * height);
        DecimalFormat df = new DecimalFormat("0.00");
        String formattedBmi = df.format(bmi);
        System.out.println("BMI: " + formattedBmi);
        String classification;
        if (bmi < 18.5)
```

```
{
    classification = "Underweight";
} else if (bmi >= 18.6 && bmi <= 24.9)
{
    classification = "Normal Weight";
} else if (bmi >= 25.0 && bmi <= 29.9)
{
    classification = "Overweight";
} else
{
    classification = "Obese";
}
System.out.println("Classification: " + classification);
scanner.close();
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 2_Q4

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Amit wants to evaluate the depreciation of his car over time to understand its current value and categorize it based on that value.

Write a program that helps him determine the current value of his car after a certain number of years of depreciation and classify it into one of three categories:

High: If the current value is greater than 10,000. Medium: If the current value is between 5,000 and 10,000, both inclusive. Low: If the current value is less than 5,000.

The depreciation rate of the car is 15% per year. The program should calculate the current value of the car after applying this depreciation over the given number of years and print the current value along with the category.

Input Format

The first line of input consists of an integer, representing the initial cost of the car.

The second line consists of an integer, representing the number of years the car has been depreciating.

Output Format

The first line of output prints a double value, representing the current value of the car, rounded off to two decimal places "Current Value: <value>".

The second line prints its category "Category: <categories>".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 20000
5

Output: Current Value: 8874.11
Category: Medium

Answer

```
import java.util.Scanner;
import java.text.DecimalFormat;
public class Main
{
    public static void main(String[] args)
    {
        Scanner scanner = new Scanner(System.in);

        double initialCost = scanner.nextDouble();
        int years = scanner.nextInt();

        double currentValue = initialCost;
        for (int i = 0; i < years; i++)
        {
```

```
        currentValue *= (1 - 0.15); // Apply 15% depreciation
    }
    DecimalFormat df = new DecimalFormat("0.00");
    String formattedCurrentValue = df.format(currentValue);

    System.out.println("Current Value: " + formattedCurrentValue);

    String category;
    if (currentValue > 10000)
    {
        category = "High";
    } else if (currentValue >= 5000 && currentValue <= 10000)
    {
        category = "Medium";
    } else
    {
        category = "Low";
    }
    System.out.println("Category: " + category);
    scanner.close();
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 2_Q5

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Ted, the computer science enthusiast, has accepted the challenge of writing a program that checks if the number of digits in an integer matches the sum of its digits.

Guide Ted in designing and writing the code to solve this problem using a 'do-while' loop.

Input Format

The input consists of an integer N, representing the number to be checked.

Output Format

If the sum is equal to the number of digits, print "The number of digits in N matches the sum of its digits."

Else, print "The number of digits in N does not match the sum of its digits."

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 20

Output: The number of digits in 20 matches the sum of its digits.

Answer

```
import java.util.Scanner;

public class Main
{
    public static void main(String[] args)
    {
        Scanner scanner = new Scanner(System.in);
        int N = scanner.nextInt();
        int originalN = N;
        int sumOfDigits = 0;
        int digitCount = 0;
        if (N == 0)
        {
            digitCount = 1;
        } else
        {
            do
            {
                sumOfDigits += N % 10;
                N /= 10;
                digitCount++;
            } while (N != 0);
        }
        if (sumOfDigits == digitCount)
        {
```

```
        System.out.println("The number of digits in " + originalN + " matches the  
sum of its digits.");
```

```
    } else
```

```
    {
```

```
        System.out.println("The number of digits in " + originalN + " does not  
match the sum of its digits.");
```

```
    }
```

```
        scanner.close();
```

```
    }
```

```
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 2_Q6

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Maya, a student in an arts and crafts class, wants to create a pattern using stars (*) in a specific format. She plans to use a program to help her construct the pattern.

Write a program that takes an integer as input and constructs the following pattern using nested for loops.

Input: 5

Output:

```
*  
* *
```

* * *
* * * *
* * * * *

* * * *

* * *

* *

*

Input Format

The input consists of a number (integer) representing the number of rows.

Output Format

The output displays the required pattern.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5

Output: *

* *
* * *
* * * *
* * * * *
* * * *
* * *
* *
*

*

*

*

*

*

Answer

```
import java.util.Scanner;
```

```
public class Main
```

```
{
```

```
public static void main(String[] args)
{
    Scanner scanner = new Scanner(System.in);
    int rows = scanner.nextInt();
    for (int i = 1; i <= rows; i++)
    {
        for (int j = 1; j <= i; j++)
        {
            System.out.print("* ");
        }
        System.out.println();
    }
    for (int i = rows - 1; i >= 1; i--)
    {
        for (int j = 1; j <= i; j++)
        {
            System.out.print("* ");
        }
        System.out.println();
    }
    scanner.close();
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 2_Q7

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

You are taking part in a coding challenge where your task is to design a program that conjures a mesmerizing numerical pyramid pattern. The enchanting pattern is fashioned using a for loop and is customized based on user input.

Participants are prompted to unveil the pyramid's magic by specifying its height - essentially dictating the number of rows in this spellbinding creation.

Write a program that employs to weave this captivating numerical pyramid as shown below.

Example

Input:

4

Output:

Input Format

The input consists of a positive integer n representing the number of rows in the pattern.

Output Format

The output prints the required pyramid pattern, as shown in the sample output.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

Output: 1

123

12345

1234567

Answer

```
import java.util.Scanner;
public class Main

{
    public static void main(String[] args)

    {
        Scanner scanner = new Scanner(System.in);
        int n = scanner.nextInt();
        for (int i = 1; i <= n; i++)
        {
            for (int j = 1; j <= n - i; j++)
            {
```

```
        System.out.print(" ");  
    }  
    for (int k = 1; k <= (2 * i - 1); k++)  
    {  
        System.out.print(k);  
    }  
    System.out.println();  
}  
scanner.close();  
}  
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 2_Q8

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

A bank generates secure codes using 3-digit numbers where each digit is unique, and the code must be divisible by 3. You are tasked with generating the first N such codes based on user input, ensuring the digits are unique and the number is divisible by 3.

Note: Use nested for loops to solve.

Input Format

The first line contains an integer N representing the number of valid codes to generate.

Output Format

The output prints N lines, each line contains a valid 3-digit code.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5

Output: 102

105

108

120

123

Answer

```
import java.util.Scanner;
public class Main
{
    public static void main(String[] args)
    {
        Scanner scanner = new Scanner(System.in);
        int N = scanner.nextInt();
        int count = 0;
        for (int i = 1; i <= 9; i++)
        {
            for (int j = 0; j <= 9; j++)
            {
                for (int k = 0; k <= 9; k++)
                {
                    if (i != j && i != k && j != k)
                    {
                        if ((i + j + k) % 3 == 0)
                        {
                            System.out.println(i * 100 + j * 10 + k);
                            count++;
                        }
                    }
                }
            }
        }
    }
}
```



```
}
```

```
}
```

```
if (count == N)
```

```
{
```

```
    return;
```

```
}
```

```
}
```

```
}
```

```
}
```

```
}
```

```
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN

Email: 240701805@rajalakshmi.edu.in

Roll no: 240701805

Phone: 6379580366

Branch: REC

Department: CSE - Section 6

Batch: 2028

Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 2_PAH

Attempt : 1

Total Mark : 40

Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Sampad is a high school teacher who wants to convert numeric grades into letter grades.

Write a program that accepts a numeric grade as input. The program should then convert this numeric grade into a letter grade based on specific conditions. The letter grades are A, B, C, D and F.

The conversion is determined by the following conditions:

If the numeric grade is 90 or higher, it's an "A" If the numeric grade is between 80 and 89 (inclusive), it's a "B" If the numeric grade is between 70 and 79 (inclusive), it's a "C" If the numeric grade is between 60 and 69 (inclusive), it's a "D" If the numeric grade is below 60, it's an "F"

Input Format

The input consists of an integer representing the numeric grade of the student.

Output Format

The output prints the letter grade corresponding to the input numeric grade as "Letter Grade: <grade>".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 85

Output: Letter Grade: B

Answer

```
import java.util.Scanner;
public class Main
{
    public static void main(String[] args)
    {
        Scanner scanner = new Scanner(System.in);
        int numericGrade = scanner.nextInt();
        String letterGrade;

        if (numericGrade >= 90)
        {
            letterGrade = "A";
        } else if (numericGrade >= 80)
        {
            letterGrade = "B";
        } else if (numericGrade >= 70)
        {
            letterGrade = "C";
        } else if (numericGrade >= 60)
        {
            letterGrade = "D";
        } else
```

```
{
    letterGrade = "F";
}
System.out.println("Letter Grade: " + letterGrade);
scanner.close();
}
}
```

Status : Correct

Marks : 10/10

2. Problem Statement

You are given a number of distribution centers (rows) and are tasked with generating a zigzag shipment route pattern. Each shipment route should alternate between left-to-right and right-to-left, as described below.

The program should print the zigzag pattern with a tab (\t) separating the columns. For each row, the shipment numbers should follow a diagonal pattern where numbers are placed in a zigzag, left to right on odd rows and right to left on even rows.

Input Format

The input consists of an integer N, which represents the number of distribution centers (rows) for the zigzag pattern.

Output Format

The output prints the zigzag pattern with N rows, formatted with a tab space (\t) separating the columns.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5

Output: 1
 2 6

```
3 7 10
4 8 11 13
5 9 12 14 15
```

Answer

```
import java.util.Scanner;
```

```
public class Main
```

```
{
```

```
    public static void main(String[] args)
```

```
{
```

```
    Scanner sc = new Scanner(System.in);
```

```
    int N = sc.nextInt();
```

```
    sc.close();
```

```
    for (int i = 1; i <= N; i++)
```

```
{
```

```
        // Print leading spaces for alignment
        for (int s = 0; s < (N - i); s++)
```

```
{
```

```
            System.out.print("  ");
```

```
}
```

```
    int currentNum = i; // First number of the row is 'i'
```

```
    int increment = N - 1;
```

```
    for (int j = 0; j < i; j++)
```

```
{  
  
    System.out.printf("%4d  ", currentNum);  
    currentNum += increment;  
    increment--;  
  
}  
  
    System.out.println();  
  
}  
  
}  
  
}
```

Status : Correct

Marks : 10/10

3. Problem Statement

Ravi wants to estimate the total utility bill for a household based on the consumption of electricity, water, and gas.

Write a program to calculate the total bill using the following criteria:

The cost per unit for electricity is 0.12, for water is 0.05, and for gas is 0.08. A discount is applied to the total cost based on the following conditions: If the total cost is 100 or more, a 10% discount is applied. If the total cost is between 50 and 99.99, a 5% discount is applied. No discount is applied if the total cost is less than 50.

The program should output the total bill after applying the discount with two decimal places.

Input Format

The input consists of three double values, representing the number of units consumed for electricity, water, and gas respectively.

Output Format

The output prints a double value, representing the total bill after applying the discount, formatted to two decimal places.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1000.0

200.0

100.0

Output: 124.20

Answer

```
import java.util.Scanner;
```

```
public class Main
```

```
{
```

```
    public static void main(String[] args)
```

```
{
```

```
    Scanner sc = new Scanner(System.in);
```

```
    double electricity = sc.nextDouble();
```

```
    double water = sc.nextDouble();
```

```
    double gas = sc.nextDouble();
```

```
    double cost = electricity * 0.12 + water * 0.05 + gas * 0.08;
```

```
    if (cost >= 100)
```

```
{
```

```
        cost *= 0.9;
```

```
} else if (cost >= 50)
{

    cost *= 0.95;

}

System.out.printf("%.2f\n", cost);

}

}
```

Status : Correct

Marks : 10/10

4. Problem Statement

Rohit is tasked with designing a program to analyze the digits of a given integer.

Write a program to help Rohit that takes an integer as input and identifies the minimum odd digit and the maximum even digit present in the number. If no odd or even digits are present, display appropriate messages.

Implement the solution using a 'while' loop to iterate through the digits of the given number.

Input Format

The input consists of an integer n.

Output Format

The first line of output prints the message "Minimum odd digit: " followed by an integer representing the smallest odd digit found in the input number.

If no odd digit exists, it prints "There are no odd digits in the number."

The second line of output prints the message "Maximum even digit: " followed by an integer representing the largest even digit found in the input number.

If no even digit exists, it prints "There are no even digits in the number."

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3465

Output: Minimum odd digit: 3

Maximum even digit: 6

Answer

```
import java.util.Scanner;

public class Main

{

    public static void main(String[] args)

    {

        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();

        int minOdd = 10;
        int maxEven = -1;

        while (n > 0)

        {

            int digit = n % 10;
            if (digit % 2 == 0)
```

```
{  
    if (digit > maxEven) maxEven = digit;
```

```
} else
```

```
{  
    if (digit < minOdd) minOdd = digit;
```

```
    }  
    n /= 10;
```

```
}
```

```
if (minOdd == 10)
```

```
{
```

```
    System.out.println("There are no odd digits in the number.");
```

```
} else
```

```
{
```

```
    System.out.println("Minimum odd digit: " + minOdd);
```

```
}
```

```
if (maxEven == -1)
```

```
{
```

```
System.out.println("There are no even digits in the number.");
```

```
} else
```

```
{
```

```
System.out.println("Maximum even digit: " + maxEven);
```

```
}
```

```
}
```

```
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 2_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Joe has a favourite number, let's call it X. He wants to check if X is divisible by the sum of its digits. If it is, he considers it a lucky number. If not, he wants to find the closest smaller number, that is divisible by the sum of digits of X. Joe has challenged his friends to solve this puzzle at his birthday party.

Example

Input:

157

Output:

157 is not divisible by the sum of its digits.

The closest smaller number that is divisible: 156

Explanation:

The sum of the digits of X is $1+5+7=13$. Since 157 is not divisible by 13, we need to find the closest smaller number that is divisible by 13. 156 is divisible by 13, it is the closest smaller number that meets the requirement.

Input Format

The input consists of an integer X, representing Joe's favourite number.

Output Format

If X is a lucky number, then the output must be in the format: "X is divisible by the sum of its digits."

If not, then the output must be in the format:

"X is not divisible by the sum of its digits."

The closest smaller number that is divisible: Y",

where X is the entered number and Y is the closest number.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 120

Output: 120 is divisible by the sum of its digits.

Answer

```
import java.util.Scanner;
public class Main
{
    public static int sumDigits(int n)
    {
        int sum = 0;
        int temp = n;
        while (temp != 0)
```

```

    {
        sum += temp % 10;
        temp /= 10;
    }

    return sum;
}

public static void main(String[] args)
{
    Scanner scanner = new Scanner(System.in);
    int X = scanner.nextInt();
    int sum = sumDigits(X);

    if (X % sum == 0)
    {
        System.out.println(X + " is divisible by the sum of its digits.");
    } else
    {
        System.out.println(X + " is not divisible by the sum of its digits.");
        for (int i = X - 1; i > 0; i--)
        {
            if (i % sum == 0)
            {
                System.out.println("The closest smaller number that is divisible: " + i);
                break;
            }
        }
    }

    scanner.close();
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Maya, a student in an arts and crafts class, wants to create a pattern using stars (*) in a specific format. She plans to use a program to help her construct the pattern.

Write a program that takes an integer as input and constructs the following pattern using nested for loops.

Input: 5

Output:

```
*
* *
* * *
* * * *
* * * * *
* * * *
* * * *
* * *
* *
*
```

Input Format

The input consists of a number (integer) representing the number of rows.

Output Format

The output displays the required pattern.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5

Output: *

```
**
***
****
*****
****
***
**
*
```

Answer

```
import java.util.Scanner;
public class Main
{
    public static void main(String[] args)
    {
        Scanner scanner = new Scanner(System.in);
        int rows = scanner.nextInt();

        for (int i = 1; i <= rows; i++)
        {
            for (int j = 1; j <= i; j++)
            {
                System.out.print("* ");
            }
            System.out.println();
        }

        for (int i = rows - 1; i >= 1; i--)
        {
            for (int j = 1; j <= i; j++)
            {
                System.out.print("* ");
            }
            System.out.println();
        }
    }
}
```



```
    scanner.close();  
}  
}
```

Status : Correct

Marks : 10/10

3. Problem Statement

Raj is solving a physics problem involving projectile motion, where he needs to calculate the time a ball hits the ground using a quadratic equation of the form $ax^2 + bx + c = 0$. Depending on the coefficients, the ball may hit the ground once, twice, or not at all in real time.

Help Raj find all real roots of the equation, if any.

Note: discriminant = $b^2 - 4ac$

Input Format

The input consists of three space-separated doubles a, b, and c, representing the coefficients of the quadratic equation.

Output Format

If there are two real roots, print:

- "Two real solutions:"
- "Root1 = <value>"
- "Root2 = <value>"

If there is one real root, print:

- "One real solution:"
- "Root = <value>"

If there are no real roots, print:

- "There are no real solutions."

Note: values are rounded to two decimal places.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1 6 9

Output: One real solution:

Root = -3.00

Answer

```
import java.util.Scanner;
import java.text.DecimalFormat;
public class Main
{
    public static void main(String[] args)
    {
        Scanner scanner = new Scanner(System.in);
        double a = scanner.nextDouble();
        double b = scanner.nextDouble();
        double c = scanner.nextDouble();

        double discriminant = b * b - 4 * a * c;
        DecimalFormat df = new DecimalFormat("0.00");
        if (discriminant > 0)
        {
            double root1 = (-b + Math.sqrt(discriminant)) / (2 * a);
            double root2 = (-b - Math.sqrt(discriminant)) / (2 * a);
            System.out.println("Two real solutions:");
            System.out.println("Root1 = " + df.format(root1));
            System.out.println("Root2 = " + df.format(root2));
        }
        else if (discriminant == 0)
        {
            double root = -b / (2 * a);
```

```

        System.out.println("One real solution:");
        System.out.println("Root = " + df.format(root));
    } else
    {
        System.out.println("There are no real solutions.");
    }
    scanner.close();
}
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Ram wants to evaluate the time required to break even on an investment based on initial costs, monthly profits, and monthly expenses. Write a program to calculate the break-even point in months and categorize the return on investment.

Compute the break-even point by using the formula: $\text{initial cost} / (\text{monthly profit} - \text{monthly expenses})$. Based on the break-even point, classify the return on investment into one of the following categories: Quick Return: If the break-even point is 3 months or fewer. Average Return: If the break-even point is between 4 and 12 months, inclusive. Long-term Return: If the break-even point exceeds 12 months.

Ram is new to programming, so he seeks your assistance in creating the program.

Note: monthly profit is always greater than monthly expenses.

Input Format

The first line of input consists of a double value representing the initial cost.

The second line consists of a double value representing the monthly profit.

The third line consists of a double value representing the monthly expenses.

Output Format

The first line prints "Break-even Point:", followed by the break-even point as a decimal number (of double datatype), formatted to two decimal places.

The second line prints "Category: ", followed by the investment return as a String, which can be one of:

- "Quick Return" if break-even point ≤ 3
- "Average Return" if break-even point ≤ 12
- "Long-term Return" if break-even point > 12

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 10000.50
5000.75
1000.10

Output: Break-even Point: 2.50
Category: Quick Return

Answer

```
import java.util.Scanner;
import java.text.DecimalFormat;
public class Main
{
    public static void main(String[] args)
    {
        Scanner scanner = new Scanner(System.in);
        double initialCost = scanner.nextDouble();
        double monthlyProfit = scanner.nextDouble();
        double monthlyExpenses = scanner.nextDouble();

        double breakEvenPoint = initialCost / (monthlyProfit - monthlyExpenses);
```

```
DecimalFormat df = new DecimalFormat("0.00");
String formattedBreakEvenPoint = df.format(breakEvenPoint);

System.out.println("Break-even Point: " + formattedBreakEvenPoint);

String category;
if (breakEvenPoint <= 3)
{
    category = "Quick Return";
} else if (breakEvenPoint <= 12)
{
    category = "Average Return";
} else
{
    category = "Long-term Return";
}

System.out.println("Category: " + category);

scanner.close();
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

REC_2028_OOPS using Java_Week 3_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 15

Section 1 : MCQ

1. What will be the output of the following code?

```
class M {  
    public static void main(String[] args) {  
        int[][] arr = {  
            {1, 2},  
            {3, 4},  
            {5, 6}  
        };  
  
        for (int i = 0; i < arr.length; i++) {  
            System.out.print(arr[i][0] + " ");  
        }  
    }  
}
```

Answer

1 3 5

Status : Correct

Marks : 1/1

2. What will be the output of the following code?

```
class Q {  
    public static void main(String[] args) {  
        int[][] a = {  
            {1, 2},  
            {3, 4}  
        };  
        for (int i = 0; i < a.length; i++) {  
            for (int j = 0; j < a[0].length; j++) {  
                System.out.print(a[i][j] + " ");  
            }  
        }  
    }  
}
```

Answer

1 2 3 4

Status : Correct

Marks : 1/1

3. What will be the output of the following code?

```
class Q {  
    public static void main(String[] args) {  
        int[] a = {1, 2, 3, 4};  
        for (int i = 0; i < a.length; i++) {  
            if (a[i] % 2 == 0)  
                a[i] = 0;  
        }  
        System.out.println(a[1] + " " + a[3]);  
    }  
}
```

Answer

0 0

Status : Correct

Marks : 1/1

4. What will be the output of the following code?

```
class ReverseArray {  
    public static void main(String[] args) {  
        int[] a = {1, 2, 3, 4};  
        for (int i = 0; i < a.length / 2; i++) {  
            int temp = a[i];  
            a[i] = a[a.length - 1 - i];  
            a[a.length - 1 - i] = temp;  
        }  
        for (int i : a)  
            System.out.print(i + " ");  
    }  
}
```

Answer

4 3 2 1

Status : Correct

Marks : 1/1

5. What will be the output of the following code?

```
class Q {  
    public static void main(String[] args) {  
        int[] nums = {3, 6, 7, 2, 8};  
        int sum = 0;  
        for (int i = 0; i < nums.length; i++) {  
            if (nums[i] % 2 == 0)  
                sum += nums[i];  
        }  
        System.out.println(sum);  
    }  
}
```


Answer

16

Status : Correct

Marks : 1/1

6. What will be the output of the given code?

```
public class Main {  
    public static void main(String[] args) {  
        int[] arr = {1, 2, 3, 4, 5};  
        int n = arr.length;  
        int temp = arr[0];  
  
        for (int i = 0; i < n - 1; i++) {  
            arr[i] = arr[i + 1];  
        }  
        arr[n - 1] = temp;  
  
        for (int num : arr) {  
            System.out.print(num + " ");  
        }  
    }  
}
```

Answer

2 3 4 5 1

Status : Correct

Marks : 1/1

7. What will be the output of the following code?

```
class Sample {  
    public static void main(String[] args) {  
        int[][] data = {  
            {1, 2},  
            {3, 4}  
        };  
        int sum = 0;
```

```

        for (int[] row : data) {
            for (int val : row) {
                sum += val;
            }
        }

        System.out.println("Sum = " + sum);
    }
}

```

Answer

Sum = 10

Status : Correct

Marks : 1/1

8. What will be the output of the following code?

```

class Q {
    public static void main(String[] args) {
        int[][] a = {
            {1, 2},
            {3, 4}
        };
        int sum = 0;
        for (int i = 0; i < a.length; i++)
            for (int j = 0; j < a[0].length; j++)
                sum += a[i][j];
        System.out.println(sum);
    }
}

```

Answer

10

Status : Correct

Marks : 1/1

9. What will be the output of the following code?

```
class Sample {  
    public static void main(String[] args) {  
        int[][] matrix = {  
            {1, 2, 3},  
            {4, 5, 6}  
        };  
        System.out.println(matrix[1][2]);  
    }  
}
```

Answer

6

Status : Correct

Marks : 1/1

10. What will be the output of the following code?

```
public class Test {  
    public static void main(String[] args) {  
        int[] x = {4, 8, 12};  
        int result = x[0] * x[2];  
        System.out.println(result);  
    }  
}
```

Answer

48

Status : Correct

Marks : 1/1

11. What will be the output of the following code?

```
class Q {  
    public static void main(String[] args) {  
        int[] a = {1, 2, 1, 3, 1, 4};  
        int count = 0;  
        for (int i = 0; i < a.length; i++) {  
            if (a[i] == 1) count++;  
        }  
    }  
}
```

```
}  
    System.out.println(count);  
}  
}
```

Answer

3

Status : Correct

Marks : 1/1

12. What will be the output of the following code?

```
class Q {  
    public static void main(String[] args) {  
        int[][] arr = {  
            {5, 6, 7},  
            {8, 9, 10}  
        };  
        System.out.println(arr[0][2]);  
    }  
}
```

Answer

7

Status : Correct

Marks : 1/1

13. What will be the output of the following code?

```
class Sample {  
    public static void main(String[] args) {  
        int[] a = {1, 2, 3};  
        int product = 1;  
        for (int i = 0; i < a.length; i++) {  
            product *= a[i];  
        }  
        System.out.println(product);  
    }  
}
```

Answer

6

Status : Correct

Marks : 1/1

14. What will be the output of the following code?

```
class Q {  
    public static void main(String[] args) {  
        int[] nums = {4, 2, 9, 5};  
        int max = nums[0];  
        for (int i = 1; i < nums.length; i++) {  
            if (nums[i] > max)  
                max = nums[i];  
        }  
        System.out.println(max);  
    }  
}
```

Answer

9

Status : Correct

Marks : 1/1

15. What will be the output of the following code?

```
class Q {  
    public static void main(String[] args) {  
        int[] a = {1, 2, 3, 4};  
        for (int i = 0; i < a.length / 2; i++) {  
            int temp = a[i];  
            a[i] = a[a.length - 1 - i];  
            a[a.length - 1 - i] = temp;  
        }  
        System.out.println(a[0]);  
    }  
}
```

Answer

4

Status : Correct

Marks : 1/1

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 3_Q1

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Rosh is intrigued by numerical patterns. Today, she stumbled upon a puzzle while working with arrays. She wants to compute the sum of the third-largest and second-smallest elements from a list of integers. She seeks your help to implement a program that solves this for her efficiently.

Input Format

The first line of input is an integer N, representing the size of the array.

The second line of input consists of N space-separated integers, representing the elements of the array.

Output Format

The output displays a single integer representing the sum of the third-largest and second-smallest elements in the array.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 10

10 20 30 40 50 60 70 80 90 100

Output: 100

Answer

```
import java.util.Arrays;
import java.util.Scanner;

public class Main
{
    public static void main(String[] args)
    {
        Scanner scanner = new Scanner(System.in);
        int n = scanner.nextInt();
        int[] arr = new int[n];
        for (int i = 0; i < n; i++)
        {
            arr[i] = scanner.nextInt();
        }
        Arrays.sort(arr);

        int secondSmallest = arr[1];
        int thirdLargest = arr[n - 3];

        System.out.println(secondSmallest + thirdLargest);

        scanner.close();
    }
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 3_Q2

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Monica is interested in finding a treasure but the key to opening is to get the sum of the main diagonal elements and secondary diagonal elements.

Write a program to help Monica find the diagonal sum of a square 2D array.

Note: The main diagonal of the array consists of the elements traversing from the top-left corner to the bottom-right corner. The secondary diagonal includes elements from the top-right corner to the bottom-left corner.

Input Format

The first line of input consists of an integer N, representing the number of rows and columns.

The following N lines consist of N space-separated integers, representing the 2D array elements.

Output Format

The first line of output prints "Sum of the main diagonal: " followed by an integer, representing the sum of the main diagonal.

The second line prints "Sum of the secondary diagonal: " followed by an integer, representing the sum of the secondary diagonal.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3

1 2 3

4 5 6

7 8 9

Output: Sum of the main diagonal: 15

Sum of the secondary diagonal: 15

Answer

```
import java.util.Scanner;
```

```
public class Main
```

```
{
```

```
    public static void main(String[] args)
```

```
{
```

```
        Scanner scanner = new Scanner(System.in);
```

```
        int n = scanner.nextInt();
```

```
        int[][] matrix = new int[n][n];
```

```
        for (int i = 0; i < n; i++)
```

```
{
```

```
            for (int j = 0; j < n; j++)
```

```
{  
    matrix[i][j] = scanner.nextInt();  
}  
  
}  
  
int mainDiagonalSum = 0;  
int secondaryDiagonalSum = 0;  
  
for (int i = 0; i < n; i++)  
{  
    mainDiagonalSum += matrix[i][i];  
    secondaryDiagonalSum += matrix[i][n - 1 - i];  
}  
  
System.out.println("Sum of the main diagonal: " + mainDiagonalSum);  
System.out.println("Sum of the secondary diagonal: " +  
secondaryDiagonalSum);  
  
scanner.close();  
}  
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 3_Q3

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

You are developing a warehouse management system for a shipping company. The system uses an integer array to represent the weights of packages in a specific order. To verify that the weight capacity is not exceeded, the program needs to calculate the sum of the weights of the first and last packages in the list.

Task:

Write a code to calculate the sum of the weights of the first and last packages in the list. The program should take an integer array as input and return the total weight of the first and last packages.

Input Format

The first line of the input is an integer N representing the size of the array.

The second line of the input is N space-separated integer values.

Output Format

The output is displayed in the following format:

"Sum of the first and last elements: <<Sum>>"

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5

10 20 30 40 50

Output: Sum of the first and last elements: 60

Answer

```
import java.util.Scanner;
```

```
public class Main
```

```
{
```

```
    public static void main(String[] args)
```

```
{
```

```
    Scanner scanner = new Scanner(System.in);
```

```
    int n = scanner.nextInt();
```

```
    int[] weights = new int[n];
```

```
    for (int i = 0; i < n; i++)
```

```
{
```

```
        weights[i] = scanner.nextInt();
```

```
}  
    int sum = weights[0] + weights[n - 1];  
    System.out.println("Sum of the first and last elements: " + sum);  
    scanner.close();
```

```
}
```

```
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 3_Q4

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Sesha is developing a weather monitoring system for a region with multiple weather stations. Each weather station collects temperature data hourly and stores it in a 2D array.

Write a program that can add the temperature data from two different weather stations to create a combined temperature record for the region.

Input Format

The first line of input consists of two space-separated integers N and M, representing the number of rows and columns of the matrices, respectively.

The next N lines consist of M space-separated integers, representing the values of the first matrix.

The following N lines consist of M space-separated integers, representing the values of the second matrix.

Output Format

The output prints the addition of the two matrices in N rows and M columns, representing the combined temperature record.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3 3

1 2 3

4 5 6

7 8 9

1 1 1

2 2 2

3 3 3

Output: 2 3 4

6 7 8

10 11 12

Answer

```
import java.util.Scanner;
```

```
public class Main
```

```
{
```

```
    public static void main(String[] args)
```

```
{
```

```
    Scanner scanner = new Scanner(System.in);
```

```
    int n = scanner.nextInt();
```

```
    int m = scanner.nextInt();
```

```
    int[][] matrix1 = new int[n][m];
```



```
for (int i = 0; i < n; i++)  
{  
  
    for (int j = 0; j < m; j++)  
  
    {  
  
        matrix1[i][j] = scanner.nextInt();  
  
    }  
}
```

```
int[][] matrix2 = new int[n][m];  
for (int i = 0; i < n; i++)  
  
{  
  
    for (int j = 0; j < m; j++)  
  
    {  
  
        matrix2[i][j] = scanner.nextInt();  
  
    }  
  
}
```

```
int[][] resultMatrix = new int[n][m];  
for (int i = 0; i < n; i++)  
  
{
```

```
        for (int j = 0; j < m; j++)
        {

            resultMatrix[i][j] = matrix1[i][j] + matrix2[i][j];

        }

    }

    for (int i = 0; i < n; i++)
    {

        for (int j = 0; j < m; j++)

        {

            System.out.print(resultMatrix[i][j] + " ");

        }

        System.out.println();

    }

    scanner.close();

}

}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 3_Q5

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Sharon is creating a program that finds the first repeated element in an integer array. The program should efficiently identify the first element that appears more than once in the given array. If no such element is found, it should appropriately display a message.

Help Sharon to complete the program.

Input Format

The first line of input consists of an integer n , representing the number of elements in the array.

The second line consists of n space-separated integers, representing the array elements.

Output Format

If a repeated element is found, print the first element that appears more than once.

If no repeated element is found, print "No repeated element found in the array".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 8

12 21 13 14 21 36 47 21

Output: 21

Answer

```
import java.util.HashSet;
import java.util.Scanner;
import java.util.Set;
```

```
public class Main
```

```
{
```

```
    public static void main(String[] args)
```

```
{
```

```
    Scanner scanner = new Scanner(System.in);
```

```
    int n = scanner.nextInt();
```

```
    Set<Integer> seenElements = new HashSet<>();
```

```
    boolean found = false;
    for (int i = 0; i < n; i++)
```

```
{  
    int currentElement = scanner.nextInt();  
    if (seenElements.contains(currentElement))  
  
    {  
  
        System.out.println(currentElement);  
        found = true;  
        break;  
    }  
    seenElements.add(currentElement);  
  
}  
  
if (!found)  
{  
  
    System.out.println("No repeated element found in the array");  
}  
  
scanner.close();  
  
}  
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

REC_2028_OOPS using Java_Week 3_PAH

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Egath is participating in a coding hackathon, and one of the challenges requires him to work with an array of integers. The task is to remove exactly one element from the array such that the sum of the remaining elements is a prime number.

Help Egath find the first possible prime sum by removing one element or determining if no such prime sum can be achieved.

Input Format

The first line of input consists of an integer N, representing the number of elements in the array.

The second line consists of N space-separated integers, representing the array elements.

Output Format

If removing one element results in a prime sum, print the sum.

If no such prime sum can be achieved by removing exactly one element, print "No valid prime sum found".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3

1 2 3

Output: 5

Answer

```
import java.util.Scanner;

public class Main
{
    public static boolean isPrime(int num)
    {
        if (num <= 1)
        {
            return false;
        }
        for (int i = 2; i <= Math.sqrt(num); i++)
        {
            if (num % i == 0)
            {
                return false;
            }
        }
        return true;
    }
}
```

```

public static void main(String[] args)
{
    Scanner scanner = new Scanner(System.in);

    int n = scanner.nextInt();

    int[] arr = new int[n];

    int totalSum = 0;
    for (int i = 0; i < n; i++)
    {
        arr[i] = scanner.nextInt();
        totalSum += arr[i];
    }
    for (int i = 0; i < n; i++)
    {
        int currentSum = totalSum - arr[i];

        if (isPrime(currentSum))
        {
            System.out.println(currentSum);
            scanner.close();
            return;
        }
    }

    System.out.println("No valid prime sum found");
    scanner.close();
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

In a customer loyalty program, reward points are logged in a sorted array as customers make transactions. Occasionally, due to system errors, duplicate entries for the same transaction may appear. To ensure accurate reward calculations, it's crucial to remove these duplicates from the list.

Write a program to process the array of reward points, removing any duplicates while preserving the order of unique entries. The program should then display the cleaned list of unique reward points and the total count of these unique points.

Input Format

The first line of input consists of an integer N, representing the number of reward points.

The second line consists of N space-separated integers, representing the reward points in sorted order.

Output Format

The first line of output prints the cleaned list of unique reward points separated by a space.

The second line of output prints an integer representing the total count of unique reward points.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3
100 100 200

Output: 100 200
2

Answer

```
import java.util.Scanner;
public class Main

{

    public static void main(String[] args)
    {
        Scanner scanner = new Scanner(System.in);
```

```

int n = scanner.nextInt();
if (n == 0)
{
    System.out.println("");
    System.out.println(0);
    scanner.close();
    return;
}
int[] arr = new int[n];

for (int i = 0; i < n; i++)
{
    arr[i] = scanner.nextInt();
}

System.out.print(arr[0] + " ");
int uniqueCount = 1;

for (int i = 1; i < n; i++)
{
    if (arr[i] != arr[i - 1])
    {
        System.out.print(arr[i] + " ");
        uniqueCount++;
    }
}

System.out.println();
System.out.println(uniqueCount);

scanner.close();
}
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Eminem is a billiard player who enjoys playing billiards and also likes solving mathematical puzzles. He notices that the billiard balls on the table are arranged in a grid, and he is curious to find the sum of the numbers written on each ball.

Write a program to find the sum of all the numbers written on each ball in the grid.

Input Format

The first line of input consists of an integer N, representing the number of rows.

The second line consists of an integer M, representing the number of columns.

The following lines N lines consist of M space-separated integers, representing the numbers written on each ball.

Output Format

The output prints an integer representing the sum of all the numbers written on each ball.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3

3

1 2 3

4 5 6

7 8 9

Output: 45

Answer

```
import java.util.Scanner;
```

```
public class Main
```

```
{  
  
    public static void main(String[] args)  
    {  
  
        Scanner scanner = new Scanner(System.in);  
  
        int n = scanner.nextInt();  
        int m = scanner.nextInt();  
        int[][] grid = new int[n][m];  
  
        int sum = 0;  
  
        for (int i = 0; i < n; i++)  
        {  
  
            for (int j = 0; j < m; j++)  
            {  
                grid[i][j] = scanner.nextInt();  
                sum += grid[i][j];  
            }  
        }  
  
        System.out.println(sum);  
  
        scanner.close();  
  
    }  
}
```

Status : Correct

Marks : 10/10

4. Problem Statement

Priya is building a system to automate image transformations using matrix operations. To do this, she needs to multiply two matrices representing pixel data and transformation rules.

Help Priya perform matrix multiplication and print the resulting matrix if the operation is valid.

Input Format

The first line of input consists of two int values, representing the number of rows R1 and columns C1 of the first matrix.

The next $R1 \times C1$ integers represent the elements of the first matrix.

The next line consists of two int values, representing the number of rows R2 and columns C2 of the second matrix.

The next $R2 \times C2$ integers represent the elements of the second matrix.

Output Format

If matrix multiplication is possible, print R1 lines, each containing C2 space-separated int values representing the resulting matrix.

Otherwise, print "Matrix multiplication not possible".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 2 3

1 2 3

4 5 6

3 2

7 8

9 10

11 12

Output: 58 64

139 154

Answer

```
import java.util.Scanner;  
public class Main
```

```
{  
    public static void main(String[] args)  
    {  
  
        Scanner scanner = new Scanner(System.in);  
        int r1 = scanner.nextInt();  
        int c1 = scanner.nextInt();  
        int[][] matrix1 = new int[r1][c1];  
        for (int i = 0; i < r1; i++)  
        {  
            for (int j = 0; j < c1; j++)  
            {  
                matrix1[i][j] = scanner.nextInt();  
            }  
        }  
  
        int r2 = scanner.nextInt();  
        int c2 = scanner.nextInt();  
        int[][] matrix2 = new int[r2][c2];  
        for (int i = 0; i < r2; i++)  
        {  
            for (int j = 0; j < c2; j++)  
            {  
                matrix2[i][j] = scanner.nextInt();  
            }  
        }  
  
        if (c1 != r2)  
        {  
            System.out.println("Matrix multiplication not possible");  
        } else
```

```

{
    int[][] resultMatrix = new int[r1][c2];
    for (int i = 0; i < r1; i++)
    {
        for (int j = 0; j < c2; j++)
        {
            int sum = 0;
            for (int k = 0; k < c1; k++)
            {
                sum += matrix1[i][k] * matrix2[k][j];
            }
            resultMatrix[i][j] = sum;
        }
    }

    for (int i = 0; i < r1; i++)
    {
        for (int j = 0; j < c2; j++)
        {
            System.out.print(resultMatrix[i][j] + " ");
        }
        System.out.println();
    }
}

scanner.close();

}

}

```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

REC_2028_OOPS using Java_Week 3_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Alex is a treasure hunter who collects valuable items during their quests. Each item has a specific point value, and Alex wants to maximize their score by strategically removing items one at a time.

The rule is simple: Alex removes the item with the highest point value in each step until no items are left, summing the values of the removed items to calculate the maximum score.

Help Alex to complete his task.

Input Format

The first line of input consists of an integer N, representing the size of the array.

The second line of input consists of N space-separated integers, representing the point values of the items.

Output Format

The output prints "Maximum Sum: " followed by the calculated maximum score after removing all items.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 14

7 14 21 28 35 42 49 56 63 70 77 84 91 98

Output: Maximum Sum: 735

Answer

```
import java.util.Scanner;
```

```
public class Main
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        Scanner scanner = new Scanner(System.in);
```

```
        int n = scanner.nextInt();
```

```
        long sum = 0;
```

```
        for (int i = 0; i < n; i++)
```

```
        {
```

```
        sum += scanner.nextInt();
    }

    System.out.println("Maximum Sum: " + sum);

    scanner.close();

}

}
```

Status : Correct

Marks : 10/10

2. Problem Statement:

Emma, a budding computer vision enthusiast, is working on a challenging image processing project. She has a square image represented as a 2D matrix of integers. As part of a special filter operation, she needs to rotate the image by 90 degrees clockwise, but there's a twist – she must perform the rotation in-place, using no extra space.

This means Emma has to rotate the matrix without creating a new one. Your task is to help her implement a Java program that takes this square matrix as input and rotates it within the same structure.

Can you help Emma efficiently rotate the image so that her project can move to the next stage?

Input Format

The first line of input contains a single integer n , representing the number of rows and columns of the square matrix (i.e., the matrix is of size $n \times n$).

The next n lines each contain n space-separated integers, representing the elements of each row of the 2D array.

Output Format

The first line of output prints "Rotated 2D Array:"

The next n lines of output print the rotated matrix.

Each line contains n space-separated integers representing a row of the rotated matrix.

Refer to the sample output for format specification.

Sample Test Case

Input: 3

1 2 3

4 5 6

7 8 9

Output: Rotated 2D Array:

7 4 1

8 5 2

9 6 3

Answer

```
import java.util.Scanner;
```

```
public class Main
```

```
{
```

```
    public static void main(String[] args)
```

```
{
```

```
    Scanner scanner = new Scanner(System.in);
```

```
    int n = scanner.nextInt();
```

```
    int[][] matrix = new int[n][n];
```

```
    for (int i = 0; i < n; i++)
```

```
{
```

```
        for (int j = 0; j < n; j++)
```

```
{
```

```
matrix[i][j] = scanner.nextInt();
```

```
}
```

```
for (int i = 0; i < n; i++)
```

```
{
```

```
    for (int j = i; j < n; j++)
```

```
{
```

```
    int temp = matrix[i][j];
```

```
    matrix[i][j] = matrix[j][i];
```

```
    matrix[j][i] = temp;
```

```
}
```

```
}
```

```
for (int i = 0; i < n; i++)
```

```
{
```

```
    int left = 0;
```

```
    int right = n - 1;
```

```
    while (left < right)
```

```
{
```

```
    int temp = matrix[i][left];
```

```
    matrix[i][left] = matrix[i][right];
```

```
    matrix[i][right] = temp;
```

```
    left++;
```

```
    right--;
```

```
}
```

```

    }
    System.out.println("Rotated 2D Array:");
    for (int i = 0; i < n; i++)
    {

        for (int j = 0; j < n; j++)
        {

            System.out.print(matrix[i][j] + " ");

        }
        System.out.println();
    }

    scanner.close();
}
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Nikila is working as an intern in a software firm and is practicing with a matrix where each row represents a set of numerical values. Her task is to identify the row with the highest sum of its elements and remove that row from the matrix. After removing the row with the highest sum, Nikila needs to print the updated matrix.

Your task is to help Nikila in implementing the same. If there are two or more rows that have same the highest sum, the firstly encountered row is deleted.

Input Format

The first line of the input consists of two space-separated integers, R and C, representing the number of rows and columns in the matrix, respectively.

The following R lines each contain, C space-separated integers representing the

matrix elements.

Output Format

The output prints the matrix after removing the row with the highest sum. Each row should be printed on a new line, with elements separated by a space.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 2 2

1 2

3 4

Output: 1 2

Answer

```
import java.util.Scanner;
```

```
public class Main
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        Scanner scanner = new Scanner(System.in);
```

```
        int r = scanner.nextInt();
```

```
        int c = scanner.nextInt();
```

```
        int[][] matrix = new int[r][c];
```

```
        for (int i = 0; i < r; i++)
```

```
        {
```

```
for (int j = 0; j < c; j++)
```

```
{
```

```
    matrix[i][j] = scanner.nextInt();
```

```
}
```

```
}
```

```
int maxSum = -1;
```

```
int rowIndexToRemove = -1;
```

```
for (int i = 0; i < r; i++)
```

```
{
```

```
    int currentRowSum = 0;
```

```
    for (int j = 0; j < c; j++)
```

```
{
```

```
        currentRowSum += matrix[i][j];
```

```
}
```

```
    if (currentRowSum > maxSum)
```

```
{
```

```
        maxSum = currentRowSum;
```

```
        rowIndexToRemove = i;
```

```
}
```

```
}
```

```
    for (int i = 0; i < r; i++)
```

```
    {
```

```
        if (i == rowIndexToRemove)
```

```
        {
```

```
            continue;
```

```
        }
```

```
        for (int j = 0; j < c; j++)
```

```
        {
```

```
            System.out.print(matrix[i][j] + " ");
```

```
        }
```

```
        System.out.println();
```

```
    }
```

```
    scanner.close();
```

```
}
```

```
}
```

Status : Correct

Marks : 10/10

4. Problem Statement:

Mason is participating in a coding challenge where he must manipulate an integer array. His task is to replace every element in the array with the next greatest element to its right. The last element of the array remains unchanged, as there is no element to its right.

Your job is to help Mason write a program that performs this transformation and outputs the modified array.

Input Format

The first line of input contains an integer n representing the number of elements in the array.

The second line of input contains n space-separated integers representing the elements of the array.

Output Format

The output prints the modified array of n integers, where each element (except the last one) is replaced by the maximum element to its right, and the last element remains unchanged.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6

12 3 91 15 12 14

Output: 91 91 15 14 14 14

Answer

```
import java.util.Scanner;
```

```
public class Main
```

```
{
```

```
public static void main(String[] args)
```

```
{
```

```
    Scanner scanner = new Scanner(System.in);
```

```
    int n = scanner.nextInt();
```

```
    int[] arr = new int[n];
```

```
    for (int i = 0; i < n; i++)
```

```
    {
```

```
        arr[i] = scanner.nextInt();
```

```
    }
```

```
    if (n > 0)
```

```
    {
```

```
        int maxFromRight = arr[n - 1];
```

```
        for (int i = n - 2; i >= 0; i--)
```

```
        {
```

```
            int temp = arr[i];
```

```
            arr[i] = maxFromRight;
```

```
            if (temp > maxFromRight)
```

```
            {
```

```
                maxFromRight = temp;
```

```
            }
```

```
}
```

```
}
```

```
    for (int i = 0; i < n; i++)
```

```
{
```

```
    System.out.print(arr[i] + " ");
```

```
}
```

```
    System.out.println();  
    scanner.close();
```

```
}
```

```
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

REC_2028_OOPS using Java_Week 4_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 13

Section 1 : MCQ

1. What will be the output of the following program?

```
class Main {  
    public static void main(String args[]) {  
        String name="Work Hard";  
        name.concat("Success");  
        System.out.println(name);  
    }  
}
```

Answer

Work HardSuccess

Status : Wrong

Marks : 0/1

2. Predict the output for the following code.

```
public class Main {  
    public static void main(String[] args) {  
        String a = "java";  
        char temp = a.charAt(1);  
        System.out.println(temp);  
    }  
}
```

Answer

a

Status : Correct

Marks : 1/1

3. What will be the output of the following code?

```
class Main {  
    public static void main(String args[]) {  
        char c[] = {'j', 'a', 'v', 'a'};  
        String s1 = new String(c);  
        String s2 = new String(s1);  
        System.out.println(s1);  
        System.out.println(s2);  
    }  
}
```

Answer

javajava

Status : Correct

Marks : 1/1

4. Predict the output for the following code.

```
class Main {  
    public static void main(String[] fruits) {  
        String fruit1 = new String("apple");  
        String fruit2 = new String("orange");  
        String fruit3 = new String("pear");  
    }  
}
```

```
fruit3 = fruit1;
fruit2 = fruit3;
fruit1 = fruit2;
System.out.println(fruit1);
System.out.println(fruit2);
System.out.println(fruit3);
    }
}
```

Answer

appleappleapple

Status : Correct

Marks : 1/1

5. What will be the output of the following program?

```
class Main {
    public static void main(String[] args) {
        String s1 = "EDUCATION";
        String s2 = new String("EDUCATION");
        String s3 = "EDUCATION";
        if (s1 == s2) {
            System.out.println("s1 and s2 equal");
        }
        else {
            System.out.println("s1 and s2 not equal");
        }
        if (s1 == s3) {
            System.out.println("s1 and s3 equal");
        }
        else {
            System.out.println("s1 and s3 not equal");
        }
    }
}
```

Answer

s1 and s2 not equal
s1 and s3 equal

Status : Correct

Marks : 1/1

6. What is the output of the following code?

```
class Main
{
    public static void main(String args[])
    {
        StringBuffer c = new StringBuffer("Hello");
        c.delete(0,2);
        System.out.println(c);
    }
}
```

Answer

llo

Status : Correct

Marks : 1/1

7. What will be the output of the following program?

```
public class Main {
    public static void main(String[] args) {
        String str = "1234.34";
        int a = Integer.parseInt(str);
        System.out.println(a);
    }
}
```

Answer

NumberFormatException

Status : Correct

Marks : 1/1

8. Predict the output for the following code:

```
class Main {
    public static void main(String args[]) {
        StringBuffer sb = new StringBuffer("I Java!");
        sb.insert(5, "like ");
        System.out.println(sb);
    }
}
```

Answer

I Javlike a!

Status : Correct

Marks : 1/1

9. What will be the output for the following code?

```
class Main {  
    public static void main(String[] args) {  
        String languages[] = { "C", "C++", "Java", "Python", "Ruby"};  
        for (String sample: languages) {  
            System.out.println(sample);  
        }  
    }  
}
```

Answer

CC++JavaPythonRuby

Status : Correct

Marks : 1/1

10. What will be the output of the following program?

```
class Main {  
    public static void main(String[] args) {  
        String s = new String("5");  
        System.out.println(1 + 1111 + s + 1 + 1010);  
    }  
}
```

Answer

1112511010

Status : Correct

Marks : 1/1

11. What will be the output of the following program?


```

class Main {
    public static void main(String args[]) {
        StringBuffer sb = new StringBuffer("Hello");
        System.out.println("buffer = " + sb);
        System.out.println("length = " + sb.length());
        System.out.println("capacity = " + sb.capacity());
    }
}

```

Answer

buffer = Hello length = 5 capacity = 21

Status : Wrong

Marks : 0/1

12. Predict the output for the following code:

```

public class Main {
    public static void main(String[] args) {
        float a = 10.0f;
        String temp = Float.toString(a);
        System.out.println(temp);
    }
}

```

Answer

10.0

Status : Correct

Marks : 1/1

13. What will be the output of the following code?

```

class Main {
    public static void main(String args[]) {
        String s1 = "Hello i love java";
        String s2 = new String(s1);
        System.out.println((s1 == s2) + " " + s1.equals(s2));
    }
}

```

Answer

false true

Status : Correct

Marks : 1/1

14. What will be the output of the following code?

```
class Main {  
    public static void main(String args[])  
    {  
        StringBuffer sb = new StringBuffer("Hello");  
        System.out.println("buffer before = " + sb);  
        System.out.println("charAt(1) before = " + sb.charAt(1));  
        sb.setCharAt(1, 'i');  
        sb.setLength(2);  
        System.out.println("buffer after = " + sb);  
        System.out.println("charAt(1) after = " + sb.charAt(1));  
    }  
}
```

Answer

buffer before = Hello
charAt(1) before = e
buffer after = Hi
charAt(1) after = i

Status : Correct

Marks : 1/1

15. What will be the output of the following program?

```
class Main {  
    public static void main(String[] args) {  
        String greet = "Welcome\n";  
        System.out.print("String: " + greet);  
        int length = greet.length();  
        System.out.print("Length: " + length);  
    }  
}
```

Answer

String: Welcome
Length: 8

Status : Correct

Marks : 1/1

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 4_Q1

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

In a publishing company, editors often need to quickly analyze passages of text to check for punctuation usage. To assist them, you are asked to write a program that counts the number of specific punctuation marks in each passage.

The punctuation marks of interest are:

Commas (,) Periods (.) Question marks (?)

Input Format

The first line of input contains an integer T, representing the number of test cases (passages).

Each of the next T lines contains a single passage of text.

Output Format

For each test case, print three integers separated by spaces, representing the number of commas, periods, and question marks in the passage.

The first line of output corresponds to the first passage, the second line to the second passage, and so on.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1

Hello, world. How are you?

Output: 1 1 1

Answer

```
import java.util.Scanner;

public class Main

{
    public static void main(String[] args)
    {
        Scanner scanner = new Scanner(System.in);

        try
        {

            if (!scanner.hasNextInt())
            {
                return;
            }

            int T = scanner.nextInt();
            scanner.nextLine();
            for (int i = 0; i < T; i++)
            {
                String passage = scanner.nextLine();
                int commas = 0;
                int periods = 0;
```

```

        int questionMarks = 0;
        for (char c : passage.toCharArray())
        {
            if (c == ',')
            {
                commas++;
            } else if (c == '.')
            {
                periods++;
            } else if (c == '?')
            {
                questionMarks++;
            }
        }
        System.out.println(commas + " " + periods + " " + questionMarks);
    }

} catch (Exception e)
{
    System.err.println("An error occurred while processing input: " +
e.getMessage());
} finally
{
    scanner.close();
}

}

}

```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 4_Q2

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Anu is developing a tool for a conference registration system. Participants submit keywords related to their fields of interest. The organizer wants to sort these keywords alphabetically to generate tags for session grouping.

Write a program that accepts at least five keywords as input arguments and outputs them in sorted alphabetical order.

Input Format

The first line of input contains an integer n, representing the number of keywords.

The second line of input contains n space-separated keywords (string).

Output Format

The output prints n space separated strings representing the sorted keyword in alphabetical order.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5

Blockchain Cloud AI Data Cybersecurity

Output: AI Blockchain Cloud Cybersecurity Data

Answer

```
import java.util.Arrays;
import java.util.Scanner;

public class Main

{
    public static void main(String[] args)

    {
        Scanner scanner = new Scanner(System.in);

        try
        {
            if (!scanner.hasNextInt())
            {
                return;
            }

            int n = scanner.nextInt();

            scanner.nextLine();

            String line = scanner.nextLine();

            String[] keywords = line.split(" ");

            Arrays.sort(keywords);
```



```
for (int i = 0; i < keywords.length; i++)
{
    System.out.print(keywords[i]);
    if (i < keywords.length - 1)
    {
        System.out.print(" ");
    }
}

System.out.println();
} catch (Exception e)
{
    System.err.println("An error occurred: " + e.getMessage());
} finally
{
    if (scanner != null)
    {
        scanner.close();
    }
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 4_Q3

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Bechan Chacha is seeking help to filter out valid mobile numbers from a list provided by his crush. He can only pick his crush's number if the list contains valid mobile numbers.

A mobile number is considered valid if:

It has exactly 10 digits. It consists only of numeric values (0–9). It does not begin with zero.

Your task is to determine whether each mobile number in the list is valid or not.

Input Format

The first line contains an integer T, representing the number of mobile numbers

to check.

The next T lines each contain a string S, representing a mobile number.

Output Format

For each mobile number S, the output print "YES" if it is valid.

Otherwise, print "NO".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1
9876543210

Output: YES

Answer

```
import java.util.*;
```

```
public class Main
```

```
{
```

```
    public static void main(String[] args)
```

```
{
```

```
    Scanner sc = new Scanner(System.in);  
    int T = sc.nextInt();  
    sc.nextLine();
```

```
for (int i = 0; i < T; i++)
{

    String number = sc.nextLine();
    if (isValidMobileNumber(number))
    {

        System.out.println("YES");

    } else
    {

        System.out.println("NO");

    }

    sc.close();

}

private static boolean isValidMobileNumber(String number)
{
```

```
if (number.length() != 10) return false;
if (number.charAt(0) == '0') return false;
for (int i = 0; i < number.length(); i++)

{

    if (!Character.isDigit(number.charAt(i)))

    {

        return false;

    }

}

return true;

}

}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 4_Q4

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Arjun is learning how to filter words from a sentence based on grammar rules. He wants to identify the valid words in a sentence.

A word is considered valid if it satisfies all these conditions:

The word contains only alphabets (a-z, A-Z). The word length is at least 2 characters. The word should not contain digits or special characters.

Your task is to read a sentence and print all the valid words in it.

Input Format

The input contains a single line containing a sentence S.

Output Format

The output prints all the valid words separated by spaces.

If no valid word exists, print "No valid words."

Refer to the sample output for formatting specifications.

Sample Test Case

Input: Hello world1 123 ab" @\$ Hi

Output: Hello Hi

Answer

```
import java.util.*;
```

```
public class Main
```

```
{
```

```
    public static void main(String[] args)
```

```
{
```

```
    Scanner sc = new Scanner(System.in);
```

```
    String sentence = sc.nextLine();
```

```
    String[] words = sentence.split(" ");
```

```
    boolean found = false;
```

```
    for (String word : words)
```

```
{
```

```
        if (isValidWord(word))
```

```
        {
```

```
            System.out.print(word + " ");  
            found = true;
```

```
        }
```

```
    }
```

```
    if (!found)
```

```
    {
```

```
        System.out.print("No valid words.");
```

```
    }
```

```
    sc.close();
```

```
}
```

```
private static boolean isValidWord(String word)
```

```
{
```



```
if (word.length() < 2) return false;
for (int i = 0; i < word.length(); i++)
{
    if (!Character.isLetter(word.charAt(i)))
    {
        return false;
    }
}
return true;
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 4_Q5

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

In a secure banking system, customers are required to create PIN codes for accessing their accounts. The bank wants to validate these PIN codes before accepting them.

A PIN code is considered valid if:

It consists of exactly 4 digits. All characters must be numeric (0–9). It cannot contain all identical digits (e.g., 1111 is invalid).

Your task is to determine whether each PIN code in the list is valid or not.

Input Format

The first line of input contains an integer T, representing the number of PIN codes to check.

The next T lines each contain a string S, representing a PIN code.

Output Format

For each PIN code S, the output print "YES" if it is valid.

Otherwise, the output print "NO".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1

1234

Output: YES

Answer

```
import java.util.*;
```

```
public class Main
```

```
{
```

```
    public static void main(String[] args)
```

```
{
```

```
    Scanner sc = new Scanner(System.in);
```

```
    int T = sc.nextInt();
```

```
    sc.nextLine();
```

```
    for (int i = 0; i < T; i++)
```

```
{
```

```
    String pin = sc.nextLine();
```

```
    if (isValidPin(pin))
```

```
    {
```

```
        System.out.println("YES");
```

```
    } else
```

```
    {
```

```
        System.out.println("NO");
```

```
    }
```

```
}
```

```
    sc.close();
```

```
}
```

```
private static boolean isValidPin(String pin)
```

```
{
```

```
if (pin.length() != 4) return false;
```

```
for (int i = 0; i < pin.length(); i++)
```

```
{
```

```
    if (!Character.isDigit(pin.charAt(i)))
```

```
    {
```

```
        return false;
```

```
    }
```

```
}
```

```
char first = pin.charAt(0);  
boolean allSame = true;  
for (int i = 1; i < pin.length(); i++)
```

```
{
```

```
    if (pin.charAt(i) != first)
```

```
    {
```

```
allSame = false;  
break;
```

```
}
```

```
}
```

```
return !allSame;
```

```
}
```

```
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

REC_2028_OOPS using Java_Week 4_PAH

Attempt : 1
Total Mark : 40
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Sana is analyzing text for a secret code. She wants to find all words in a sentence that start and end with the same letter. These words are considered "special words" for her analysis.

Your task is to write a program that extracts and prints all words that start and end with the same letter (case-insensitive).

If no such word exists, print "No special words found".

Input Format

The input contains a single line containing a sentence with multiple words.

Output Format

The output prints all words that start and end with the same letter separated by a space.

If no word satisfies the condition, print "No special words found".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: Anna went to the civic center

Output: Anna civic

Answer

```
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;
```

```
public class Main
```

```
{
```

```
    public static void main(String[] args)
```

```
{
```

```
    Scanner scanner = new Scanner(System.in);
    String sentence = scanner.nextLine();
    scanner.close();
```

```
    String[] words = sentence.split(" ");
    List<String> specialWords = new ArrayList<>();
```

```
    for (String word : words)
```

```
{
```

```
        if (word.length() > 0)
```



```
{  
    char firstChar = Character.toLowerCase(word.charAt(0));  
    char lastChar = Character.toLowerCase(word.charAt(word.length() - 1));  
  
    if (firstChar == lastChar)  
  
    {  
  
        specialWords.add(word);  
    }  
  
}  
  
if (specialWords.isEmpty())  
{  
    System.out.println("No special words found");  
}  
else  
{  
  
    System.out.println(String.join(" ", specialWords));  
}
```

```
}  
}
```

Status : Correct

Marks : 10/10

2. Problem Statement

At a digital library, the system needs to analyze passages to identify the frequency of vowels, since they are key for linguistic research. You are asked to write a program that counts the number of vowels in each passage of text.

The vowels of interest are:

a, e, i, o, u (both uppercase and lowercase).

Input Format

The first line of input contains an integer T, representing the number of test cases (passages).

Each of the next T lines contains a single passage of text.

Output Format

For each test case, print a single integer representing the total number of vowels in the passage.

The first line of output corresponds to the first passage, the second line to the second passage, and so on.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1
Hello World
Output: 3

Answer

-

Status : -

Marks : 0/10

3. Problem Statement

Riya is preparing a puzzle game for her friends. She wants to include a feature that highlights special words in a sentence — specifically, palindromic words (words that read the same forward and backward).

Your task is to help Riya by writing a program that extracts all palindrome words from the given sentence. If there are no palindromes, print "No palindromes found".

Input Format

The input contains a single string S representing a sentence.

Output Format

The output prints all palindromic words separated by a space.

If no palindrome exists, print "No palindromes found".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: madam went to school

Output: madam

Answer

-

Status : -

Marks : 0/10

4. Problem Statement

Ravi is analyzing text messages for his research on typing patterns. He wants to count the number of uppercase letters, lowercase letters, and digits in a sentence to understand typing trends.

Your task is to help Ravi by writing a program that takes a sentence and prints the count of uppercase letters, lowercase letters, and digits.

Input Format

The input contains a single line containing a sentence (string).

Output Format

The output prints three integers separated by spaces:

- Number of uppercase letters
- Number of lowercase letters
- Number of digits

Refer to the sample output for formatting specifications.

Sample Test Case

Input: Hello World 123

Output: 2 8 3

Answer

-

Status : -

Marks : 0/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

REC_2028_OOPS using Java_Week 4_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Meera is practicing her English vocabulary. She wants to focus on words that have more vowels in them, as they help improve her pronunciation. She decides to extract only those words from a sentence that contain at least two vowels.

Your task is to help Meera by writing a program that finds such words from the given sentence.

Input Format

The input contains a string representing the sentence.

Output Format

The output prints all the words that contain at least two vowels, separated by a space.

If no such word exists, print "No words with two vowels".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: This is an example sentence

Output: example sentence

Answer

```
import java.util.*;  
public class Main
```

```
{
```

```
    public static void main(String[] args)
```

```
{
```

```
    Scanner sc = new Scanner(System.in);  
    String sentence = sc.nextLine();
```

```
    String[] words = sentence.split(" ");  
    boolean found = false;
```

```
    for (String word : words)
```

```
{
```

```
if (countVowels(word) >= 2)
```

```
{
```

```
    System.out.print(word + " ");  
    found = true;
```

```
}
```

```
}
```

```
if (!found)
```

```
{
```

```
    System.out.print("No words with two vowels");
```

```
}
```

```
    sc.close();
```

```
}
```

```
private static int countVowels(String word)
```

```
{
```

```
int count = 0;
for (int i = 0; i < word.length(); i++)
{

    char ch = Character.toLowerCase(word.charAt(i));
    if (ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u')

    {

        count++;

    }

}

return count;

}
}
```

Status : Correct

Marks : 10/10

2. Problem Statement

Anjali is preparing a report on text complexity. She wants to identify all words in a sentence that contain at least one digit so she can analyze numeric mentions.

Your task is to write a program that extracts and prints all words containing at least one digit from a given sentence.

If no such word exists, print "No words with digits found".

Input Format

The input contains a single line containing a sentence with multiple words.

Output Format

The output prints all words containing at least one digit separated by a space.

If no word contains a digit, print "No words with digits found".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: The model X100 and Y200 are available

Output: X100 Y200

Answer

```
import java.util.*;
```

```
public class Main
```

```
{
```

```
    public static void main(String[] args)
```

```
{
```

```
    Scanner sc = new Scanner(System.in);  
    String sentence = sc.nextLine();
```

```
String[] words = sentence.split(" ");  
boolean found = false;
```

```
for (String word : words)
```

```
{
```

```
    if (containsDigit(word))
```

```
    {
```

```
        System.out.print(word + " ");  
        found = true;
```

```
    }
```

```
}
```

```
if (!found)
```

```
{
```

```
    System.out.print("No words with digits found");
```

```
}
```

```
sc.close();
```

```
}
```

```
private static boolean containsDigit(String word)
```

```
{
```

```
    for (int i = 0; i < word.length(); i++)
```

```
{
```

```
    if (Character.isDigit(word.charAt(i)))
```

```
{
```

```
        return true;
```

```
}
```

```
}
```

```
    return false;
```

```
}
```

```
}
```

Status : Correct

Marks : 10/10

3. Problem Statement

Riya is preparing for a vocabulary test. Her teacher told her to focus on long words in her practice sentences, specifically words that have at least 5 letters.

Riya wants to write a program that will help her identify such words quickly.

Your task is to help Riya by printing all the words in a given sentence that have a length greater than or equal to 5.

If no such word exists, display "No long words found".

Input Format

The input contains a single line containing a sentence with multiple words.

Output Format

The output prints all words having length ≥ 5 , separated by a space.

If no such word is found, print "No long words found".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: The quick brown fox jumps over the lazy dog

Output: quick brown jumps

Answer

```
import java.util.*;
```

```
public class Main
```

```
{
```

```
public static void main(String[] args)
```

```
{
```

```
    Scanner sc = new Scanner(System.in);  
    String sentence = sc.nextLine();
```

```
    String[] words = sentence.split(" ");  
    boolean found = false;
```

```
    for (String word : words)
```

```
    {
```

```
        if (word.length() >= 5)
```

```
        {
```

```
            System.out.print(word + " ");  
            found = true;
```

```
        }
```

```
    }
```

```
    if (!found)
```

```
    {
```

```
System.out.print("No long words found");
```

```
}
```

```
sc.close();
```

```
}
```

```
}
```

Status : Correct

Marks : 10/10

4. Problem Statement

In a university library, librarians need to track the usage of special characters in students' notes.

To help them, you are asked to write a program that counts the number of specific symbols in each passage of text.

The symbols of interest are:

Exclamation marks (!) Colons (:) Semicolons (;)

Input Format

The first line of input contains an integer T, representing the number of test cases (passages).

Each of the next T lines contains a single passage of text.

Output Format

For each test case, print three integers separated by spaces, representing the number of exclamation marks, colons, and semicolons in the passage.

The first line of output corresponds to the first passage, the second line to the second passage, and so on.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1

Hello! How are you

Output: 1 0 0

Answer

```
import java.util.Scanner;
```

```
public class Main
```

```
{
```

```
    public static void main(String[] args)
```

```
{
```

```
    Scanner scanner = new Scanner(System.in);
```

```
    if (!scanner.hasNextInt())
```

```
{
```

```
        scanner.close();
```

```
        return;
```

```
}
```

```
    int T = scanner.nextInt();
```

```
scanner.nextLine();

for (int t = 0; t < T; t++)
{

    String passage;
    if (scanner.hasNextLine())
    {

        passage = scanner.nextLine();
    } else
    {

        break;
    }

    int exclamations = 0;
    int colons = 0;
    int semicolons = 0;

    for (int i = 0; i < passage.length(); i++)
    {

        char ch = passage.charAt(i);
```



```
        if (ch == '!')
```

```
{
```

```
            exclamations++;
```

```
        } else if (ch == ':')
```

```
{
```

```
            colons++;
```

```
        } else if (ch == ';')
```

```
{
```

```
            semicolons++;
```

```
}
```

```
}
```

```
        System.out.println(exclamations + " " + colons + " " + semicolons);
```

```
    }
```

```
    scanner.close();
```

```
}
```

```
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 5_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 15

Section 1 : MCQ

1. What will be the output of the following code?

```
class A {  
    int x = 50;  
}  
  
public class Main {  
    public static void main(String[] args) {  
        A obj1 = new A();  
        A obj2 = obj1;  
        obj2.x = 100;  
        System.out.println(obj1.x);  
    }  
}
```

Answer

100

Status : Correct

Marks : 1/1

2. What will be the output of the following code?

```
class Ball {  
    int size = 11;  
}  
  
class Game {  
    public static void main(String[] args) {  
        Ball b1 = new Ball();  
        Ball b2 = new Ball();  
        b2.size = 10;  
        System.out.println(b1.size);  
    }  
}
```

Answer

11

Status : Correct

Marks : 1/1

3. What will be the output of the following code?

```
class A {  
    int p = 5;  
    int q = 2;  
}  
  
class Main {  
    public static void main(String[] args) {  
        A obj = new A();  
        System.out.println(obj.p + obj.q);  
    }  
}
```

Answer

7

Status : Correct

Marks : 1/1

4. What will be the output of the following code?

```
class Person {  
    int age = 18;  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Person p = new Person();  
        p.age += 2;  
        System.out.println("Age: " + p.age);  
    }  
}
```

Answer

Age: 20

Status : Correct

Marks : 1/1

5. What will be the output of the following code?

```
class Box {  
    int length = 5;  
    int width = 4;  
  
    int area() {  
        return length * width;  
    }  
  
    public static void main(String[] args) {  
        Box b = new Box();  
        System.out.println("Area = " + b.area());  
    }  
}
```

Answer

Area = 20

Status : Correct

Marks : 1/1

6. What will be the output of the following code?

```
class Box {  
    int volume(int l, int b, int h) {  
        return l * b * h;  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Box b = new Box();  
        System.out.println(b.volume(2, 3, 4));  
    }  
}
```

Answer

24

Status : Correct

Marks : 1/1

7. What will be the output of the following code?

```
class Demo {  
    void printMessage() {  
        System.out.println("Hello from Demo");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Demo d = new Demo();  
        d.printMessage();  
    }  
}
```

```
}
```

Answer

Hello from Demo

Status : Correct

Marks : 1/1

8. What will be the output of the following code?

```
class MathUtils {  
    int add(int x) {  
        return x + x;  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        MathUtils m = new MathUtils();  
        System.out.println(m.add(5));  
    }  
}
```

Answer

10

Status : Correct

Marks : 1/1

9. What will be the output of the following code?

```
class A {  
    int y = 30;  
}  
  
public class Main {  
    public static void main(String[] args) {  
        A a1 = new A();  
        A a2 = new A();  
        a1.y = 50;  
        System.out.println(a2.y);  
    }  
}
```

Answer

30

Status : Correct

Marks : 1/1

10. What will be the output of the following code?

```
class Person {  
    String name;  
    void setName(String n) {  
        name = n;  
    }  
    void printName() {  
        System.out.println(name);  
    }  
}  
  
class Test {  
    public static void main(String[] args) {  
        Person p = new Person();  
        p.printName();  
    }  
}
```

Answer

null

Status : Correct

Marks : 1/1

11. What will be the output of the following code?

```
class Sample {  
    int x = 10;  
  
    void display() {  
        System.out.println("x = " + x);  
    }  
}
```



```
}  
public static void main(String[] args) {  
    Sample s = new Sample();  
    s.display();  
}  
}
```

Answer

x = 10

Status : Correct

Marks : 1/1

12. What will be the output of the following code?

```
class A {  
    int val = 20;  
}  
  
public class Main {  
    public static void main(String[] args) {  
        A obj1 = new A();  
        A obj2 = obj1;  
        obj2.val += 5;  
        System.out.println(obj1.val);  
    }  
}
```

Answer

25

Status : Correct

Marks : 1/1

13. What is the output of the following code?

```
class Box {  
    int height;  
    Box(int height) {  
        this.height = height;  
    }  
}
```

```

    }
    void modifyHeight(Box b) {
        b.height += 10;
    }
}

public class Main {
    public static void main(String[] args) {
        Box b1 = new Box(20);
        b1.modifyHeight(b1);
        System.out.println(b1.height);
    }
}

```

Answer

30

Status : Correct

Marks : 1/1

14. What will be the output of the following code?

```

class Alpha {
    void greet(String name) {
        System.out.println("Hello " + name);
    }
}

public class Main {
    public static void main(String[] args) {
        Alpha obj = new Alpha();
        obj.greet("Anu");
    }
}

```

Answer

Hello Anu

Status : Correct

Marks : 1/1

15. What will be the output of the following code?

```
class Test {  
    private int value;  
    Test(int value) {  
        this.value = value;  
    }  
    public int getValue() {  
        return value;  
    }  
}  
public class Main {  
    public static void main(String[] args) {  
        Test obj = new Test(10);  
        System.out.println(obj.value);  
    }  
}
```

Answer

Compile-time error

Status : Correct

Marks : 1/1

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 5_Q2

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

You are working as a developer for CityBank, which wants to build a basic account management system.

Each customer at the bank has:

An Account Number (integer) A Customer Name (string) An Initial Balance (double)

The bank allows two types of transactions:

Deposit – increases the balance. Withdrawal – decreases the balance only if enough funds are available.

If the withdrawal amount is greater than the balance, the withdrawal should not happen, and the balance should remain the same.

You are required to implement this system using:

A class with attributes for account details. A constructor to initialize account details. Setter methods to update details if needed. Getter methods to retrieve details. Objects of the class to represent customers.

Finally, display each customer's account details after all transactions.

Input Format

The first line of input contains an integer N, representing the number of customers.

For each customer:

- The next line contains the account number (integer).
- The following line contains the customer name (string).
- The next line contains the initial balance (double).
- The next line contains the deposit amount (double).
- The next line contains the withdrawal amount (double).

Output Format

For each customer, print the details in the following format:

1. Account Number: <account_number>
2. Customer Name: <customer_name>
3. Final Balance: <final_balance> (rounded to one decimal place)

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1

1234

Rahul Sharma

5000

2000

3000

Output: Account Number: 1234

Customer Name: Rahul Sharma

Final Balance: 4000.0

Answer

```
import java.util.Scanner;
import java.text.DecimalFormat;

class BankAccount

{
    private int accountNumber;
    private String customerName;
    private double balance;

    public BankAccount(int accountNumber, String customerName, double
initialBalance)
    {
        this.accountNumber = accountNumber;
        this.customerName = customerName;
        this.balance = initialBalance;
    }
    public void setCustomerName(String customerName)

    {
        this.customerName = customerName;
    }
    public int getAccountNumber()
    {
        return accountNumber;
    }
    public String getCustomerName()
    {
        return customerName;
    }
    public double getBalance()
    {
        return balance;
    }
    public void deposit(double amount)
    {
        if (amount > 0)
```

```
        this.balance += amount;
    }
}

public void withdraw(double amount)
{
```

```
    if (amount > 0 && amount <= this.balance)
```

```
    {
```

```
        this.balance -= amount;
```

```
    }
```

```
}
```

```
}
```

```
public class Main
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        Scanner scanner = new Scanner(System.in);
```

```
        DecimalFormat df = new DecimalFormat("0.0");
```

```
        int n = scanner.nextInt();
```

```
        scanner.nextLine();
```

```
        for (int i = 0; i < n; i++)
```

```
        {
```

```
            int accNum = scanner.nextInt();
```

```
            scanner.nextLine();
```

```
String name = scanner.nextLine();
double initialBalance = scanner.nextDouble();
double depositAmount = scanner.nextDouble();
double withdrawalAmount = scanner.nextDouble();

BankAccount account = new BankAccount(accNum, name,
initialBalance);

account.deposit(depositAmount);
account.withdraw(withdrawalAmount);

System.out.println("Account Number: " + account.getAccountNumber());
System.out.println("Customer Name: " + account.getCustomerName());
System.out.println("Final Balance: " + df.format(account.getBalance()));
}

scanner.close();

}

}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 5_Q3

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Neha is working as a developer for CityElectricity Board, which wants to build a household electricity billing system.

Each customer's electricity account has:

A Customer ID (integer) A Customer Name (string) Units Consumed (double)

The electricity bill is calculated based on these rules:

For the first 100 units 5 units charge per unit For the next 100 units (101–200) 7 units charge per unit For units above 200 10 units charge per unit If the total bill exceeds 2000 units, a 5% discount is applied on the final bill.

Neha has been asked to implement this system using:

A class with attributes for customer details. A constructor to initialize customer details. Setter methods to update details if needed. Getter methods to retrieve details. Objects of the class to represent customers.

Finally, display each customer's details and final bill amount.

Input Format

The first line of input contains an integer N, representing the number of customers.

For each customer:

- The next line contains the Customer ID (integer).
- The following line contains the Customer Name (string).
- The next line contains the Units Consumed (double).

Output Format

For each customer, print the details in the following format:

Customer ID: <customer_id>

Customer Name: <customer_name>

Final Bill: <final_bill> (rounded to one decimal place)

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1

1001

Ravi Kumar

80

Output: Customer ID: 1001

Customer Name: Ravi Kumar

Final Bill: 400.0

Answer

```
import java.util.Scanner;
```

```
import java.text.DecimalFormat;
class ElectricityAccount
{

    private int customerId;
    private String customerName;
    private double unitsConsumed;
    private static final double RATE_1 = 5.0;
    private static final double RATE_2 = 7.0;
    private static final double RATE_3 = 10.0;
    private static final double DISCOUNT_THRESHOLD = 2000.0;
    private static final double DISCOUNT_RATE = 0.05;
    ElectricityAccount(int customerId, String customerName, double
unitsConsumed)

    {

        this.customerId = customerId;
        this.customerName = customerName;
        this.unitsConsumed = unitsConsumed;

    }
    public void setCustomerName(String customerName)
    {

        this.customerName = customerName;

    }
    public int getCustomerId()
    {

        return customerId;
```

```
}  
public String getCustomerName()  
{
```

```
    return customerName;
```

```
}  
public double getUnitsConsumed()  
{
```

```
    return unitsConsumed;
```

```
}  
public double calculateBill()  
{
```

```
    double units = this.unitsConsumed;  
    double billAmount = 0.0;  
    if (units > 0)
```

```
{
```

```
    double tier1Units = Math.min(units, 100);  
    billAmount += tier1Units * RATE_1;  
    units -= tier1Units;
```

```
}
```

```
    if (units > 0)
```

```
{
```

```
double tier2Units = Math.min(units, 100);  
billAmount += tier2Units * RATE_2;  
units -= tier2Units;
```

```
}
```

```
if (units > 0)
```

```
{
```

```
    billAmount += units * RATE_3;
```

```
}
```

```
if (billAmount > DISCOUNT_THRESHOLD)
```

```
{
```

```
    billAmount *= (1.0 - DISCOUNT_RATE);
```

```
}
```

```
return billAmount;
```

```
}
```

```
}
```

```
public class Main
```

```
{
```

```
    public static void main(String[] args)
```

```
{
```

```
    Scanner scanner = new Scanner(System.in);
```

```
DecimalFormat df = new DecimalFormat("0.0");
int n = scanner.nextInt();
scanner.nextLine();
for (int i = 0; i < n; i++)

{

    int customerId = scanner.nextInt();
    scanner.nextLine();
    String name = scanner.nextLine();
    double unitsConsumed = scanner.nextDouble();
    ElectricityAccount account = new ElectricityAccount(customerId, name,
unitsConsumed);
    double finalBill = account.calculateBill();
    System.out.println("Customer ID: " + account.getCustomerId());
    System.out.println("Customer Name: " + account.getCustomerName());
    System.out.println("Final Bill: " + df.format(finalBill));

}

scanner.close();

}

}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 5_Q4

Attempt : 1
Total Mark : 10
Marks Obtained : 0

Section 1 : Coding

1. Problem Statement

You are working as a developer for CityCab, a taxi service company that wants to build a ride fare management system.

Each customer booking has:

A Booking ID (integer) A Customer Name (string) A Distance Travelled in km (double)

The fare calculation rules are:

Base Fare = 50 units (flat charge for every ride). Per km charge = 10 units/km. If the distance is greater than 20 km, a 10% discount is applied on the total fare.

You are required to implement this system using:

A class with attributes for booking details. A constructor to initialize booking details. Setter methods to update details if needed. Getter methods to retrieve details. Objects of the class to represent customer rides.

Finally, display each booking's details and final fare.

Input Format

The first line of input contains an integer N, representing the number of bookings.

For each booking:

- The next line contains the booking ID (integer).
- The following line contains the customer's name (string).
- The next line contains the distance travelled (double).

Output Format

For each booking, print the details in the following format:

1. Booking ID: <booking_id>
2. Customer Name: <customer_name>
3. Final Fare: <final_fare> (rounded to one decimal place)

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1
1234
Rahul Sharma
15

Output: Booking ID: 1234
Customer Name: Rahul Sharma
Final Fare: 200.0

Answer

```
import java.util.Scanner;  
import java.text.DecimalFormat;
```



```
class Booking
```

```
{
```

```
    private int bookingId;  
    private String customerName;  
    private double distance;  
    private double fare;  
    private static final double BASE_FARE = 95.0;  
    private static final double RATE_PER_UNIT = 7.0;
```

```
    Booking(int bookingId, String customerName, double distance)
```

```
{
```

```
        this.bookingId = bookingId;  
        this.customerName = customerName;  
        this.distance = distance;  
        calculateFare();
```

```
}
```

```
    public int getBookingId()
```

```
{
```

```
        return bookingId;
```

```
}
```

```
    public String getCustomerName()
```

```
{
```

```
        return customerName;
```

```
}  
    public double getFare()  
    {  
  
        return fare;  
  
    }
```

```
    private void calculateFare()  
    {
```

```
        // Logic derived from sample cases: Final Fare = 95.0 + (Distance * 7.0)  
        this.fare = BASE_FARE + this.distance * RATE_PER_UNIT;
```

```
    }
```

```
    }
```

```
    public class Main
```

```
    {
```

```
        public static void main(String[] args)
```

```
        {
```

```
            Scanner sc = new Scanner(System.in);
```

```
            // Read N (number of customers) using line reading for robust input  
            handling
```

```
            int n = Integer.parseInt(sc.nextLine());
```

```
DecimalFormat df = new DecimalFormat("0.0");
for (int i = 0; i < n; i++)
{

    // Read ID
    int id = Integer.parseInt(sc.nextLine());

    // Read Name
    String name = sc.nextLine();

    // Read Distance
    double distance = Double.parseDouble(sc.nextLine());

    Booking booking = new Booking(id, name, distance);

    System.out.println("Booking ID: " + booking.getBookingId());
    System.out.println("Customer Name: " + booking.getCustomerName());
    System.out.println("Final Fare: " + df.format(booking.getFare()));

}

sc.close();
}
}
```

Status : Wrong

Marks : 0/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 5_Q5

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Ram is working as a developer for BrightEdu Coaching Center, which wants to build a student fee management system.

Each student's enrollment has:

An Enrollment ID (integer) A Student Name (string) The Number of Subjects (integer)

The fee calculation rules are:

Registration Fee = 1000 units (flat for every student). Per Subject Fee = 800 units. If the student enrolls in more than 5 subjects, a 20% scholarship (discount) is applied on the total fee.

Ram has been asked to implement this system using:

A class with attributes for student details. A constructor to initialize student details. Setter methods to update details if needed. Getter methods to retrieve details. Objects of the class to represent student enrollments.

Finally, display each student's details and final fee.

Input Format

The first line of input contains an integer N, representing the number of students.

For each student:

- The next line contains the Enrollment ID (integer).
- The following line contains the student's name (string).
- The next line contains the Number of subjects (integer).

Output Format

For each student, print the details in the following format:

- Enrollment ID: <enrollment_id>
- Student Name: <student_name>
- Final Fee: <final_fee> (rounded to one decimal place)

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1

1234

Ravi Kumar

3

Output: Enrollment ID: 1234

Student Name: Ravi Kumar

Final Fee: 3400.0

Answer

```
import java.util.Scanner;  
import java.text.DecimalFormat;
```

```
class StudentEnrollment
```

```
{  
    private int enrollmentId;  
    private String studentName;  
    private int numSubjects;  
    private double finalFee;  
  
    private static final double REGISTRATION_FEE = 1000.0;  
    private static final double SUBJECT_FEE = 800.0;  
    private static final int SUBJECT_DISCOUNT_THRESHOLD = 5;  
    private static final double SCHOLARSHIP_RATE = 0.20;  
  
    StudentEnrollment(int enrollmentId, String studentName, int numSubjects)  
    {  
  
        this.enrollmentId = enrollmentId;  
        this.studentName = studentName;  
        this.numSubjects = numSubjects;  
        calculateFinalFee();  
  
    }  
  
    // Getters  
    public int getEnrollmentId()  
    {  
  
        return enrollmentId;  
  
    }  
  
    public String getStudentName()  
    {
```

```
        return studentName;
```

```
    }
```

```
    public double getFinalFee()
```

```
    {
```

```
        return finalFee;
```

```
    }
```

```
    // Setters (Included for completeness as requested in the problem statement,  
    although not used in Main)
```

```
    public void setEnrollmentId(int enrollmentId)
```

```
    {
```

```
        this.enrollmentId = enrollmentId;
```

```
    }
```

```
    public void setStudentName(String studentName)
```

```
    {
```

```
        this.studentName = studentName;
```

```
    }
```

```
    public void setNumSubjects(int numSubjects)
```

```
    {
```

```
this.numSubjects = numSubjects;  
calculateFinalFee();
```

```
}
```

```
private void calculateFinalFee()
```

```
{
```

```
    double totalSubjectFee = this.numSubjects * SUBJECT_FEE;  
    double baseFee = REGISTRATION_FEE + totalSubjectFee;  
    double discount = 0.0;
```

```
    if (this.numSubjects > SUBJECT_DISCOUNT_THRESHOLD)
```

```
{
```

```
        discount = baseFee * SCHOLARSHIP_RATE;
```

```
}
```

```
    this.finalFee = baseFee - discount;
```

```
}
```

```
}
```

```
public class Main
```

```
{
```

```
    public static void main(String[] args)
```

```
{
```



```
Scanner sc = new Scanner(System.in);

int n = Integer.parseInt(sc.nextLine());

DecimalFormat df = new DecimalFormat("0.0");

for (int i = 0; i < n; i++)

{

    int id = Integer.parseInt(sc.nextLine());
    String name = sc.nextLine();
    int subjects = Integer.parseInt(sc.nextLine());

    StudentEnrollment enrollment = new StudentEnrollment(id, name,
subjects);

    System.out.println("Enrollment ID: " + enrollment.getEnrollmentId());
    System.out.println("Student Name: " + enrollment.getStudentName());
    System.out.println("Final Fee: " + df.format(enrollment.getFinalFee()));

}

sc.close();

}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 5_PAH

Attempt : 1
Total Mark : 50
Marks Obtained : 0

Section 1 : Coding

1. Problem Statement

Neha is working as a developer for CityQuiz Platform, which wants to build a system to calculate quiz scores and identify top scorers among participants.

Each participant's record has:

Participant ID (integer) Participant Name (string) An array of scores in 5 quiz rounds (integers, each between 0 and 100)

The system must calculate:

Total Score = sum of scores in all 5 rounds. Average Score = Total Score ÷ 5. If a participant scores above 80 in all rounds, a bonus of 10 points is added to the total score. Identify the Top Scorer among all participants. If

two participants have the same total score, the one with the lower Participant ID is considered the top scorer.

Neha has been asked to implement this system using:

A class with attributes for participant details. A constructor to initialize participant details. Getter and setter methods to retrieve or update participant details. A method to calculate total score and average score (including bonus if applicable). Objects of the class to represent participants.

Finally, display each participant's details and announce the Top Scorer.

Input Format

The first line of input contains an integer N, representing the number of participants.

For each participant:

- Next line: Participant ID (integer)
- Next line: Participant Name (string)
- Next line: 5 integers separated by spaces (scores for 5 quiz rounds)

Output Format

For each participant:

- Participant ID: <participant_id>
- Participant Name: <participant_name>
- Total Score: <total_score>
- Average Score: <average_score>

Finally, print "Top Scorer: <participant_name> with <total_score> points"

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1

1001

Ravi Kumar

85 90 88 92 87

Output: Participant ID: 1001

Participant Name: Ravi Kumar

Total Score: 452

Average Score: 90

Top Scorer: Ravi Kumar with 452 points

Answer

```
import java.util.Scanner;  
import java.util.ArrayList;  
import java.util.List;
```

```
class Participant
```

```
{
```

```
    private int participantId;  
    private String participantName;  
    private int[] scores = new int[5];  
    private int totalScore;  
    private int averageScore;
```

```
    Participant(int participantId, String participantName, int[] scores)
```

```
{
```

```
        this.participantId = participantId;  
        this.participantName = participantName;  
        System.arraycopy(scores, 0, this.scores, 0, 5);  
        calculateScores();
```

```
}
```

```
    public int getParticipantId()
```

```
{
```

return participantId;

}

public String getParticipantName()

{

return participantName;

}

public int getTotalScore()

{

return totalScore;

}

public int getAverageScore()

{

return averageScore;

}

public int[] getScores()

{

```
return scores;
```

```
}
```

```
public void setParticipantId(int participantId)
```

```
{
```

```
    this.participantId = participantId;
```

```
}
```

```
public void setParticipantName(String participantName)
```

```
{
```

```
    this.participantName = participantName;
```

```
}
```

```
public void setScores(int[] scores)
```

```
{
```

```
    System.arraycopy(scores, 0, this.scores, 0, 5);  
    calculateScores();
```

```
}
```

```
private void calculateScores()
```

```
{
```

```
    int sum = 0;
```

```
        boolean allAbove80 = true;
        for (int score : this.scores)
        {

            sum += score;
            if (score <= 80)

            {

                allAbove80 = false;
            }

        }

        this.totalScore = sum;
        if (allAbove80)
        {

            this.totalScore += 10;

        }

        this.averageScore = this.totalScore / 5;

    }

}

public class Main
```

```
{  
    public static void main(String[] args)  
    {  
  
        Scanner sc = new Scanner(System.in);  
  
        int n = Integer.parseInt(sc.nextLine());  
  
        List<Participant> participants = new ArrayList<>();  
        for (int i = 0; i < n; i++)  
        {  
  
            int id = Integer.parseInt(sc.nextLine());  
  
            String name = sc.nextLine();  
  
            String[] scoreStrings = sc.nextLine().split(" ");  
            int[] scores = new int[5];  
            for(int j = 0; j < 5; j++)  
            {  
  
                scores[j] = Integer.parseInt(scoreStrings[j]);  
  
            }  
  
            Participant participant = new Participant(id, name, scores);  
            participants.add(participant);  
  
        }  
  
        Participant topScorer = null;
```



```
for (Participant p : participants)
```

```
{
```

```
    System.out.println("Participant ID: " + p.getParticipantId());  
    System.out.println("Participant Name: " + p.getParticipantName());  
    System.out.println("Total Score: " + p.getTotalScore());  
    System.out.println("Average Score: " + p.getAverageScore());
```

```
    if (topScorer == null)
```

```
    {
```

```
        topScorer = p;
```

```
    } else
```

```
    {
```

```
        if (p.getTotalScore() > topScorer.getTotalScore())
```

```
        {
```

```
            topScorer = p;
```

```
        } else if (p.getTotalScore() == topScorer.getTotalScore())
```

```
        {
```

```
            if (p.getParticipantId() < topScorer.getParticipantId())
```

```
            {
```

```
        topScorer = p;
    }

}

}

}

if (topScorer != null)
{

    System.out.println("Top Scorer: " + topScorer.getParticipantName() + "
with " + topScorer.getTotalScore() + " points");

}

    sc.close();
}

}
```

Status : Skipped

Marks : 0/10

2. Problem Statement

Neha is working as a developer for CityMovie Theatre, which wants to build a system to calculate total ticket cost for movie-goers based on the number of tickets and type of seats booked.

Each customer's booking has:

Booking ID (integer) Customer Name (string) Number of Tickets (integer) Seat Type (string: "Standard", "Premium", "VIP")

The ticket prices are:

Standard – 250 units per ticket Premium – 400 units per ticket VIP – 600 units per ticket

The calculation rules:

Total Amount = Number of Tickets × Seat Price

If a customer books more than 4 tickets, they get a 10% discount on the total amount.

If the booking is for VIP seats and the total amount exceeds 3000 units, a 5% luxury tax is added after any discount.

Neha has been asked to implement this system using:

A class with attributes for booking details. A constructor to initialize booking details. Getter and Setter methods to retrieve and update booking details if required. A method to calculate the final ticket cost. Objects of the class to represent bookings.

Finally, display each customer's details and final ticket amount.

Input Format

The first line contains an integer N, representing the number of bookings.

For each booking:

- The next line contains the Booking ID (integer).
- The next line contains the Customer Name (string).
- The next line contains Number of Tickets (integer).
- The next line contains Seat Type ("Standard", "Premium", or "VIP").

Output Format

For each booking, print:

- Booking ID: <booking_id>
- Customer Name: <customer_name>
- Final Ticket Amount: <final_amount> (rounded to one decimal place)

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1

1001

Ravi Kumar

3

Standard

Output: Booking ID: 1001

Customer Name: Ravi Kumar

Final Ticket Amount: 750.0

Answer

```
import java.util.Scanner;
```

```
import java.text.DecimalFormat;
```

```
class MovieBooking
```

```
{
```

```
    private int bookingId;
```

```
    private String customerName;
```

```
    private int numTickets;
```

```
    private String seatType;
```

```
    private double finalAmount;
```

```
    private static final int PRICE_STANDARD = 250;
```

```
    private static final int PRICE_PREMIUM = 400;
```

```
    private static final int PRICE_VIP = 600;
```

```
    private static final int DISCOUNT_THRESHOLD = 4;
```

```
    private static final double DISCOUNT_RATE = 0.10; // 10%
```

```
    private static final double LUXURY_TAX_RATE = 0.05; // 5%
```

```
    MovieBooking(int bookingId, String customerName, int numTickets, String  
seatType)
```

```
{  
    this.bookingId = bookingId;  
    this.customerName = customerName;  
    this.numTickets = numTickets;  
    this.seatType = seatType;  
    calculateFinalAmount();  
}
```

```
public int getBookingId()
```

```
{  
  
    return bookingId;  
}
```

```
public String getCustomerName()
```

```
{  
  
    return customerName;  
}
```

```
public int getNumTickets()
```

```
{  
  
    return numTickets;  
}
```

```
public String getSeatType()
```

```
{
```

```
    return seatType;
```

```
}
```

```
public double getFinalAmount()
```

```
{
```

```
    return finalAmount;
```

```
}
```

```
public void setBookingId(int bookingId)
```

```
{
```

```
    this.bookingId = bookingId;
```

```
}
```

```
public void setCustomerName(String customerName)
```

```
{
```

```
    this.customerName = customerName;
```

```
}
```

```
public void setNumTickets(int numTickets)
```

```
{  
  
    this.numTickets = numTickets;  
    calculateFinalAmount();  
  
}  
  
public void setSeatType(String seatType)  
  
{  
  
    this.seatType = seatType;  
    calculateFinalAmount();  
  
}  
  
private void calculateFinalAmount()  
  
{
```

```
    double seatPrice = 0;  
    switch (this.seatType)  
    {  
  
        case "Standard":  
            seatPrice = PRICE_STANDARD;  
            break;  
        case "Premium":  
            seatPrice = PRICE_PREMIUM;  
            break;  
        case "VIP":  
            seatPrice = PRICE_VIP;  
            break;  
        default:
```

```
        seatPrice = 0;  
        break;
```

```
    }
```

```
    double totalAmount = this.numTickets * seatPrice;
```

```
    // Apply 10% discount if more than 4 tickets are booked  
    if (this.numTickets > DISCOUNT_THRESHOLD)
```

```
    {
```

```
        totalAmount -= totalAmount * DISCOUNT_RATE;
```

```
    }
```

```
    // Apply 5% luxury tax if VIP seats and total amount exceeds 3000  
    if (this.seatType.equals("VIP") && totalAmount > 3000.0)
```

```
    {
```

```
        totalAmount += totalAmount * LUXURY_TAX_RATE;
```

```
    }
```

```
    this.finalAmount = totalAmount;
```

```
    }
```

```
    }
```

```
public class Main
```

```
{
```



```
public static void main(String[] args)
{

    Scanner sc = new Scanner(System.in);

    int n = Integer.parseInt(sc.nextLine());

    DecimalFormat df = new DecimalFormat("0.0");

    for (int i = 0; i < n; i++)
    {

        int id = Integer.parseInt(sc.nextLine());

        String name = sc.nextLine();

        int tickets = Integer.parseInt(sc.nextLine());

        String seatType = sc.nextLine();

        MovieBooking booking = new MovieBooking(id, name, tickets, seatType);

        System.out.println("Booking ID: " + booking.getBookingId());
        System.out.println("Customer Name: " + booking.getCustomerName());
        System.out.println("Final Ticket Amount: " +
df.format(booking.getFinalAmount()));

    }

    sc.close();

}

}
```

Status : Skipped

Marks : 0/10

3. Problem Statement

Ravi is working as a developer for SecureLogin Systems, which wants to build a system to evaluate the strength of user passwords.

Each user record has:

User ID (integer) User Name (string) Password (string)

The system must calculate whether a password is strong or weak.

A password is considered strong if it meets all of the following conditions:

At least 8 characters long. Contains at least one uppercase letter. Contains at least one lowercase letter. Contains at least one digit. Contains at least one special character (from !@#\$%^&*).

Ravi has been asked to implement this system using:

A class with attributes for user details. A constructor to initialize user details. Getter and setter methods to retrieve or update user details. A method to check whether the password is strong. Objects of the class to represent users.

Finally, display each user's details and indicate whether their password is Strong or Weak.

Input Format

The first line contains an integer N, representing the number of users.

For each user:

The next line contains the User ID (integer).

The next line contains the User Name (string).

The next line contains the Password (string).

Output Format

For each user, print the details in the following format:

User ID: <user_id>

User Name: <user_name>

Password: <password>

Password Strength: <Strong/Weak>

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1

1001

Ravi Kumar

Abc@1234

Output: User ID: 1001

User Name: Ravi Kumar

Password: Abc@1234

Password Strength: Strong

Answer

```
import java.util.Scanner;
```

```
import java.util.regex.Pattern;
```

```
class UserAccount
```

```
{
```

```
    private int userId;
```

```
    private String userName;
```

```
    private String password;
```

```
    UserAccount(int userId, String userName, String password)
```

```
{
```

```
        this.userId = userId;
```

```
this.userName = userName;  
this.password = password;
```

```
}
```

```
public int getUserId()
```

```
{
```

```
    return userId;
```

```
}
```

```
public String getUserName()
```

```
{
```

```
    return userName;
```

```
}
```

```
public String getPassword()
```

```
{
```

```
    return password;
```

```
}
```

```
public void setUserId(int userId)
```

```
{
```

```
    this.userId = userId;
```

```
}
```

```
public void setUsername(String userName)
```

```
{
```

```
    this.userName = userName;
```

```
}
```

```
public void setPassword(String password)
```

```
{
```

```
    this.password = password;
```

```
}
```

```
public String checkPasswordStrength()
```

```
{
```

```
    if (this.password.length() < 8)
```

```
{
```

```
        return "Weak";
```

```
}
```

```
    Pattern uppercase = Pattern.compile(".*[A-Z].*");
```

```
    if (!uppercase.matcher(this.password).matches())
```

```
{
```

```
    return "Weak";
```

```
}
```

```
    Pattern lowercase = Pattern.compile(".*[a-z].*");  
    if (!lowercase.matcher(this.password).matches())
```

```
{
```

```
    return "Weak";
```

```
}
```

```
    Pattern digit = Pattern.compile(".*[0-9].*");  
    if (!digit.matcher(this.password).matches())
```

```
{
```

```
    return "Weak";
```

```
}
```

```
    Pattern special = Pattern.compile(".*[!@#$%^&*.]*");  
    if (!special.matcher(this.password).matches())
```

```
{
```

```
    return "Weak";
```

```
}
```

```
    return "Strong";
```

```
}
```

```
}
```

```
public class Main
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        int n = Integer.parseInt(sc.nextLine());
```

```
        for (int i = 0; i < n; i++)
```

```
        {
```

```
            int id = Integer.parseInt(sc.nextLine());
```

```
            String name = sc.nextLine();
```

```
            String password = sc.nextLine();
```

```
            UserAccount user = new UserAccount(id, name, password);
```

```
            String strength = user.checkPasswordStrength();
```

```
            System.out.println("User ID: " + user.getUserId());
```

```
            System.out.println("User Name: " + user.getUserName());
```

```
            System.out.println("Password: " + user.getPassword());
```

```
            System.out.println("Password Strength: " + strength);
```

```
        }
```

```
        sc.close();  
    }  
  
}
```

Status : Skipped

Marks : 0/10

4. Problem Statement

Each customer at the bank has an Account Number, Customer Name, and an Initial Balance. The bank allows two types of transactions:

Deposit – Increases the balance. Withdrawal – Decreases the balance, but only if enough funds are available. If the withdrawal amount exceeds the available balance, the transaction should be skipped, and the balance should remain unchanged.

You are required to implement this banking system by:

Creating a class with the necessary attributes to store account details.

Using a constructor to initialize the account details when a new account is created. Providing setter methods to update the details if required. Providing getter methods to retrieve account details. Creating objects of this class to represent different customers, where each customer can perform deposits and withdrawals.

Instructions:

Implement the class to store account details. Implement the logic for performing deposit and withdrawal transactions. Ensure that withdrawals don't exceed the available balance. After performing the transactions, print the account number, customer name, and final balance.

Input Format

The first line of input contains an integer N, representing the number of customers.

For each customer:

- The next line contains the account number (integer).
- The following line contains the customer name (string).
- The next line contains the initial balance (double).
- The next line contains the deposit amount (double).
- The next line contains the withdrawal amount (double).

Output Format

For each customer, print the details in the following format:

1. Account Number: <account_number>
2. Customer Name: <customer_name>
3. Final Balance: <final_balance> (rounded to one decimal place)

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1

1234

Rahul Sharma

5000

2000

3000

Output: Account Number: 1234

Customer Name: Rahul Sharma

Final Balance: 4000.0

Answer

```
import java.util.Scanner;
```

```
class Account
```

```
{
```

```
    private int accountNumber;  
    private String customerName;  
    private double balance;
```

```
Account(int accountNumber, String customerName, double balance)
```

```
{
```

```
    this.accountNumber = accountNumber;  
    this.customerName = customerName;  
    this.balance = balance;
```

```
}
```

```
    public void setAccountNumber(int accountNumber)
```

```
{
```

```
    this.accountNumber = accountNumber;
```

```
}
```

```
    public void setCustomerName(String customerName)
```

```
{
```

```
    this.customerName = customerName;
```

```
}
```

```
    public void setBalance(double balance)
```

```
{
```

```
    this.balance = balance;
```

```
}
```

```
public int getAccountNumber()
```

```
{
```

```
    return accountNumber;
```

```
}
```

```
public String getCustomerName()
```

```
{
```

```
    return customerName;
```

```
}
```

```
public double getBalance()
```

```
{
```

```
    return balance;
```

```
}
```

```
public void deposit(double amount)
```

```
{
```

```
    if (amount >= 0)
```

```
{
```

```
        this.balance += amount;
```

```
}
```

```
}
```

```
public void withdraw(double amount)
```

```
{
```

```
    if (amount > 0 && amount <= this.balance)
```

```
    {
```

```
        this.balance -= amount;
```

```
    }
```

```
}
```

```
}
```

```
public class Main
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        int n = Integer.parseInt(sc.nextLine());
```

```
        for (int i = 0; i < n; i++)
```

```
{  
  
    int accNo = Integer.parseInt(sc.nextLine());  
    String name = sc.nextLine();  
    double initBal = Double.parseDouble(sc.nextLine());  
    double deposit = Double.parseDouble(sc.nextLine());  
    double withdraw = Double.parseDouble(sc.nextLine());  
  
    // Corrected constructor call variables: accNo, name, initBal  
    Account acc = new Account(accNo, name, initBal);  
    acc.deposit(deposit);  
    acc.withdraw(withdraw);  
  
    System.out.println("Account Number: " + acc.getAccountNumber());  
    System.out.println("Customer Name: " + acc.getCustomerName());  
    // Corrected output formatting to one decimal place  
    System.out.printf("Final Balance: %.1f\n", acc.getBalance());  
  
}  
  
    sc.close();  
  
}
```

Status : Skipped

Marks : 0/10

5. Problem Statement

Anjali is working as a developer for CityFitness Gym, which wants to build a system to calculate monthly membership fees for gym members based on the type of membership and the number of personal training sessions booked.

Each member's record has:

Member ID (integer) Member Name (string) Membership Type (string: "Basic", "Premium", "Elite") Number of Personal Training Sessions (integer)

The monthly fees are:

Basic – 1000 units Premium – 1500 units Elite – 2000 units

The cost of personal training sessions is 500 units per session.

The calculation rules:

Total Amount = Membership Fee + (Number of Personal Training Sessions × 500) If the number of sessions is more than 5, a 10% discount is applied on the total amount. If the member has Elite membership and the total amount exceeds 4000, an additional 5% service tax is added after discount.

Anjali has been asked to implement this system using:

A class with attributes for member details. A constructor to initialize member details. Getter and Setter methods to retrieve and update member details if required. A method to calculate the final monthly fee. Objects of the class to represent members.

Finally, display each member's details and the final monthly fee.

Input Format

The first line contains an integer N, representing the number of members.

For each member:

- Next line contains Member ID (integer)
- Next line contains Member Name (string)
- Next line contains Membership Type ("Basic", "Premium", "Elite")
- Next line contains Number of Personal Training Sessions (integer)

Output Format

For each member, print:

- Member ID: <member_id>
- Member Name: <member_name>
- Final Monthly Fee: <final_fee> (The final fee must be rounded to one decimal place)

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1

1001

Ravi Kumar

Basic

3

Output: Member ID: 1001

Member Name: Ravi Kumar

Final Monthly Fee: 2500.0

Answer

```
import java.util.Scanner;
```

```
class GymMembership
```

```
{
```

```
    private int memberId;
```

```
    private String memberName;
```

```
    private String membershipType;
```

```
    private int sessions;
```

```
    private double finalFee;
```

```
    public GymMembership(int memberId, String memberName, String  
membershipType, int sessions)
```

```
{
```

```
    this.memberId = memberId;
```

```
    this.memberName = memberName;
```

```
    this.membershipType = membershipType;
```

```
    this.sessions = sessions;
```

```
    calculateFinalFee();
```

```
}
```

```
private void calculateFinalFee()
```

```
{
```

```
    double baseFee = 0;
```

```
    if (membershipType.equals("Basic"))
```

```
    {
```

```
        baseFee = 1000;
```

```
    } else if (membershipType.equals("Premium"))
```

```
    {
```

```
        baseFee = 1500;
```

```
    } else if (membershipType.equals("Elite"))
```

```
    {
```

```
        baseFee = 2000;
```

```
    } else
```

```
    {
```

```
        baseFee = 0;
```



```
}
```

```
double trainingCost = sessions * 500;
```

```
double totalAmount = baseFee + trainingCost;
```

```
if (sessions > 5)
```

```
{
```

```
    totalAmount *= 0.90;
```

```
}
```

```
if (membershipType.equals("Elite") && totalAmount > 4000)
```

```
{
```

```
    totalAmount *= 1.05;
```

```
}
```

```
this.finalFee = totalAmount;
```

```
}
```

```
public int getMemberId()
```

```
{
```

```
    return memberId;
```

```
}
```

```
public String getMemberName()
```

```
{  
    return memberName;  
}
```

```
public double getFinalFee()
```

```
{  
    return finalFee;  
}
```

```
}
```

```
public class Main
```

```
{
```

```
    public static void main(String[] args)
```

```
{
```

```
    Scanner sc = new Scanner(System.in);
```

```
    int n = Integer.parseInt(sc.nextLine());
```

```
    for (int i = 0; i < n; i++)
```

```
{
```

```
        int id = Integer.parseInt(sc.nextLine());  
        String name = sc.nextLine();
```

```
String type = sc.nextLine();
int sessions = Integer.parseInt(sc.nextLine());

GymMembership member = new GymMembership(id, name, type,
sessions);

System.out.println("Member ID: " + member.getMemberId());
System.out.println("Member Name: " + member.getMemberName());
System.out.printf("Final Monthly Fee: %.1f\n", member.getFinalFee());

}

sc.close();
}

}
```

Status : Skipped

Marks : 0/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

REC_2028_OOPS using Java_Week 5_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Arjun is working as a developer for CityWater Supply Board, which wants to build a household water billing system.

Each household's water account has:

A Customer ID (integer) A Customer Name (string) Liters Consumed (double)

The water bill is calculated based on these rules:

For the first 500 liters 2 per liter For the next 500 liters (501–1000) 3 per liter
For liters above 1000 5 per liter If the total bill exceeds 3000, a 10% discount is applied on the final bill.

Arjun has been asked to implement this system using:

A class with attributes for customer details. A constructor to initialize customer details. Setter methods to update details if needed. Getter methods to retrieve details. Objects of the class to represent customers.

Finally, display each customer's details and final bill amount.

Input Format

The first line of input contains an integer N, representing the number of customers.

For each customer:

- The next line contains the Customer ID (integer).
- The following line contains the Customer Name (string).
- The next line contains the Liters Consumed (double).

Output Format

For each customer, print the details in the following format:

Customer ID: <customer_id>

Customer Name: <customer_name>

Final Bill: <final_bill> (rounded to one decimal place)

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1

1001

Ravi Kumar

300

Output: Customer ID: 1001

Customer Name: Ravi Kumar

Final Bill: 600.0

Answer

```
import java.util.Scanner;
```

```
class WaterAccount
```

```
{
```

```
    private int customerId;  
    private String customerName;  
    private double litersConsumed;  
    private double finalBill;
```

```
    public WaterAccount(int customerId, String customerName, double  
    litersConsumed)
```

```
{
```

```
    this.customerId = customerId;  
    this.customerName = customerName;  
    this.litersConsumed = litersConsumed;  
    calculateFinalBill();
```

```
}
```

```
    private void calculateFinalBill()
```

```
{
```

```
    double bill = 0;  
    double consumed = this.litersConsumed;
```

```
    // Tier 3: Above 1000 liters (Rs 5/liter)  
    if (consumed > 1000)
```

```
{
```

```
    double tier3Liters = consumed - 1000;  
    bill += tier3Liters * 5.0;  
    consumed = 1000;
```

```
}
```

```
// Tier 2: 501 to 1000 liters (Rs 3/liter)  
if (consumed > 500)
```

```
{
```

```
    double tier2Liters = consumed - 500;  
    bill += tier2Liters * 3.0;  
    consumed = 500;
```

```
}
```

```
// Tier 1: 0 to 500 liters (Rs 2/liter)  
if (consumed > 0)
```

```
{
```

```
    bill += consumed * 2.0;
```

```
}
```

```
// Apply 10% discount if total bill exceeds 3000  
if (bill > 3000)
```

```
{
```

```
    bill *= 0.90; // Apply 10% discount
```

```
}
```

```
this.finalBill = bill;
```

```
}
```

```
// Getters
```

```
public int getCustomerId()
```

```
{
```

```
    return customerId;
```

```
}
```

```
public String getCustomerName()
```

```
{
```

```
    return customerName;
```

```
}
```

```
public double getFinalBill()
```

```
{
```

```
    return finalBill;
```

```
}
```

```
}
```

```
public class Main
```

```
{
```

```
    public static void main(String[] args)
```



```
{  
  
    Scanner sc = new Scanner(System.in);  
  
    int n = Integer.parseInt(sc.nextLine());  
  
    for (int i = 0; i < n; i++)  
  
    {  
  
        int id = Integer.parseInt(sc.nextLine());  
        String name = sc.nextLine();  
        double liters = Double.parseDouble(sc.nextLine());  
  
        WaterAccount customer = new WaterAccount(id, name, liters);  
  
        System.out.println("Customer ID: " + customer.getId());  
        System.out.println("Customer Name: " + customer.getName());  
        // Output format requires rounding to one decimal place  
        System.out.printf("Final Bill: %.1f\n", customer.getFinalBill());  
  
    }  
  
    sc.close();  
}  
  
}
```

Status : Skipped

Marks : 0/10

2. Problem Statement

Meera is working as a developer for CityGas Supply Board, which wants to build a household gas billing system.

Each household's gas account has:

A Customer ID (integer) A Customer Name (string) Units Consumed in cubic meters (double)

The gas bill is calculated based on these rules:

For the first 50 units 4 per unit For the next 100 units (51–150) 6 per unit For units above 150 8 per unit If the total bill exceeds 2000, a 15% discount is applied on the final bill.

Meera has been asked to implement this system using:

A class with attributes for customer details. A constructor to initialize customer details. Setter methods to update details if needed. Getter methods to retrieve details. Objects of the class to represent customers.

Finally, display each customer's details and final bill amount.

Input Format

The first line of input contains an integer N, representing the number of customers.

For each customer:

- The next line contains the Customer ID (integer).
- The following line contains the Customer Name (string).
- The next line contains the Units Consumed (double).

Output Format

For each customer, print the details in the following format:

Customer ID: <customer_id>

Customer Name: <customer_name>

Final Bill: <final_bill> (The final bill must be rounded to one decimal place.)

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1

1001

Ravi Kumar

30

Output: Customer ID: 1001

Customer Name: Ravi Kumar

Final Bill: 120.0

Answer

```
import java.util.Scanner;
```

```
class GasAccount
```

```
{
```

```
    private int customerId;  
    private String customerName;  
    private double unitsConsumed;  
    private double finalBill;
```

```
    public GasAccount(int customerId, String customerName, double  
unitsConsumed)
```

```
{
```

```
    this.customerId = customerId;  
    this.customerName = customerName;  
    this.unitsConsumed = unitsConsumed;  
    calculateFinalBill();
```

```
}
```

```
    private void calculateFinalBill()
```

```
{
```

```
    double bill = 0;  
    double consumed = this.unitsConsumed;
```

```
// Tier 3: Above 150 units (Rs 8/unit)
if (consumed > 150)
```

```
{
```

```
    double tier3Units = consumed - 150;
    bill += tier3Units * 8.0;
    consumed = 150;
```

```
}
```

```
// Tier 2: 51 to 150 units (Rs 6/unit)
if (consumed > 50)
```

```
{
```

```
    double tier2Units = consumed - 50;
    bill += tier2Units * 6.0;
    consumed = 50;
```

```
}
```

```
// Tier 1: 0 to 50 units (Rs 4/unit)
if (consumed > 0)
```

```
{
```

```
    bill += consumed * 4.0;
```

```
}
```

```
// Apply 15% discount if total bill exceeds 2000
if (bill > 2000)
```

```
{
```

```
bill *= 0.85; // Apply 15% discount
```

```
}
```

```
this.finalBill = bill;
```

```
}
```

```
// Getters
```

```
public int getCustomerId()
```

```
{
```

```
return customerId;
```

```
}
```

```
public String getCustomerName()
```

```
{
```

```
return customerName;
```

```
}
```

```
public double getFinalBill()
```

```
{
```

```
return finalBill;
```

```
}
```

```
}
```

```
public class Main
```

```
{
```

```
    public static void main(String[] args)
```

```
{
```

```
    Scanner sc = new Scanner(System.in);
```

```
    int n = Integer.parseInt(sc.nextLine());
```

```
    for (int i = 0; i < n; i++)
```

```
{
```

```
    int id = Integer.parseInt(sc.nextLine());
```

```
    String name = sc.nextLine();
```

```
    double units = Double.parseDouble(sc.nextLine());
```

```
    GasAccount customer = new GasAccount(id, name, units);
```

```
    System.out.println("Customer ID: " + customer.getId());
```

```
    System.out.println("Customer Name: " + customer.getName());
```

```
    // Output format requires rounding to one decimal place
```

```
    System.out.printf("Final Bill: %.1f\n", customer.getFinalBill());
```

```
}
```

```
    sc.close();
```

```
}
```

```
}
```

Status : Skipped

Marks : 0/10

3. Problem Statement

Anjali is working as a developer for the City Basketball Association, which wants to build a system to track and find the top scorer among basketball players.

Each player's record has:

Player ID (integer) Player Name (string) An array of points scored in 5 matches (integers)

The system must calculate:

The total score of each player (sum of all match points). Identify the highest scorer among all players. If two or more players have the same total score, the one with the lower Player ID is considered the top scorer.

Anjali has been asked to implement this system using:

A class with attributes for player details. A constructor to initialize player details. Getter and Setter methods to retrieve and update player details if required. A method to calculate the total score. Objects of the class to represent players.

Finally, display each player's details and announce the Top Scorer.

Input Format

The first line of input contains an integer N (number of players).

For each player:

- The next line contains the Player ID (integer).
- The following line contains the Player Name (string).
- The next line contains 5 integers separated by spaces (points scored in 5 matches).

Output Format

For each player the output prints the following details:

- Player ID: <player_id>
- Player Name: <player_name>
- Total Score: <total_score>

Finally, print "Top Scorer: <player_name> with <total_score> points"

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1

1001

Ravi Kumar

10 20 30 40 50

Output: Player ID: 1001

Player Name: Ravi Kumar

Total Score: 150

Top Scorer: Ravi Kumar with 150 points

Answer

```
import java.util.Scanner;  
import java.util.ArrayList;  
import java.util.Arrays;  
import java.util.Comparator;
```

```
class BasketballPlayer
```

```
{
```

```
    private int playerId;  
    private String playerName;  
    private int[] matchScores;  
    private int totalScore;
```

```
    public BasketballPlayer(int playerId, String playerName, int[] matchScores)
```



```
{  
    this.playerId = playerId;  
    this.playerName = playerName;  
    this.matchScores = matchScores;  
    calculateTotalScore();  
}
```

```
private void calculateTotalScore()
```

```
{  
    this.totalScore = Arrays.stream(matchScores).sum();  
}
```

```
public int getPlayerId()
```

```
{  
    return playerId;  
}
```

```
public String getPlayerName()
```

```
{  
    return playerName;  
}
```

```
public int getTotalScore()
```

```
{
```

```
    return totalScore;
```

```
}
```

```
}
```

```
public class Main
```

```
{
```

```
    public static void main(String[] args)
```

```
{
```

```
    Scanner sc = new Scanner(System.in);
```

```
    int n = Integer.parseInt(sc.nextLine());
```

```
    ArrayList<BasketballPlayer> players = new ArrayList<>();
```

```
    for (int i = 0; i < n; i++)
```

```
{
```

```
        int id = Integer.parseInt(sc.nextLine());
```

```
        String name = sc.nextLine();
```

```
        String[] scoreStrings = sc.nextLine().split(" ");
```

```
        int[] scores = new int[5];
```

```
        for(int j = 0; j < 5; j++)
```

```
{
```

```
scores[j] = Integer.parseInt(scoreStrings[j]);
```

```
}
```

```
BasketballPlayer player = new BasketballPlayer(id, name, scores);  
players.add(player);
```

```
System.out.println("Player ID: " + player.getPlayerId());  
System.out.println("Player Name: " + player.getPlayerName());  
System.out.println("Total Score: " + player.getTotalScore());
```

```
}
```

```
BasketballPlayer finalTopScorer = null;  
int maxScore = -1;  
int minIdWithMaxScore = Integer.MAX_VALUE;
```

```
for (BasketballPlayer player : players)
```

```
{
```

```
    if (player.getTotalScore() > maxScore)
```

```
    {
```

```
        maxScore = player.getTotalScore();  
        minIdWithMaxScore = player.getPlayerId();  
        finalTopScorer = player;
```

```
    } else if (player.getTotalScore() == maxScore)
```

```
    {
```

```
        if (player.getPlayerId() < minIdWithMaxScore)
```

```
        {
```

```
        minIdWithMaxScore = player.getPlayerId();
        finalTopScorer = player;

    }

}

}

if (finalTopScorer != null)
{

    System.out.println("Top Scorer: " + finalTopScorer.getPlayerName() + "
with " + finalTopScorer.getTotalScore() + " points");

}

    sc.close();
}

}
```

Status : Skipped

Marks : 0/10

4. Problem Statement

Anjali is now working as a developer for the City Marathon Association, which wants to build a system to track and find the fastest runner among marathon participants.

Each runner's record has:

Runner ID (integer) Runner Name (string) An array of times (in minutes) taken in 5 marathon events (integers)

The system must calculate:

The average time of each runner (sum of all times / 5). Identify the fastest runner (the one with the lowest average time). If two or more runners have the same average time, the one with the lower Runner ID is considered the fastest runner.

Anjali has been asked to implement this system using:

A class with attributes for runner details. A constructor to initialize runner details. Getter and Setter methods to retrieve and update runner details if required. A method to calculate the average time. Objects of the class to represent runners.

Finally, display each runner's details and announce the Fastest Runner.

Input Format

The first line of input contains an integer N (number of runners).

For each runner:

- The next line contains the Runner ID (integer).
- The following line contains the Runner Name (string).
- The next line contains 5 integers separated by spaces (times in minutes for 5 marathon events).

Output Format

For each runner the output prints the following details:

- Runner ID: <runner_id>
- Runner Name: <runner_name>
- Average Time: <average_time>

Finally, print "Fastest Runner: <runner_name> with <average_time> minutes"

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1

1001

Ravi Kumar

240 250 245 255 260

Output: Runner ID: 1001

Runner Name: Ravi Kumar

Average Time: 250

Fastest Runner: Ravi Kumar with 250 minutes

Answer

```
import java.util.Scanner;  
import java.util.ArrayList;
```

```
class MarathonRunner
```

```
{
```

```
    private int runnerId;  
    private String runnerName;  
    private int[] eventTimes;  
    private double averageTime;
```

```
    public MarathonRunner(int runnerId, String runnerName, int[] eventTimes)
```

```
{
```

```
        this.runnerId = runnerId;  
        this.runnerName = runnerName;  
        this.eventTimes = eventTimes;  
        calculateAverageTime();
```

```
}
```

```
private void calculateAverageTime()
```

```
{
```

```
    int totalTime = 0;
```

```
    for (int time : this.eventTimes)
```

```
    {
```

```
        totalTime += time;
```

```
    }
```

```
    this.averageTime = (double) totalTime / 5.0;
```

```
}
```

```
public int getRunnerId()
```

```
{
```

```
    return runnerId;
```

```
}
```

```
public String getRunnerName()
```

```
{
```

```
    return runnerName;
```

```
}
```

```
public double getAverageTime()
```

```
{
```

```
    return averageTime;
```

```
}
```

```
}
```

```
public class Main
```

```
{
```

```
    public static void main(String[] args)
```

```
{
```

```
        Scanner sc = new Scanner(System.in);
```

```
        int n = Integer.parseInt(sc.nextLine());
```

```
        ArrayList<MarathonRunner> runners = new ArrayList<>();
```

```
        for (int i = 0; i < n; i++)
```

```
        {
```

```
            int id = Integer.parseInt(sc.nextLine());
```

```
            String name = sc.nextLine();
```

```
            String[] timeStrings = sc.nextLine().split(" ");
```

```
            int[] times = new int[5];
```

```
            for(int j = 0; j < 5; j++)
```

```
            {
```



```
times[j] = Integer.parseInt(timeStrings[j]);
```

```
}
```

```
MarathonRunner runner = new MarathonRunner(id, name, times);  
runners.add(runner);
```

```
System.out.println("Runner ID: " + runner.getRunnerId());  
System.out.println("Runner Name: " + runner.getRunnerName());  
System.out.printf("Average Time: %.0f\n", runner.getAverageTime());
```

```
}
```

```
MarathonRunner fastestRunner = null;  
double minAverageTime = Double.MAX_VALUE;  
int minIdWithMinTime = Integer.MAX_VALUE;
```

```
for (MarathonRunner runner : runners)
```

```
{
```

```
    if (runner.getAverageTime() < minAverageTime)
```

```
    {
```

```
        minAverageTime = runner.getAverageTime();  
        minIdWithMinTime = runner.getRunnerId();  
        fastestRunner = runner;
```

```
    } else if (runner.getAverageTime() == minAverageTime)
```

```
    {
```

```
        if (runner.getRunnerId() < minIdWithMinTime)
```

```
        {
```

```
minIdWithMinTime = runner.getRunnerId();  
fastestRunner = runner;
```

```
}
```

```
}
```

```
}
```

```
if (fastestRunner != null)
```

```
{
```

```
    System.out.printf("Fastest Runner: %s with %.0f minutes\n",  
        fastestRunner.getRunnerName(), fastestRunner.getAverageTime());
```

```
}
```

```
    sc.close();
```

```
}
```

```
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

REC_2028_OOPS using Java_Week 6_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 14

Section 1 : MCQ

1. What will be the output of the following program?

```
class Vehicle {  
    String type = "Vehicle";  
}  
  
class Car extends Vehicle {  
    String type = "Car";  
}  
  
class Test {  
    public static void main(String[] args) {  
        Car c = new Car();  
        System.out.println(c.type);  
    }  
}
```

Answer

Car

Status : Correct

Marks : 1/1

2. What will be the output of the following program?

```
class A {  
    int x = 10;  
}  
  
class B extends A {  
    int x = 20;  
}  
  
class C extends B {  
    int x = 30;  
  
    void display() {  
        System.out.println(x);  
        System.out.println(super.x);  
    }  
}  
  
class Test {  
    public static void main(String[] args) {  
        C obj = new C();  
        obj.display();  
    }  
}
```

Answer

3020

Status : Correct

Marks : 1/1

3. What will be the output of the following code?

```
class A {  
    void display() {  
        System.out.println("Display A");  
    }  
}  
  
class B extends A {  
    void display() {  
        System.out.println("Display B");  
    }  
}  
  
class C extends B {  
    void display() {  
        super.display();  
    }  
}  
  
class Test {  
    public static void main(String[] args) {  
        C obj = new C();  
        obj.display();  
    }  
}
```

Answer

Display B

Status : Correct

Marks : 1/1

4. What will be the output of the following Java program?

```
class Vehicle {  
    void startEngine() {  
        System.out.println("Vehicle engine started");  
    }  
}  
  
class Car extends Vehicle {
```

```
void startEngine() {  
    System.out.println("Car engine started");  
}  
}
```

```
class Main {  
    public static void main(String[] args) {  
        Vehicle myVehicle = new Car();  
        myVehicle.startEngine();  
    }  
}
```

Answer

Car engine started

Status : Correct

Marks : 1/1

5. What will be the output of the following Java program?

```
class Parent {  
    void show() {  
        System.out.println("Parent class");  
    }  
}  
class Child extends Parent {  
    void show() {  
        System.out.println("Child class");  
    }  
}  
class Test {  
    public static void main(String[] args) {  
        Parent obj = new Child();  
        obj.show();  
    }  
}
```

Answer

Child class

Status : Correct

Marks : 1/1

6. What will be the output of the following Java program?

```
class A {  
    void display() {  
        System.out.println("Class A");  
    }  
}  
  
class B extends A {  
    void show() {  
        System.out.println("Class B");  
    }  
}  
  
class C extends B {  
    void print() {  
        System.out.println("Class C");  
    }  
}  
  
class Test {  
    public static void main(String[] args) {  
        C obj = new C();  
        obj.display();  
        obj.show();  
        obj.print();  
    }  
}
```

Answer

Class A
Class B
Class C

Status : Correct

Marks : 1/1

7. What will be the output of the following program?

```
class A {  
    public int i;  
    private int j;
```

```

    }
    class B extends A {
        void display() {
            super.j = super.i + 1;
            System.out.println(super.i + " " + super.j);
        }
    }
    class inheritance {
        public static void main(String args[]) {
            B obj = new B();
            obj.i=1;
            obj.j=2;
            obj.display();
        }
    }
}

```

Answer

Compile Time Error

Status : Correct

Marks : 1/1

8. What will be the output of the following Java program?

```

class Vehicle {
    void start() {
        System.out.println("Vehicle starts");
    }
}
class Car extends Vehicle {

    void start() {
        System.out.println("Car starts");
    }
}
class ElectricCar extends Car {
    void start() {
        System.out.println("Electric Car starts silently");
    }
}

```



```
class Test {  
    public static void main(String[] args) {  
        Vehicle v = new ElectricCar();  
        v.start();  
    }  
}
```

Answer

Electric Car starts silently

Status : Correct

Marks : 1/1

9. Which of the following is the correct way for class B to inherit from class A?

Answer

class B extends A {}

Status : Correct

Marks : 1/1

10. What will be the output of the following Java program?

```
class A {  
    int value = 10;  
    void display() {  
        System.out.println("A's display: " + value);  
    }  
}  
class B extends A {  
    int value = 20;  
    void display() {  
        System.out.println("B's display: " + value);  
    }  
}  
class Test {  
    public static void main(String[] args) {  
        A obj = new B();  
        obj.display();  
    }  
}
```

```
        System.out.println("Value: " + obj.value);
    }
}
```

Answer

B's display: 20 Value: 10

Status : Correct

Marks : 1/1

11. Which of the following is true about method overriding in Java?

Answer

The method must have the same name, same parameters, and must be in different classes with an inheritance relationship

Status : Correct

Marks : 1/1

12. Select the correct keyword for implementing inheritance through the class.

Answer

extends

Status : Correct

Marks : 1/1

13. What will be the output of the following Java program?

```
class Test {
    void display(int a, int b) {
        System.out.println("Method 1");
    }
    void display(double a, double b) {
        System.out.println("Method 2");
    }
    public static void main(String[] args) {
        Test obj = new Test();
        obj.display(10, 10.0);
    }
}
```

```
}
```

Answer

Compilation error

Status : Wrong

Marks : 0/1

14. What will be the output of the following Java program?

```
class Test {  
    void show(int a) {  
        System.out.println("Integer method");  
    }  
    void show(String s) {  
        System.out.println("String method");  
    }  
    public static void main(String[] args) {  
        Test obj = new Test();  
        obj.show(null);  
    }  
}
```

Answer

String method

Status : Correct

Marks : 1/1

15. What will be the output of the following code?

```
class A {  
    int sum(int x) {  
        return x + 2;  
    }  
}  
  
class B extends A {  
    int sum(int x) {  
        return super.sum(x) * 2;  
    }  
}
```

```
}
```

```
class C extends B {  
    int sum(int x) {  
        return super.sum(x) - 3;  
    }  
}
```

```
class Test {  
    public static void main(String[] args) {  
        C obj = new C();  
        System.out.println(obj.sum(4));  
    }  
}
```

Answer

9

Status : Correct

Marks : 1/1

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN

Email: 240701805@rajalakshmi.edu.in

Roll no: 240701805

Phone: 6379580366

Branch: REC

Department: CSE - Section 6

Batch: 2028

Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 6_Q1

Attempt : 1

Total Mark : 10

Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Elsa subscribes to a premium service with a base monthly cost, a service tax and an extra feature cost. Assist her in writing an inheritance program that takes input for these values and calculates the total monthly cost.

Refer to the below class diagram:

Input Format

The first line of input consists of a double value, representing the base monthly cost.

The second line consists of a double value, representing the service tax.

The third line consists of a double value, representing the extra feature cost.

Output Format

The output prints "Rs. X" where X is a double value, rounded off to two decimal places.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 10.0

2.5

5.0

Output: Rs. 17.50

Answer

```
import java.util.Scanner;
```

```
import java.util.Scanner;
```

```
class Subscription
```

```
{
```

```
    protected double baseMonthlyCost;
```

```
    protected double serviceTax;
```

```
    public Subscription(double baseMonthlyCost, double serviceTax)
```

```
{
```

```
        this.baseMonthlyCost = baseMonthlyCost;
```

```
        this.serviceTax = serviceTax;
```

```
}
```

```
    // Setters omitted for brevity/simplicity as they weren't strictly used in main logic
```

```
}
```

```
class PremiumSubscription extends Subscription
```

```
{
```

```
    private double extraFeatureCost;
```

```
    public PremiumSubscription(double baseMonthlyCost, double serviceTax,  
double extraFeatureCost)
```

```
{
```

```
    super(baseMonthlyCost, serviceTax);  
    this.extraFeatureCost = extraFeatureCost;
```

```
}
```

```
    public double calculateMonthlyCost()
```

```
{
```

```
        return this.baseMonthlyCost + this.serviceTax + this.extraFeatureCost;
```

```
}
```

```
}
```

```
class Solution
```

```
{
```

```
    // Changed from public class Solution to class Solution to avoid compilation  
issue
```

```
    public static void main(String[] args)
```

```
{
```

```

Scanner sc = new Scanner(System.in);

double baseMonthlyCost = Double.parseDouble(sc.nextLine());
double serviceTax = Double.parseDouble(sc.nextLine());
double extraFeatureCost = Double.parseDouble(sc.nextLine());

PremiumSubscription premiumSubscription = new
PremiumSubscription(baseMonthlyCost, serviceTax, extraFeatureCost);

double totalMonthlyCost = premiumSubscription.calculateMonthlyCost();

// Output format: Rs. X (rounded off to two decimal places)
System.out.printf("Rs. %.2f\n", totalMonthlyCost);

sc.close();

}

}

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        double baseMonthlyCost = scanner.nextDouble();
        double serviceTax = scanner.nextDouble();
        double extraFeatureCost = scanner.nextDouble();

        PremiumSubscription premiumSubscription = new
PremiumSubscription(baseMonthlyCost, serviceTax, extraFeatureCost);

        double totalMonthlyCost = premiumSubscription.calculateMonthlyCost();

        System.out.printf("Rs. %.2f\n", totalMonthlyCost);

        scanner.close();
    }
}

```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 6_Q2

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Alice is managing an online store and wants to implement a program using inheritance to calculate the selling price of products after applying discounts.

Guide her by following the instructions:

Create a base class called Product with a public double attribute price. Create a subclass called DiscountedProduct, which extends Product and includes a private double attribute discount rate. This subclass has a method called calculateSellingPrice() to determine the final selling price after applying the discount.

Formula: Discounted selling price = price * (1 - discount rate)

Input Format

The first line of input consists of a double value p, the initial price of the product.

The second line consists of a double value d, the discount rate.

Output Format

The output prints "Rs. X", where X is a double value, representing the calculated discounted selling price, rounded off to two decimal places.

If the discount rate is greater than 1, print "Not applicable".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 50.00

0.20

Output: Rs. 40.00

Answer

```
import java.util.Scanner;
```

```
import java.util.Scanner;
```

```
class Product
```

```
{
```

```
    public double price;
```

```
    public Product(double price)
```

```
{
```

```
        this.price = price;
```

```
}
```

```
}
```

```
class DiscountedProduct extends Product
```

```
{
```

```
    private double discountRate;
```

```
    public DiscountedProduct(double price, double discountRate)
```

```
{
```

```
        super(price);
```

```
        this.discountRate = discountRate;
```

```
}
```

```
    public double calculateSellingPrice()
```

```
{
```

```
        return price * (1.0 - this.discountRate);
```

```
}
```

```
}
```

```
class Solution
```

```
{
```

```
    public static void main(String[] args)
```

```
{
```

```
Scanner sc = new Scanner(System.in);

double price = Double.parseDouble(sc.nextLine());
double discountRate = Double.parseDouble(sc.nextLine());

if (discountRate > 1.0)

{

    System.out.println("Not applicable");

} else
{

    DiscountedProduct product = new DiscountedProduct(price,
discountRate);
    double finalPrice = product.calculateSellingPrice();

    System.out.printf("Rs. %.2f\n", finalPrice);

}

    sc.close();

}

}

class ProductPricing {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        double initialPrice = scanner.nextDouble();
        double discountRate = scanner.nextDouble();
        DiscountedProduct discountedProduct = new
DiscountedProduct(initialPrice, discountRate);
        double sellingPrice = discountedProduct.calculateSellingPrice();
```

```
    if (sellingPrice >= 0) {  
        System.out.printf("Rs. %.2f%n", sellingPrice);  
    } else {  
        System.out.println("Not applicable");  
    }  
    scanner.close();  
}  
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 6_Q3

Attempt : 1
Total Mark : 10
Marks Obtained : 6.5

Section 1 : Coding

1. Problem Statement

Preethi is working on a project to automate sales tax calculations for items in a store. She wants to create a program that takes the price of an item and the sales tax rate as input and calculates the final price of the item after applying the sales tax.

Write a program using the class SalesTaxCalculator, which contains an overloaded method named calculateFinalPrice to handle both integer and double inputs. The program should also include a Main class that takes user input, calls the appropriate method from SalesTaxCalculator, and prints the final price of the item.

Formula Used: Final price = price + ((price * sales tax rate) / 100)

Input Format

The first line of input consists of an integer price (the price of the item for integer inputs).

The second line of input consists of an integer taxRate (the sales tax rate for integer inputs).

The third line of input consists of a double price (the price of the item for double inputs).

The fourth line of input consists of a double taxRate (the sales tax rate for double inputs).

Output Format

The first line of output prints an integer, representing the final price of the item after applying the sales tax for integer inputs (a and b).

The second line prints a double value, representing the final price of the item after applying the sales tax for double-value inputs (m and n), rounded to two decimal places.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 100

10

100.0

5.0

Output: 110

105.00

Answer

```
import java.util.Scanner;
```

```
import java.util.Scanner;
```

```
class SalesTaxCalculator
```

```
{
```

```
public static int calculateFinalPrice(int price, int taxRate)
```

```
{
```

```
    double finalPrice = price + ((price * taxRate) / 100.0);  
    return (int) Math.round(finalPrice);
```

```
}
```

```
public static double calculateFinalPrice(double price, double taxRate)
```

```
{
```

```
    return price + ((price * taxRate) / 100.0);
```

```
}
```

```
}
```

```
class Solution
```

```
{
```

```
    public static void main(String[] args)
```

```
{
```

```
        Scanner sc = new Scanner(System.in);
```

```
        int intPrice = Integer.parseInt(sc.nextLine());  
        int intTaxRate = Integer.parseInt(sc.nextLine());
```

```
        double doublePrice = Double.parseDouble(sc.nextLine());  
        double doubleTaxRate = Double.parseDouble(sc.nextLine());
```



```

        sc.close();

        int finalIntPrice = SalesTaxCalculator.calculateFinalPrice(intPrice,
intTaxRate);
        System.out.println(finalIntPrice);

        double finalDoublePrice =
SalesTaxCalculator.calculateFinalPrice(doublePrice, doubleTaxRate);
        System.out.printf("%.2f\n", finalDoublePrice);

    }

}

class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int intPrice = scanner.nextInt();
        int intTaxRate = scanner.nextInt();
        double doublePrice = scanner.nextDouble();
        double doubleTaxRate = scanner.nextDouble();

        int finalPriceInt = SalesTaxCalculator.calculateFinalPrice(intPrice,
intTaxRate);
        double finalPriceDouble =
SalesTaxCalculator.calculateFinalPrice(doublePrice, doubleTaxRate);

        System.out.println(finalPriceInt);
        System.out.format("%.2f", finalPriceDouble);
    }
}

```

Status : Partially correct

Marks : 6.5/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 6_Q4

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Mr.Kapoor wants to create a program to calculate the volume of a Cuboid and a Cube using method overriding.

Implements a base class Cuboid with attributes for length, width, and height. Include a method calculateVolume() that computes the volume of the cuboid.

Extends the base class with a subclass Cube representing a cube, where all sides are equal. Override the calculateVolume() method in the Cube class to compute the volume of the cube.

The program should take user input for the dimensions of the cuboid and the side length of the cube and display the calculated volumes with two decimal places.

Input Format

The first line of input consists of 3 space-separated double values, representing the cuboid length, width, and height, respectively.

The second line consists of a double value, representing the side length of the cube.

Output Format

The first line of output prints the volume of the cuboid, rounded off to two decimal places.

The second line prints the volume of the cube, rounded off to two decimal places.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 60.0 60.0 60.0
50.0

Output: Volume of Cuboid: 216000.00
Volume of Cube: 125000.00

Answer

```
import java.util.Scanner;  
import java.util.Scanner;
```

```
class Cuboid
```

```
{
```

```
    private double length;  
    private double width;  
    private double height;
```

```
    public Cuboid(double length, double width, double height)
```

```
{  
    this.length = length;  
    this.width = width;  
    this.height = height;  
  
}
```

```
    public double calculateVolume()
```

```
{  
    return length * width * height;  
  
}
```

```
class Cube extends Cuboid
```

```
{  
    private double side;  
    public Cube(double side)  
  
    {
```

```
        super(side, side, side);  
        this.side = side;
```

```
    }  
    @Override  
    public double calculateVolume()
```

```
{  
    return side * side * side;  
}  
}
```

```
class Solution
```

```
{  
    public static void main(String[] args)  
    {  
  
        Scanner sc = new Scanner(System.in);  
  
        double cuboidLength = sc.nextDouble();  
        double cuboidWidth = sc.nextDouble();  
        double cuboidHeight = sc.nextDouble();  
  
        double cubeSide = sc.nextDouble();  
  
        sc.close();  
  
        Cuboid cuboid = new Cuboid(cuboidLength, cuboidWidth, cuboidHeight);  
        double cuboidVolume = cuboid.calculateVolume();  
  
        Cube cube = new Cube(cubeSide);  
        double cubeVolume = cube.calculateVolume();  
  
        System.out.printf("Volume of Cuboid: %.2f\n", cuboidVolume);  
        System.out.printf("Volume of Cube: %.2f\n", cubeVolume);  
    }  
}
```

```
}  
public class Main {  
    public static void main(String[] args) {  
        Scanner scanner = new Scanner(System.in);  
  
        double cuboidLength = scanner.nextDouble();  
        double cuboidWidth = scanner.nextDouble();  
        double cuboidHeight = scanner.nextDouble();  
  
        // Regular object instantiation for Cuboid  
        Cuboid cuboid = new Cuboid(cuboidLength, cuboidWidth, cuboidHeight);  
        System.out.printf("Volume of Cuboid: %.2f\n", cuboid.calculateVolume());  
  
        double cubeSide = scanner.nextDouble();  
  
        // Upcasting - Using superclass reference for subclass object (DMD)  
        Cuboid cube = new Cube(cubeSide); // Upcasting  
        System.out.printf("Volume of Cube: %.2f", cube.calculateVolume()); // Calls  
        Cube's method dynamically  
  
        scanner.close();  
    }  
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 6_Q5

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem statement:

Tim was tasked with developing a grocery shopping app. You have a class hierarchy that includes Item, Produce, and OrganicProduce. Your goal is to calculate the total cost of a shopping list, which may contain a mix of regular produce and organic produce items. Additionally, you need to apply discounts to organic items. Apply a 10% discount on organic produce items

Class Hierarchy:

Item: Base class for all items.

Produce: Subclass of Item for regular produce items.

OrganicProduce: Subclass of Produce for organic produce items.

Input Format

The first line of input consists of an integer, 'n'.

For each 'n' item, the user will provide:

- A string 'type' representing the item type ('Regular' or 'Organic').
- A string 'name' represents the item name.
- A double 'price' represents the item price.

Output Format

The output will display the total cost of the shopping list, including discounts on organic items.

Refer to the sample output for format specifications.

Sample Test Case

Input: 1

Regular Banana 1.99

Output: 1.99

Answer

```
import java.util.Scanner;
```

```
import java.util.Scanner;
```

```
class Item
```

```
{
```

```
    protected String name;  
    protected double price;
```

```
    public Item(String name, double price)
```

```
{
```



```
this.name = name;  
this.price = price;
```

```
}
```

```
public double getPrice()
```

```
{
```

```
    return price;
```

```
}
```

```
public double calculateCost()
```

```
{
```

```
    return price;
```

```
}
```

```
}
```

```
class Produce extends Item
```

```
{
```

```
    public Produce(String name, double price)
```

```
{
```

```
        super(name, price);
```

```
}
```

```
@Override
public double calculateCost()
```

```
{
```

```
    // No discount for regular produce
    return price;
```

```
}
```

```
}
```

```
class OrganicProduce extends Produce
```

```
{
```

```
    private static final double DISCOUNT_RATE = 0.10; // 10% discount
```

```
    public OrganicProduce(String name, double price)
```

```
{
```

```
        super(name, price);
```

```
}
```

```
@Override
public double calculateCost()
```

```
{
```

```
    // Apply 10% discount to organic items
    return price * (1 - DISCOUNT_RATE);
```

```
}
```

```
}
```

```
class Solution
```

```
{
```

```
    public static void main(String[] args)
```

```
{
```

```
    Scanner sc = new Scanner(System.in);
```

```
    // Read number of items
```

```
    int n = sc.nextInt();
```

```
    sc.nextLine(); // Consume the newline
```

```
    double totalCost = 0.0;
```

```
    for (int i = 0; i < n; i++)
```

```
{
```

```
    String inputLine = sc.nextLine();
```

```
    // Split input: Type, Name, Price
```

```
    String[] parts = inputLine.split(" ");
```

```
    String type = parts[0];
```

```
    String name = parts[1];
```

```
    double price = Double.parseDouble(parts[2]);
```

```
    Item item;
```

```
    if (type.equalsIgnoreCase("Organic"))
```

```
{
```

```
        item = new OrganicProduce(name, price);
```

```
    } else
```

```
    {
```

```
        // Assumes "Regular"
```

```
        item = new Produce(name, price);
```

```
    }
```

```
    // Calculate and accumulate the cost  
    totalCost += item.calculateCost();
```

```
}
```

```
sc.close();
```

```
// Output the total cost rounded to two decimal places  
System.out.printf("%.2f\n", totalCost);
```

```
}
```

```
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);
```

```
        int n = sc.nextInt();  
        sc.nextLine(); // Consume newline
```

```
        double totalCost = 0.0;
```

```
        for (int i = 0; i < n; i++) {  
            String type = sc.next();  
            String name = sc.next();  
            double price = sc.nextDouble();
```

```
        if (type.equals("Regular")) {  
            Item item = new Produce(name, price);  
            totalCost += item.calculateCost();  
        } else if (type.equals("Organic")) {  
            Item item = new OrganicProduce(name, price);  
            totalCost += item.calculateCost();  
        }  
    }  
  
    System.out.printf("%.2f%n", totalCost);  
}  
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

REC_2028_OOPS using Java_Week 6_PAH

Attempt : 1
Total Mark : 40
Marks Obtained : 30

Section 1 : Coding

1. Problem Statement

Sharon, a software developer, is working on a project to automate velocity calculations for various objects. She wants to create a class named VelocityCalculator with overloaded methods calculateVelocity to calculate the velocity. One method will accept distance in meters and time in seconds as integers, while another will accept distance and time as doubles.

Help her in completing the project.

Formula: $\text{Velocity} = \text{distance} / \text{time}$

Input Format

The first line of input consists of an integer, representing the distance in meters

(for the integer method).

The second line consists of an integer, representing the time in seconds (for the integer method).

The third line consists of a double value, representing the distance in meters (for the double method).

The fourth line consists of a double value, representing the time in seconds (for the double method).

Output Format

The first line prints the velocity calculated using the integer inputs in the format:

Velocity with integer inputs: <velocity> m/s

The second line prints the velocity calculated using the double inputs in the format:

Velocity with double inputs: <velocity> m/s

Note:

The velocity for the double inputs should be printed with two decimal places.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 100

10

100.5

10.2

Output: Velocity with integer inputs: 10 m/s

Velocity with double inputs: 9.85 m/s

Answer

```
import java.util.Scanner;
```

```
import java.util.Scanner;
```

```
class VelocityCalculator
```

```
{
```

```
    public static int calculateVelocity(int distance, int time)
```

```
{
```

```
        return distance / time;
```

```
}
```

```
    public static double calculateVelocity(double distance, double time)
```

```
{
```

```
        return distance / time;
```

```
}
```

```
}
```

```
class Solution
```

```
{
```

```
    public static void main(String[] args)
```

```
{
```

```
        Scanner sc = new Scanner(System.in);
```



```

int intDistance = sc.nextInt();
int intTime = sc.nextInt();

double doubleDistance = sc.nextDouble();
double doubleTime = sc.nextDouble();

sc.close();

int velocityInt = VelocityCalculator.calculateVelocity(intDistance, intTime);

double velocityDouble =
VelocityCalculator.calculateVelocity(doubleDistance, doubleTime);

System.out.println("Velocity with integer inputs: " + velocityInt + " m/s");
System.out.printf("Velocity with double inputs: %.2f m/s\n", velocityDouble);
}
}

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        int distanceInt = scanner.nextInt();
        int timeInt = scanner.nextInt();

        double distanceDouble = scanner.nextDouble();
        double timeDouble = scanner.nextDouble();

        int velocityInt = VelocityCalculator.calculateVelocity(distanceInt, timeInt);
        double velocityDouble =
VelocityCalculator.calculateVelocity(distanceDouble, timeDouble);

        System.out.println("Velocity with integer inputs: " + velocityInt + " m/s");
        System.out.printf("Velocity with double inputs: %.2f m/s", velocityDouble);

        scanner.close();
    }
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Ram is designing a program to calculate the Body Mass Index (BMI). Your task is to assist him by following the given specifications.

Create a base class BMIcalculator with a method calculateBMI() to compute BMI using the formula $\text{weight} / (\text{height} * \text{height})$.

Extend the class with a subclass CustomBMIcalculator that overrides the method calculateBMI() to calculate BMI based on custom criteria, assigning categories such as "Underweight," "Normal Weight," "Overweight," or "Obese."

BMI < 18.5, category = "Underweight" BMI >= 18.5 & < 24.9, category = "Normal Weight" BMI >= 25 & < 29.9, category = "Overweight" else category = "Obese"

Implement user input for weight and height and display both the standard and custom BMI calculations.

Input Format

The first line of input consists of a double value, representing the weight in kgs.

The second line consists of a double value, representing the height in meters.

Output Format

The first line of output prints: "Standard BMI Calculation:"

The second line of output prints: "BMI: " followed by the calculated BMI value (to two decimal places).

The third line of output prints: "Custom BMI Calculation:"

The fourth line of output prints: "Category: " followed by the BMI category.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 69.7

2.6

Output: Standard BMI Calculation:

BMI: 10.31

Custom BMI Calculation:

Category: Underweight

Answer

```
import java.util.Scanner;
```

```
// You are using Java
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        Scanner scanner = new Scanner(System.in);
```

```
        double weight = scanner.nextDouble();
```

```
        double height = scanner.nextDouble();
```

```
        BMlcalculator bmiCalculator = new BMlcalculator(weight, height);
```

```
        System.out.println("Standard BMI Calculation:");
```

```
        bmiCalculator.displayBMI();
```

```
        CustomBMlcalculator customBMlcalculator = new  
CustomBMlcalculator(weight, height);
```

```
        System.out.println("Custom BMI Calculation:");
```

```
        customBMlcalculator.displayCustomBMI();
```

```
        scanner.close();
```

```
    }  
}
```

Status : Wrong

Marks : 0/10

3. Problem Statement

In a company, each manager has a unique employee ID and a monthly salary. You are required to design a program that will calculate and display the annual(12 months) salary of a manager based on the input details provided by the user.

Implement the solution using a single inheritance approach.

Employee: The base class with attributes name and employeeID.

Manager: The derived class inheriting from Employee, with an additional attribute salary.

Input Format

The first line of input consists of a string name, representing the manager's name.

The second line of input consists of an integer employeeID, representing the manager's employee ID.

The third line of input consists of a double salary, representing the manager's monthly salary.

Output Format

The first line of output prints: Name: <name>

The second line of output prints: Annual Salary: Rs. <annual_salary> (rounded to two decimal places).

Refer to the sample output for formatting specifications.

Sample Test Case

Input: Davis

234

28750.75

Output: Name: Davis

Annual Salary: Rs. 345009.00

Answer

```
import java.util.Scanner;  
import java.text.DecimalFormat;  
  
import java.util.Scanner;  
import java.text.DecimalFormat;
```

```
class Employee
```

```
{
```

```
    protected String name;  
    protected int employeeID;
```

```
    public Employee(String name, int employeeID)
```

```
{
```

```
        this.name = name;  
        this.employeeID = employeeID;
```

```
}
```

```
    public String getName()
```

```
{
```

```
        return name;
```

```
}
```

```
}
```

```
class Manager extends Employee
```

```
{
```

```
    private double salary;
```

```
    public Manager(String name, int employeeID, double salary)
```

```
{
```

```
super(name, employeeID);  
this.salary = salary;
```

```
}
```

```
public double calculateAnnualSalary()
```

```
{
```

```
    return this.salary * 12;
```

```
}
```

```
}
```

```
class Solution
```

```
{
```

```
    public static void main(String[] args)
```

```
{
```

```
    Scanner sc = new Scanner(System.in);
```

```
    String name = sc.nextLine();  
    int employeeID = sc.nextInt();  
    double monthlySalary = sc.nextDouble();
```

```
    sc.close();
```

```
    Manager manager = new Manager(name, employeeID, monthlySalary);
```

```
    double annualSalary = manager.calculateAnnualSalary();
```

```
    DecimalFormat df = new DecimalFormat("0.00");
```

```

String formattedSalary = df.format(annualSalary);

System.out.println("Name: " + manager.getName());
System.out.println("Annual Salary: Rs. " + formattedSalary);

}

}

class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        DecimalFormat df = new DecimalFormat("0.00");

        String name = scanner.nextLine();
        int employeeID = scanner.nextInt();
        double salary = scanner.nextDouble();

        Manager manager = new Manager(name, employeeID, salary);

        System.out.println("Name: " + manager.name);
        System.out.println("Annual Salary: Rs. " +
df.format(manager.calculateAnnualSalary()));

        scanner.close();
    }
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

John is planning a long road trip and wants to calculate the distance his car can travel based on its speed and fuel capacity. As John knows that different cars have different fuel efficiencies, he wants a program that can help him estimate the travel distance for any given car.

To do this, you are tasked with creating a program that calculates the travel distance of a car based on its speed and fuel capacity. The calculation is simple and follows the formula:

$\text{Travel Distance} = \text{Speed} * \text{Fuel Capacity}$

You need to model this system using a Vehicle class and a Car class. The Vehicle class will have attributes for the speed and fuel capacity, while the Car class will inherit from the Vehicle class and include a method to calculate the travel distance.

Input Format

The first line of input consists of a double value representing the speed of the car in km/h.

The second line of input consists of a double value representing the fuel capacity of the car in liters.

Output Format

The first line should print "Speed: X km/h", where X is the speed of the car, rounded to two decimal places.

The second line should print "Fuel Capacity: Y liters", where Y is the fuel capacity of the car, rounded to two decimal places.

The third line should print "Travel Distance: Z km", where Z is the total travel distance the car can cover, rounded to two decimal places.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 10.0

1.0

Output: Speed: 10.00 km/h

Fuel Capacity: 1.00 liters

Travel Distance: 10.00 km

Answer

```
import java.util.Scanner;  
import java.util.Scanner;  
import java.text.DecimalFormat;
```



```
class Vehicle
```

```
{
```

```
    protected double speed;  
    protected double fuelCapacity;
```

```
    public Vehicle(double speed, double fuelCapacity)
```

```
{
```

```
    this.speed = speed;  
    this.fuelCapacity = fuelCapacity;
```

```
}
```

```
}
```

```
class Car extends Vehicle
```

```
{
```

```
    public Car(double speed, double fuelCapacity)
```

```
{
```

```
        super(speed, fuelCapacity);
```

```
}
```

```
    public double calculateTravelDistance()
```

```
{
```

```
        return this.speed * this.fuelCapacity;  
    }  
}
```

```
    public String getFormattedSpeed(DecimalFormat df)  
{  
  
        return "Speed: " + df.format(this.speed) + " km/h";  
  
    }  
}
```

```
    public String getFormattedFuelCapacity(DecimalFormat df)  
{  
  
        return "Fuel Capacity: " + df.format(this.fuelCapacity) + " liters";  
  
    }  
}
```

```
class Solution  
{
```

```
    public static void main(String[] args)  
{
```

```
        Scanner sc = new Scanner(System.in);  
        DecimalFormat df = new DecimalFormat("0.00");  
        double speed = sc.nextDouble();  
        double fuelCapacity = sc.nextDouble();
```

```

        sc.close();

        Car car = new Car(speed, fuelCapacity);

        double travelDistance = car.calculateTravelDistance();

        System.out.println(car.getFormattedSpeed(df));
        System.out.println(car.getFormattedFuelCapacity(df));
        System.out.println("Travel Distance: " + df.format(travelDistance) + " km");

    }

}

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        double speed = scanner.nextDouble();
        double fuelCapacity = scanner.nextDouble();

        Car car = new Car(speed, fuelCapacity);

        System.out.println("Speed: " + String.format("%.2f", car.speed) + " km/h");
        System.out.println("Fuel Capacity: " + String.format("%.2f", car.fuelCapacity)
+ " liters");
        System.out.println("Travel Distance: " + String.format("%.2f",
car.calculateTravelDistance()) + " km");

        scanner.close();
    }
}

```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

REC_2028_OOPS using Java_Week 6_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 30

Section 1 : Coding

1. Problem Statement

A painter needs to determine the cost to paint different shapes based on their surface area. The program should be designed to handle the area of a sphere and calculate the total painting cost using the following formulas:

Area of sphere: $\text{Area} = 4 * \pi * r^2$ where $\pi = 3.14$
Total painting cost: $\text{Cost} = \text{cost per square meter} * \text{area of sphere}$

The program will consist of three classes:

Shape class: This class should set the shape type and radius.

Area class: This class should extend Shape to calculate the area.
Cost class: This class should extend Area to calculate the total painting cost.

Input Format

The input consists of a string representing the shape type, a double value

representing the radius, and another double value representing the cost per square meter on each line.

Output Format

For a valid shape type of "Sphere":

- The first line prints: "Area of Sphere is: <calculated_area>" rounded to two decimal places.
- The second line prints: "Cost to paint the shape is: <total_painting_cost>" rounded to two decimal places.

For any other shape types, print: "Invalid type".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: Sphere

3.4

5.8

Output: Area of Sphere is: 145.19

Cost to paint the shape is: 842.12

Answer

```
import java.util.Scanner;
```

```
import java.util.Scanner;
```

```
import java.text.DecimalFormat;
```

```
class Shape
```

```
{
```

```
    protected String shapeType;  
    protected double radius;
```

```
    public void setShape(String type, double r)
```

```
{
```

```
this.shapeType = type;  
this.radius = r;
```

```
}
```

```
public String getShapeType()
```

```
{
```

```
    return shapeType;
```

```
}
```

```
}
```

```
class Area extends Shape
```

```
{
```

```
    private double area;
```

```
    private final double PI = 3.14;
```

```
    public void calculateArea()
```

```
{
```

```
        if ("Sphere".equals(this.shapeType))
```

```
{
```

```
            this.area = 4 * PI * radius * radius;
```

```
        } else
```

```
{  
    this.area = 0.0;  
}
```

```
}  
  
public double getArea()
```

```
{  
    return area;  
}
```

```
}  
  
class Cost extends Area
```

```
{  
    private double costPerSquareMeter;  
    private double totalPaintingCost;  
  
    public void setCost(double cost)  
    {
```

```
        this.costPerSquareMeter = cost;
```

```
    }
```

```
public void calculateCost()
```

```
{
```

```
    if ("Sphere".equals(this.shapeType))
```

```
    {
```

```
        this.totalPaintingCost = this.costPerSquareMeter * getArea();
```

```
    } else
```

```
    {
```

```
        this.totalPaintingCost = 0.0;
```

```
    }
```

```
}
```

```
public double getTotalPaintingCost()
```

```
{
```

```
    return totalPaintingCost;
```

```
}
```

```
}
```

```
class Solution
```

```
{
```



```
public static void main(String[] args)
{

    Scanner sc = new Scanner(System.in);
    DecimalFormat df = new DecimalFormat("0.00");

    if (!sc.hasNextLine()) return;
    String shapeType = sc.nextLine();

    if (!sc.hasNextDouble()) return;
    double radius = sc.nextDouble();

    if (!sc.hasNextDouble()) return;
    double costPerSquareMeter = sc.nextDouble();

    // Check for valid type before proceeding with calculations
    if (!"Sphere".equals(shapeType))
    {

        System.out.println("Invalid type");
        return;
    }

    Cost painterCost = new Cost();

    // This is the line that your error log refers to, ensuring it uses the correct
    types.
    painterCost.setShape(shapeType, radius);
    painterCost.setCost(costPerSquareMeter);

    painterCost.calculateArea();
    painterCost.calculateCost();

    System.out.println("Area of Sphere is: " + df.format(painterCost.getArea()));
    System.out.println("Cost to paint the shape is: " +
    df.format(painterCost.getTotalPaintingCost()));
```

```

    }
}

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        String s = scanner.next();
        Cost shape = new Cost();
        shape.setShape(s, scanner);
        double costToPaint = scanner.nextDouble();
        shape.calculateArea();
        shape.setCost(costToPaint);
        shape.calculateCost();
    }
}

```

Status : Wrong

Marks : 0/10

2. Problem Statement

Bob has been tasked with creating a program using CircleUtils class to calculate and display the circumference and area of the circle.

The program should allow Bob to input the radius of a circle as both an integer and a double and compute both the circumference and area of the circle using separate overloaded methods:

calculateCircumference- To calculate the circumference using the formula $2 * 3.14 * radius$
 calculateArea- To calculate the area $3.14 * radius * radius$

Write a program to help Bob.

Input Format

The first line of input consists of an integer m, representing the radius of the circle as a whole number.

The second line consists of a double value n, representing the radius of the circle as a decimal number.

Output Format

The first line of output displays two space-separated double values, rounded to two decimal places, representing the circumference of the circle with the integer radius and the double radius, respectively.

The second line displays two space-separated double values, rounded to two decimal places, representing the area of the circle with the integer radius and the double radius, respectively.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5

3.50

Output: 31.40 21.98

78.50 38.47

Answer

```
import java.util.Scanner;
```

```
import java.util.Scanner;
```

```
import java.text.DecimalFormat;
```

```
class CircleUtils
```

```
{
```

```
    private static final double PI = 3.14;
```

```
    public static double calculateCircumference(int radius)
```

```
{
```

```
        return 2 * PI * radius;
```

```
}
```

```
public static double calculateCircumference(double radius)
```

```
{
```

```
    return 2 * PI * radius;
```

```
}
```

```
public static double calculateArea(int radius)
```

```
{
```

```
    return PI * radius * radius;
```

```
}
```

```
public static double calculateArea(double radius)
```

```
{
```

```
    return PI * radius * radius;
```

```
}
```

```
}
```

```
class Solution
```

```
{
```

```
    public static void main(String[] args)
```

```
{
```

```

Scanner sc = new Scanner(System.in);
DecimalFormat df = new DecimalFormat("0.00");

int m = sc.nextInt();
double n = sc.nextDouble();

double circ_m = CircleUtils.calculateCircumference(m);
double circ_n = CircleUtils.calculateCircumference(n);

double area_m = CircleUtils.calculateArea(m);
double area_n = CircleUtils.calculateArea(n);

System.out.print(df.format(circ_m) + " ");
System.out.println(df.format(circ_n));

System.out.print(df.format(area_m) + " ");
System.out.println(df.format(area_n));

}

}

class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        int radiusInt = scanner.nextInt();
        double radiusDouble = scanner.nextDouble();

        CircleUtils circleUtils = new CircleUtils();

        double circumferenceInt = circleUtils.calculateCircumference(radiusInt);
        double circumferenceDouble =
circleUtils.calculateCircumference(radiusDouble);
        double areaInt = circleUtils.calculateArea(radiusInt);
        double areaDouble = circleUtils.calculateArea(radiusDouble);

        System.out.format("%.2f %.2f\n", circumferenceInt, circumferenceDouble);
        System.out.format("%.2f %.2f", areaInt, areaDouble);

        scanner.close();
    }
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

A bank provides two types of deposit schemes: Fixed Deposits (FD) and Recurring Deposits (RD). Customers want to calculate the interest they can earn based on their selected scheme.

Develop a Java program using inheritance to compute the interest for FD and RD. The program should include:

A base class Account with attributes accountHolder and principalAmount, along with a method for interest calculation. A subclass FixedDeposit that calculates interest for FD. A subclass RecurringDeposit that calculates interest for RD.

Formulas Used:

Interest for FD: $(\text{principal amount} * \text{duration in years} * \text{rate of interest}) / 100$

Interest for RD: $(\text{maturity amount} * \text{duration in months} * \text{rate of interest}) / (12 * 100)$, where maturity amount = monthly deposit * duration in months.

Input Format

The first line of input consists of the choice (1 for FD, 2 for RD).

If the choice is 1, the following lines consist of account holder (string), principal amount (double), duration in years (int), and rate of interest (double).

If the choice is 2, the following lines consist of account holder (string), monthly deposit (int), duration in months (int), and rate of interest (double).

Output Format

The output prints the calculated interest with one decimal place in the following format.

For choice 1: "Interest for FD: <calculated interest >"

For choice 2: "Interest for FD: <calculated interest >"

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1

Alice

50000.56

5

6.5

Output: Interest for FD: 16250.2

Answer

```
import java.util.Scanner;
```

```
import java.util.Scanner;
```

```
import java.text.DecimalFormat;
```

```
class Account
```

```
{
```

```
    protected String accountHolder;
```

```
    protected double rateOfInterest;
```

```
    public Account(String accountHolder, double rateOfInterest)
```

```
{
```

```
        this.accountHolder = accountHolder;
```

```
        this.rateOfInterest = rateOfInterest;
```

```
}
```

```
    public double calculateInterest()
```

```
{  
    return 0.0;  
}  
}
```

```
class FixedDeposit extends Account
```

```
{  
    private double principalAmount;  
    private int durationYears;  
  
    public FixedDeposit(String accountHolder, double principalAmount, int  
durationYears, double rateOfInterest)  
  
    {  
  
        super(accountHolder, rateOfInterest);  
        this.principalAmount = principalAmount;  
        this.durationYears = durationYears;  
    }  
  
    @Override  
    public double calculateInterest()  
  
    {
```

```
        return (principalAmount * durationYears * rateOfInterest) / 100.0;
```

```
    }  
}
```



```
}
```

```
class RecurringDeposit extends Account
```

```
{
```

```
    private int monthlyDeposit;
```

```
    private int durationMonths;
```

```
    public RecurringDeposit(String accountHolder, int monthlyDeposit, int  
durationMonths, double rateOfInterest)
```

```
{
```

```
        super(accountHolder, rateOfInterest);
```

```
        this.monthlyDeposit = monthlyDeposit;
```

```
        this.durationMonths = durationMonths;
```

```
}
```

```
    @Override
```

```
    public double calculateInterest()
```

```
{
```

```
        double maturityAmount = (double)this.monthlyDeposit *  
this.durationMonths;
```

```
        return (maturityAmount * this.durationMonths * rateOfInterest) / (12.0 *  
100.0);
```

```
}
```

```
}
```

```
class Solution
```

```
{  
  
    public static void main(String[] args)  
  
    {  
  
        Scanner sc = new Scanner(System.in);  
        DecimalFormat df = new DecimalFormat("0.0");  
  
        if (!sc.hasNextInt()) return;  
        int choice = sc.nextInt();  
        sc.nextLine();  
  
        double interest = 0.0;  
  
        if (choice == 1)  
  
        {  
  
            String accountHolder = sc.nextLine();  
            double principalAmount = sc.nextDouble();  
            int durationYears = sc.nextInt();  
            double rateOfInterest = sc.nextDouble();  
  
            FixedDeposit fd = new FixedDeposit(accountHolder, principalAmount,  
            durationYears, rateOfInterest);  
            interest = fd.calculateInterest();  
  
            System.out.println("Interest for FD: " + df.format(interest));  
  
        } else if (choice == 2)  
  
        {  
  
            String accountHolder = sc.nextLine();  
            int monthlyDeposit = sc.nextInt();
```

```
int durationMonths = sc.nextInt();
double rateOfInterest = sc.nextDouble();

RecurringDeposit rd = new RecurringDeposit(accountHolder,
monthlyDeposit, durationMonths, rateOfInterest);
interest = rd.calculateInterest();

System.out.println("Interest for RD: " + df.format(interest));

} else

{

}

}

}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int choice = sc.nextInt();

        switch (choice) {
            case 1:
                sc.nextLine();
                String fdName = sc.nextLine();
                double fdPrincipal = sc.nextDouble();
                int fdDuration = sc.nextInt();
                double fdRate = sc.nextDouble();

                FixedDeposit fd = new FixedDeposit(fdName, fdPrincipal, fdDuration,
fdRate);
                System.out.printf("Interest for FD: %.1f", fd.calculateInterest());
                break;
```

```

case 2:
    sc.nextLine();
    String rdName = sc.nextLine();
    int rdDeposit = sc.nextInt();
    int rdDuration = sc.nextInt();
    double rdRate = sc.nextDouble();

    RecurringDeposit rd = new RecurringDeposit(rdName, rdDeposit,
rdDuration, rdRate);
    System.out.printf("Interest for RD: %.1f", rd.calculateInterest());
    break;

default:
    System.out.println("Invalid Choice");
}
}
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Teena is launching a new airline, Boeing747, and needs to calculate the total revenue generated from ticket sales based on the ticket cost and seat availability. Teena's airline offers two types of seats: regular and premium. The ticket cost and seat availability for both types of seats need to be considered for revenue calculation.

To help with this, Teena wants to implement a system using multilevel inheritance with three classes:

Airline: This class will have the ticket cost as an attribute and defines the method `setCost(double cost)` and `double getCost()`. **Indigo:** This class will extend **Airline** and add the seat availability attribute and defines the method `getSeatAvailability()` and `setSeatAvailability(int seatAvailability)`. **Boeing747:** This class will extend **Indigo** and include a method `calculateTotalRevenue()` based on the ticket cost and seat availability .

Teena needs to calculate the total revenue using the formula:

Total Revenue = ticket cost * seat availability

Help Teena implement this system for calculating the revenue of her airline.

Input Format

The first line of input consists of a double value, representing the flight's ticket cost.

The second line consists of an integer, representing seat availability.

Output Format

The first line of output prints "Ticket Cost: Rs. " followed by a double value representing the ticket cost rounded to one decimal place.

The second line of output prints "Seat Availability: X seats" where X is an integer value representing the seat availability.

The third line of output prints "Total Revenue: Rs. " followed by a double value representing the total revenue rounded to one decimal place.

Refer to the sample output for the exact text and format.

Sample Test Case

Input: 1000.0
100

Output: Ticket Cost: Rs. 1000.0
Seat Availability: 100 seats
Total Revenue: Rs. 100000.0

Answer

```
import java.util.Scanner;  
import java.util.Scanner;  
import java.text.DecimalFormat;
```

```
class Airline
```

```
{
```

```
private double ticketCost;
```

```
public void setCost(double cost)
```

```
{
```

```
    this.ticketCost = cost;
```

```
}
```

```
public double getCost()
```

```
{
```

```
    return this.ticketCost;
```

```
}
```

```
}
```

```
class Indigo extends Airline
```

```
{
```

```
    private int seatAvailability;
```

```
    public void setSeatAvailability(int seatAvailability)
```

```
{
```

```
        this.seatAvailability = seatAvailability;
```

```
}
```

```
public int getSeatAvailability()
```

```
{
```

```
    return this.seatAvailability;
```

```
}
```

```
}
```

```
class Boeing747 extends Indigo
```

```
{
```

```
    public double calculateTotalRevenue()
```

```
{
```

```
        return getCost() * getSeatAvailability();
```

```
}
```

```
}
```

```
class Solution
```

```
{
```

```
    public static void main(String[] args)
```

```
{
```

```
        Scanner sc = new Scanner(System.in);
```

```
        DecimalFormat df = new DecimalFormat("0.0");
```

```

        if (!sc.hasNextDouble()) return;
        double cost = sc.nextDouble();

        if (!sc.hasNextInt()) return;
        int seatAvailability = sc.nextInt();

        Boeing747 boeing = new Boeing747();

        boeing.setCost(cost);
        boeing.setSeatAvailability(seatAvailability);

        double totalRevenue = boeing.calculateTotalRevenue();
        System.out.println("Ticket Cost: Rs. " + df.format(boeing.getCost()));
        System.out.println("Seat Availability: " + boeing.getSeatAvailability() + "
seats");
        System.out.println("Total Revenue: Rs. " + df.format(totalRevenue));

    }

}

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        Boeing747 plane = new Boeing747();

        double ticketCost = scanner.nextDouble();
        plane.setCost(ticketCost);
        int seatAvailability = scanner.nextInt();
        plane.setSeatAvailability(seatAvailability);

        System.out.printf("Ticket Cost: Rs. %.1f\n", plane.getCost());
        System.out.println("Seat Availability: " + plane.getSeatAvailability() + "
seats");
        System.out.printf("Total Revenue: Rs. %.1f\n",
plane.calculateTotalRevenue());
    }
}

```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

REC_2028_OOPS using Java_Week 7_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 15

Section 1 : MCQ

1. What happens when an implementing class does not override a default method from an interface?

Answer

The default method's implementation from the interface will be used.

Status : Correct

Marks : 1/1

2. Consider a class implementing an interface and extending a class, both having a method with the same name. Which method gets called?

Answer

The method from the superclass

Status : Correct

Marks : 1/1

3. What is the output of the following code?

```
interface X {  
    default void show() {  
        System.out.println("X's Default Method");  
    }  
}
```

```
interface Y {  
    default void show() {  
        System.out.println("Y's Default Method");  
    }  
}
```

```
class Z implements X, Y {  
    public void show() {  
        System.out.println("Z's Method");  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Z obj = new Z();  
        obj.show();  
    }  
}
```

Answer

Z's Method

Status : Correct

Marks : 1/1

4. What is the output of the following code?

```
interface MathOperations {  
    static int square(int x) {  
        return x * x;  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println(MathOperations.square(5));  
    }  
}
```

Answer

25

Status : Correct

Marks : 1/1

5. How do you call a static method from an interface MyInterface?

Answer

MyInterface.staticMethod();

Status : Correct

Marks : 1/1

6. What is the output of the following code?

```
interface A {  
    default void show() {  
        System.out.println("A's Default Method");  
    }  
}
```

```
interface B {  
    default void show() {  
        System.out.println("B's Default Method");  
    }  
}
```

```
class C implements A, B {  
    public void show() {  
        A.super.show();  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        C obj = new C();  
        obj.show();  
    }  
}
```

Answer

A's Default Method

Status : Correct

Marks : 1/1

7. What is the output of the following code?

```
interface A {  
    static void display() {  
        System.out.println("Static method in A");  
    }  
}
```

```
class B implements A {  
    static void display() {  
        System.out.println("Static method in B");  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        B.display();  
    }  
}
```

Answer

Static method in B

Status : Correct

Marks : 1/1

8. If a class implements two interfaces that have the same default

method, what must the class do?

Answer

The class must override the method to resolve ambiguity.

Status : Correct

Marks : 1/1

9. How can a class explicitly call a default method from an interface if there is a naming conflict?

Answer

Using InterfaceName.super.methodName();

Status : Correct

Marks : 1/1

10. What is the output of the following code?

```
interface A {  
    default void show() {  
        System.out.println("A's Default Method");  
    }  
}
```

```
class B {  
    public void show() {  
        System.out.println("B's Method");  
    }  
}
```

```
class C extends B implements A {  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        C obj = new C();  
        obj.show();  
    }  
}
```

Answer

B's Method

Status : Correct

Marks : 1/1

11. Can a Java interface contain both default and static methods?

Answer

Yes, an interface can have both default and static methods.

Status : Correct

Marks : 1/1

12. What is the primary purpose of static methods in Java interfaces?

Answer

They allow an interface to provide helper methods without requiring an implementing class.

Status : Correct

Marks : 1/1

13. Which of the following is the correct way to declare an interface in Java?

Answer

```
interface Vehicle { void start();}
```

Status : Correct

Marks : 1/1

14. Which of the following statements about Java interfaces is true?

Answer

A class can implement multiple interfaces.

Status : Correct

Marks : 1/1

15. Which of the following statements is true regarding default methods

in Java interfaces?

Answer

A default method can be overridden in a class implementing the interface.

Status : Correct

Marks : 1/1

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 7_Q1

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement:

Rajiv is analyzing the energy consumption in his household and wants to calculate the total cost based on the daily energy usage. He is given the rate per unit of electricity and the energy consumed for multiple days. To structure this calculation efficiently, he decides to use an interface-based approach.

Implement an interface CostCalculator with the necessary methods to retrieve energy details and compute the cost. The calculations should be handled in the EnergyConsumptionTracker class, while the EnergyConsumptionApp class should only handle input and output.

Formula

Energy Cost for one day = Energy Consumed per day * Rate Per Unit

Input Format

The first line of input consists of the rate per unit as an 'R' (a double value).

The second line of input consists of the number of days 'N' (an integer).

The third line of input consists of the daily energy consumption values for each day 'D' (double values), separated by space.

Output Format

The first line of the output prints: "Day-wise Energy Cost:"

The next N lines of the output print the day-wise energy costs(double type) and the total energy cost (double type) in Indian Rupees in the following format: "Day [day_number]: Rs. [energy_cost]"

The last line of the output prints: "Total Energy Cost: Rs. [total_cost]"

Note: energy_cost and total_cost are rounded off to two decimal points

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 0.01

3

10.0 20.0 30.0

Output: Day-wise Energy Cost:

Day 1: Rs. 0.10

Day 2: Rs. 0.20

Day 3: Rs. 0.30

Total Energy Cost: Rs. 0.60

Answer

```
import java.util.Scanner;
```

```
interface CostCalculator
```

{

void getEnergyDetails(Scanner scanner);
void calculateAndDisplayCost();

}

class EnergyConsumptionTracker implements CostCalculator

{

private double ratePerUnit;
private int numDays;
private double[] energyConsumptionArray;

public EnergyConsumptionTracker(double ratePerUnit, int numDays)

{

this.ratePerUnit = ratePerUnit;
this.numDays = numDays;
this.energyConsumptionArray = new double[numDays];

}

public void getEnergyDetails(Scanner scanner)

{

```
for (int i = 0; i < numDays; i++)
```

```
{
```

```
    energyConsumptionArray[i] = scanner.nextDouble();
```

```
}
```

```
}
```

```
public void calculateAndDisplayCost()
```

```
{
```

```
    double totalCost = 0;
```

```
    System.out.println("Day-wise Energy Cost:");
```

```
    for (int i = 0; i < numDays; i++)
```

```
    {
```

```
        double energyCost = energyConsumptionArray[i] * ratePerUnit;
```

```
        totalCost += energyCost;
```

```
        System.out.printf("Day %d: Rs. %.2f\n", i + 1, energyCost);
```

```
    }
```

```
    System.out.printf("Total Energy Cost: Rs. %.2f\n", totalCost);
```

```
}  
}  
  
class EnergyConsumptionApp {  
    public static void main(String[] args) {  
        Scanner scanner = new Scanner(System.in);  
  
        double ratePerUnit = scanner.nextDouble();  
        int numDays = scanner.nextInt();  
  
        CostCalculator tracker = new EnergyConsumptionTracker(ratePerUnit,  
numDays);  
  
        tracker.getEnergyDetails(scanner);  
        tracker.calculateAndDisplayCost();  
  
        scanner.close();  
    }  
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 7_Q2

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Jaheer is working on a health monitoring system to help individuals calculate their Body Mass Index (BMI). He has implemented a basic BMI calculator and an interface called HealthCalculator. It should have a method called calculateBMI.

You are tasked with creating a program that takes weight and height as input, calculates the BMI using the BMICalculator class, and displays the result. If the height or weight is less than or equal to zero, then return -1.

Formula: $BMI = \text{weight} / (\text{height} * \text{height})$

Input Format

The first line of input consists of a double value W, the person's weight in kilograms.

The second line consists of a double value H, the height of the person in meters.

Output Format

The output displays "BMI: " followed by a double value, representing the calculated BMI, rounded off to two decimal places.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 70.0

1.75

Output: BMI: 22.86

Answer

```
import java.util.Scanner;
```

```
import java.util.*;
```

```
interface HealthCalculator
```

```
{
```

```
    double calculateBMI(double weight, double height);
```

```
}
```

```
class BMICalculator implements HealthCalculator
```

```
{
```

```
public double calculateBMI(double weight, double height)
```

```
{
```

```
    if (weight <= 0 || height <= 0)
```

```
    {
```

```
        return -1;
```

```
    }
```

```
    return weight / (height * height);
```

```
}
```

```
}
```

```
class Main {
```

```
    public static void main(String[] args) {
```

```
        Scanner scanner = new Scanner(System.in);
```

```
        double weight = scanner.nextDouble();
```

```
        double height = scanner.nextDouble();
```

```
        BMICalculator bmiCalculator = new BMICalculator();
```

```
        double bmi = bmiCalculator.calculateBMI(weight, height);
```

```
        System.out.printf("BMI: %.2f\n", bmi);
```

```
        scanner.close();
```

```
    }
```

```
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 7_Q3

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

A financial analyst, Alex, needs a program to calculate simple interest for various financial transactions. He requires a straightforward tool that takes in the principal amount, interest rate, and time in years and computes the interest.

The formula to be used is: $\text{Interest} = \text{Principal} \times \text{Rate} \times \text{Time} / 100$

Implement this functionality using the InterestCalculator interface and the SimpleInterestCalculator class.

Input Format

The first line of input consists of the principal amount P as a double value.

The second line of input consists of the annual interest rate r as a double value.

The third line of input consists of the number of years t as a positive integer, which is an integer value.

Output Format

The output displays the calculated simple interest in the following format: "Simple Interest: [interest_value]", Here, [interest_value] should be replaced with the actual interest value calculated by the program.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 1000.00

5.00

2

Output: Simple Interest: 100.0

Answer

```
import java.util.Scanner;
```

```
interface InterestCalculator
```

```
{
```

```
    double simpleInterest(double principal, double rate, int time);
```

```
}
```

```
class SimpleInterestCalculator implements InterestCalculator
```

```
{
```

```
public double simpleInterest(double principal, double rate, int time)
{

    double interest = (principal * rate * time) / 100;
    return interest;
}

}

class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        double principal = scanner.nextDouble();

        double rate = scanner.nextDouble();

        int time = scanner.nextInt();

        InterestCalculator calculator = new SimpleInterestCalculator();

        double interest = calculator.simpleInterest(principal, rate, time);

        System.out.println("Simple Interest: " + interest);

    }
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 7_Q4

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Maria, a software developer, is working on an inventory management system project using Java that utilizes an inventory interface to manage a store's products.

The interface should define two methods: `addProduct`, which adds a product by accepting its name, price, and quantity, and `calculateTotalValue`, which computes the total value of all products in the inventory. Implement the interface in a class called `SimpleInventory`, which internally manages a list of `Product` objects.

Each `Product` object should encapsulate the product's name, price, and quantity and include a method to calculate its value as $\text{price} \times \text{quantity}$. The system should allow users to dynamically add products to the inventory and calculate the total value of all products stored.

Help Maria achieve the task.

Input Format

The first line of input consists of an integer to choose one of the following options:

- 1 - to add a product to the inventory.
- 2 - to calculate and view the total inventory value.
- 3 - to exit the program.

For Choice 1 (Add Product):

The next input line is the string representing the product name as a string (single or multi-word, without quotes).

The next line is a double value representing the price as a decimal value

The next line is an integer value representing the quantity as an integer

For Choices 2 and 3, no additional input is required

Output Format

The output displays the results of the commands as follows:

- For the addProduct command, the program should display "Product added to inventory."
- For choice 2, the program should display "Total inventory value [totalvalue].
"The total value should be displayed with one decimal place. If there is no product in the inventory, print the total as 0.0.
- For choice 3, the program should exit

If the choice is not 1, 2, or 3, then print "Invalid choice. Please select a valid option (1/2/3).".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 1

Laptop

800.0

3

2

5

3

Output: Product added to inventory.

Total inventory value: \$2400.0

Invalid choice. Please select a valid option (1/2/3).

Answer

```
import java.util.Scanner;
```

```
interface Inventory
```

```
{
```

```
    void addProduct(String name,double price,int quantity);
```

```
    double calculateTotalValue();
```

```
}
```

```
class SimpleInventory implements Inventory
```

```
{
```

```
    class Product
```

```
{
```

```
String name;  
double price;  
int quantity;
```

```
Product(String name,double price,int quantity)
```

```
{
```

```
    this.name=name;  
    this.price=price;  
    this.quantity=quantity;
```

```
}
```

```
    double getValue()
```

```
{
```

```
        return price*quantity;
```

```
}
```

```
}
```

```
private java.util.ArrayList<Product> products=new java.util.ArrayList<>();  
SimpleInventory()
```

```
{
```

```
}  
SimpleInventory(int size)
```

```
{
```

```
    products=new java.util.ArrayList<>(size);
```

```
}
```

```
    public void addProduct(String name,double price,int quantity)
```

```
{
```

```
        products.add(new Product(name,price,quantity));  
        System.out.println("Product added to inventory.");
```

```
}
```

```
    public double calculateTotalValue()
```

```
{
```

```
        double total=0.0;  
        for(Product p:products)
```



```
{
```

```
total+=p.getValue();
```

```
}
```

```
return total;
```

```
}
```

```
}
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        Scanner scanner = new Scanner(System.in);
```

```
        Inventory inventory = new SimpleInventory(10);
```

```
        while (true) {
```

```
            int choice = scanner.nextInt();
```

```
            if (choice == 1) {
```

```
                scanner.nextLine();
```

```
                String productName = scanner.nextLine();
```

```
                double price = scanner.nextDouble();
```

```
                int quantity = scanner.nextInt();
```

```
                inventory.addProduct(productName, price, quantity);
```

```
            } else if (choice == 2) {
```

```
                double totalValue = inventory.calculateTotalValue();
```

```
                System.out.println("Total inventory value: $" + totalValue);
```

```
            } else if (choice == 3) {
```

```
                break;
```

```
            } else {
```

```
                System.out.println("Invalid choice. Please select a valid option  
(1/2/3).");
```

```
            }
```

```
        }
```

```
        scanner.close();
```

```
    }
```

```
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 7_Q5

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Raj is curious about how old he is in the current year.

He has asked you to create a simple program that calculates a person's age based on their birth year. You decide to implement this functionality using the AgeCalculator interface and the HumanAgeCalculator class.

Note: The current year is 2024. Calculate the current age by using the formula: current year - birth year.

Input Format

The input consists of an integer representing the birth year.

Output Format

The output displays "You are X years old." where X is an integer representing the calculated age based on the entered birth year.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1934

Output: You are 90 years old.

Answer

```
import java.util.Scanner;
```

```
import java.util.*;
```

```
interface AgeCalculator
```

```
{
```

```
    int calculateAge(int birthYear);
```

```
}
```

```
class HumanAgeCalculator implements AgeCalculator
```

```
{
```

```
    public int calculateAge(int birthYear)
```

```
{
```

```
        int currentYear = 2024;
        return currentYear - birthYear;

    }

}

class AgeCalculatorApp {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        AgeCalculator ageCalculator = new HumanAgeCalculator();

        int birthYear = scanner.nextInt();
        int age = ageCalculator.calculateAge(birthYear);

        System.out.println("You are " + age + " years old.");
    }
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

REC_2028_OOPS using Java_Week 7_PAH

Attempt : 1
Total Mark : 40
Marks Obtained : 0

Section 1 : Coding

1. Problem Statement

Sophia is developing a matrix analysis tool for a data analytics company. The tool needs to analyze square matrices and extract insights from the matrix diagonals.

To organize the code properly, Sophia creates an interface named Matrix that declares a method for finding the smallest and largest elements along the principal and secondary diagonals of the matrix.

Sophia then creates a class named MatrixAnalyzer that implements the Matrix interface. This class provides the logic to process a given square matrix and print:

The smallest and largest elements in the principal diagonal (from top-left to bottom-right). The smallest and largest elements in the secondary

diagonal (from top-right to bottom-left).

Your task is to implement the Matrix interface and the MatrixAnalyzer class. The main driver program (in the class Main) will read the input matrix, create an instance of MatrixAnalyzer, and invoke its method to display the results.

Input Format

The first line contains an integer n, representing the size of the square matrix.

The next n lines each contain n integers separated by spaces, representing the elements of the matrix.

Output Format

The output prints the four lines:

"Smallest Element - 1: <smallest element in the principal diagonal>" (integer)

"Largest Element - 1: <largest element in the principal diagonal>" (integer)

"Smallest Element - 2: <smallest element in the secondary diagonal>" (integer)

"Largest Element - 2: <largest element in the secondary diagonal>" (integer)

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5

7 8 9 0 1

2 3 4 5 6

5 4 2 0 8

23 5 6 8 9

12 5 6 7 32

Output: Smallest Element - 1: 2

Largest Element - 1: 32

Smallest Element - 2: 1

Largest Element - 2: 12

Answer

-

Status : Skipped

Marks : 0/10

2. Problem Statement

Develop a program for managing employee information that caters to both full-time and part-time employees. The program should be capable of computing the salary for each category of employee and presenting their particulars. To achieve this, create two classes, FullTimeEmployee and PartTimeEmployee, that adhere to the Employee interface.

The program is expected to accept input data, including the name and monthly salary for full-time employees, as well as the name, hourly rate, and hours worked for part-time employees. Subsequently, it should calculate and exhibit the employee details and their respective salaries.

For Full-Time employees, the annual salary should be calculated as 12 times the monthly salary.

For Part-Time employees, the salary calculation should be based on the formula: hourly rate * hours worked.

Input Format

The first line of input should be a string representing the name of a full-time employee.

The second line of input should be an integer representing the monthly salary of the full-time employee.

The third line of input should be a string representing the name of a part-time employee.

The fourth line of input should be an integer representing the hourly rate of the part-time employee.

The fifth line of input should be an integer representing the number of hours worked by the part-time employee.

Output Format

The output displays the following details:

Full-Time Employee Details:

Name: [Full-Time Employee Name] (string)

Monthly Salary: \$[Monthly Salary] (integer)

Annual Salary: \$[12 times Monthly Salary] (integer)

Part-Time Employee Details:

Name: [Part-Time Employee Name] (string)

Hourly Rate: \$[Hourly Rate] (integer)

Hours Worked: [Hours Worked] hours (integer)

Monthly Salary: \$[Calculated Monthly Salary] (integer)

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: John Smith

15000

Mary Johnson

100

100

Output: Full-Time Employee Details:

Name: John Smith

Monthly Salary: \$15000

Annual Salary: \$180000

Part-Time Employee Details:

Name: Mary Johnson

Hourly Rate: \$100

Hours Worked: 100 hours

Monthly Salary: \$10000

Answer

-

Status : -

Marks : 0/10

3. Problem Statement

Oviya is fascinated by automorphic numbers and wants to create a program to determine whether a given number is an automorphic number or not.

An automorphic number is a number whose square ends with the same digits as the number itself. For example, $25 = (25)^2 = 625$

Oviya has defined two interfaces: NumberInput for taking user input and AutomorphicChecker for checking if a given number is automorphic. The class AutomorphicNumber implements both interfaces.

Help her complete the task.

Input Format

The input consists of a single integer n.

Output Format

If the input number is an automorphic number, print "n is an automorphic number". Otherwise, print "n is not an automorphic number".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 25

Output: 25 is an automorphic number

Answer

-

Status : -

Marks : 0/10

4. Problem Statement:

Alice has been tasked with implementing a simple calculator interface and a corresponding class for performing basic addition and subtraction operations. The task is to create an interface called Calculator with two methods: add and subtract. The add method should take two numbers as input and return their sum, while the subtract method should take two numbers as input and return their difference.

Implement a class called SimpleCalculator that implements the Calculator interface. This class should provide the functionality for adding and subtracting numbers. Write a code that satisfies the above requirements.

Input Format

The first line of input consists of a single integer, representing the choice

If the choice is 1 or 2, the next two lines consist of 2 double values, representing the numbers to do addition or subtraction.

Output Format

The output prints a float-value with one decimal value representing the sum of two number or difference of two number.

Refer to the sample output for format specification.

Sample Test Case

Input: 1

5.5

3.5

Output: Result: 9.0

Answer

-

Status : -

Marks : 0/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

REC_2028_OOPS using Java_Week 7_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 0

Section 1 : Coding

1. Problem Statement

Maria, an online store owner, is looking to implement a pricing system that calculates the final price of products after applying discounts. She needs a program that takes the original price of a product and the discount percentage as input and computes the final discounted price. The discount is applied as a percentage of the original price. Maria wants to ensure that the final price is formatted to display exactly two decimal places.

Implement this functionality using the PriceCalculator interface and the DiscountCalculator class.

Input Format

The first line of input consists of the original price (a double value).

The second line of input consists of a discount percentage (a double value).

Output Format

The output displays the final price after the discount, adhering to the following format: "Final Price after discount: \$[final_price]".

Here, [final_price] should be replaced with the calculated final price, formatted as a currency value with two decimal places.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 100.0
10.0

Output: Final Price after discount: \$90.00

Answer

-

Status : Skipped

Marks : 0/10

2. Problem Statement

John is developing a car loan calculator and has structured his program using two interfaces, Principal and InterestRate, defining methods for principal and interest rate retrieval.

The Loan class implements these interfaces, taking principal and annual interest rates as parameters. User input is solicited for these values, and the program ensures their validity before performing calculations. If input values are invalid (less than or equal to zero), an error message is displayed.

Note: Total interest = principal * interest rate * years

Input Format

The first line of input consists of a double value P, representing the principal.

The second line consists of a double value R, representing the annual interest rate.

The third line consists of an integer value N, representing the loan duration in years.

Output Format

If the input values are valid, print "Total interest paid: Rs. " followed by a double value, representing the total interest paid, rounded off to two decimal places.

If the input values are invalid (negative or zero values for principal, annual interest rate, or loan duration), print "Invalid input values!".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 20000.00
0.05
5

Output: Total interest paid: Rs.5000.00

Answer

-

Status : -

Marks : 0/10

3. Problem Statement:

Rathish is planning a road trip and needs a program to convert speeds between miles per hour (MPH) and kilometers per hour (KPH).

Create an interface, SpeedConverter, with a method convertSpeed(double mph). Implement the interface with MPHtoKPHConverter class, allowing Rathish to input MPH and receive the converted speed in KPH, rounded to two decimal points.

Formula: Speed in KPH = 1.60934 * Speed in MPH.

Input Format

The input consists of a single double-point number representing the speed in miles per hour (MPH).

Output Format

The output displays the converted speed (double-point number) in kilometers per hour (KPH) rounded off to two decimal points in the following format:

"Speed in KPH: <<converted speed>>".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 1.0

Output: Speed in KPH: 1.61

Answer

-

Status : -

Marks : 0/10

4. Problem Statement

Alex and Bob are designing a control system for household appliances, and one of the appliances is a washing machine. You want to create a program to help them that models the washing machine as a motor and calculates its electricity consumption based on its capacity.

Define an interface named Motor with the following methods:

void run() double consume(double capacity)

Create a class called WashingMachine that implements the Motor interface.

In the WashingMachine class:

Implement the run() method to print "Washing machine is running." Implement a consume() method to print "Washing machine is consuming electricity." Implement the consume(double capacity) method to calculate the electricity consumption (in kWh) of the washing machine based on its capacity. The formula for electricity consumption is (capacity * 0.05).

Input Format

The input consists of a double value representing the capacity of the washing machine in kW.

Output Format

The first line of output prints "Washing machine is running."

The second line prints "Washing machine is consuming electricity."

The third line prints "Electricity consumption: X kWh" where X is a double value, rounded off to two decimal places, representing the electricity consumption.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 2.5

Output: Washing machine is running.

Washing machine is consuming electricity.

Electricity consumption: 0.13 kWh

Answer

-

Status : -

Marks : 0/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

REC_2028_OOPS using Java_Week 8_MCQ

Attempt : 2
Total Mark : 15
Marks Obtained : 15

Section 1 : MCQ

1. What is the purpose of a custom exception in Java?

Answer

To create user-defined exceptions for specific scenarios

Status : Correct

Marks : 1/1

2. What will be the output for the following code?

```
class InvalidUsernameException extends Exception {  
    public InvalidUsernameException(String message) {  
        super(message);  
    }  
}
```

```

class Test {
    public static void main(String[] args) {
        try {
            String username = "abc";
            if (username.length() < 5) {
                throw new InvalidUsernameException("Username must be at
least 5 characters long");
            }
        } catch (InvalidUsernameException e) {
            System.out.println(e.getMessage());
        }
    }
}

```

Answer

Username must be at least 5 characters long

Status : Correct

Marks : 1/1

3. What will be the output for the following code?

```
import java.io.*;
```

```

class OutOfStockException extends Exception {
    public OutOfStockException(String message) {
        super(message);
    }
}

```

```

class Test {
    public static void main(String[] args) {
        try {
            int stock = 0;
            if (stock == 0) {
                throw new OutOfStockException("Item is out of stock");
            }
        } catch (OutOfStockException e) {
            System.out.println(e.getMessage());
        }
    }
}

```

Answer

Item is out of stock

Status : Correct

Marks : 1/1

4. What will be the output for the following code?

```
import java.io.*;
```

```
class TemperatureTooHighException extends Exception {  
    public TemperatureTooHighException(String message) {  
        super(message);  
    }  
}
```

```
class Test {  
    public static void main(String[] args) {  
        try {  
            int temperature = 110;  
            if (temperature > 100) {  
                throw new TemperatureTooHighException("Temperature too  
high");  
            }  
        } catch (TemperatureTooHighException e) {  
            System.out.println(e.getMessage());  
        }  
    }  
}
```

Answer

Temperature too high

Status : Correct

Marks : 1/1

5. What will be the output of the following code?

```
class MyException extends Exception {  
    public MyException() {  
        super("Default Exception Message");  
    }  
}  
  
class Test {  
    public static void main(String[] args) {  
        try {  
            throw new MyException();  
        } catch (MyException e) {  
            System.out.println(e.getMessage());  
        }  
    }  
}
```

Answer

Default Exception Message

Status : Correct

Marks : 1/1

6. Which of the following is true about custom exceptions?

Answer

Custom exceptions must extend either Exception or RuntimeException

Status : Correct

Marks : 1/1

7. What will be the output for the following code?

```
import java.io.*;  
  
class UnderageException extends Exception {  
    public UnderageException(String message) {  
        super(message);  
    }  
}
```

```
class Test {  
    public static void main(String[] args) {  
        try {  
            int age = 17;  
            if (age < 18) {  
                throw new UnderageException("Underage, cannot proceed");  
            }  
        } catch (UnderageException e) {  
            System.out.println(e.getMessage());  
        }  
    }  
}
```

Answer

Underage, cannot proceed

Status : Correct

Marks : 1/1

8. Which keyword is used to explicitly throw a custom exception?

Answer

throw

Status : Correct

Marks : 1/1

9. What will be the output for the following code?

```
import java.io.*;
```

```
class NegativeAgeException extends Exception {  
    public NegativeAgeException(String message) {  
        super(message);  
    }  
}
```

```
class Test {  
    public static void main(String[] args) {  
        try {
```

```
int age = -5;
if (age < 0) {
    throw new NegativeAgeException("Age cannot be negative");
}
} catch (NegativeAgeException e) {
    System.out.println(e.getMessage());
}
}
```

Answer

Age cannot be negative

Status : Correct

Marks : 1/1

10. What will happen if a checked custom exception is thrown inside a method without being caught or declared?

Answer

Compilation Error

Status : Correct

Marks : 1/1

11. How do you create an unchecked custom exception?

Answer

By extending RuntimeException

Status : Correct

Marks : 1/1

12. What will be the output for the following code?

```
class InvalidVotingAgeException extends Exception {
    public InvalidVotingAgeException(String message) {
        super(message);
    }
}
```

```

class Test {
    public static void main(String[] args) {
        try {
            int age = 15;
            if (age < 18) {
                throw new InvalidVotingAgeException("You are not eligible to
vote");
            }
            System.out.println("Eligible to vote");
        } catch (InvalidVotingAgeException e) {
            System.out.println(e.getMessage());
        }
    }
}

```

Answer

You are not eligible to vote

Status : Correct

Marks : 1/1

13. What will be the output for the following code?

```

class NegativeBalanceException extends Exception {
    public NegativeBalanceException(String message) {
        super(message);
    }
}

```

```

class Test {
    public static void main(String[] args) {
        try {
            double balance = -500;
            if (balance < 0) {
                throw new NegativeBalanceException("Balance cannot be
negative");
            }
        } catch (NegativeBalanceException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}

```


Answer

Error: Balance cannot be negative

Status : Correct

Marks : 1/1

14. what is the output of the following code?

```
class MyException extends Exception {  
    public MyException(String message) {  
        super(message);  
    }  
}  
  
class Test {  
    static void check() throws MyException {  
        throw new MyException("Custom Exception Occurred");  
    }  
  
    public static void main(String[] args) {  
        try {  
            check();  
        } catch (Exception e) {  
            System.out.println(e.getMessage());  
        }  
    }  
}
```

Answer

Custom Exception Occurred

Status : Correct

Marks : 1/1

15. what is the output of the following code?

```
class MyException extends Exception {  
    public MyException(String message) {
```

```
        super(message);
    }
}

class Test {
    public static void main(String[] args) {
        try {
            throw new MyException("Error occurred");
        } catch (MyException e) {
            System.out.println(e);
        }
    }
}
```

Answer

MyException: Error occurred

Status : Correct

Marks : 1/1

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 8_Q1

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Write a program to validate the email address and display suitable exceptions if there is any mistake.

Create 3 custom exception classes as below

DotException AtTheRateException DomainException

A typical email address should have a "." character, and a "@" character, and also the domain name should be valid. Valid domain names for practice be 'in', 'com', 'net', or 'biz'.

Display Invalid Dot usage, Invalid @ usage, or Invalid Domain message based on email id.

Get the email address from the user, validate the email by checking the

above-mentioned criteria, and print the validity status of the input email address.

Input Format

The first line of input contains the email to be validated.

Output Format

The output prints a Valid email address or an Invalid email address along with the suitable exception

If email ends with . or contains not exactly one . after @, it throws:

DotException: Invalid Dot usage

Invalid email address

If @ appears not exactly once, it throws:

AtTheRateException: Invalid @ usage

Invalid email address

If the part after the last dot is not among accepted domains:

DomainException: Invalid Domain

Invalid email address

If all conditions satisfied then print:

Valid email address

Refer to the sample input and output for format specifications.

Sample Test Case

Input: sample@gmail.com

Output: Valid email address

Answer

```
import java.util.Scanner;
```

```
class DomainException extends Exception
```

```
{
```

```
    String expDescription;
```

```
    DomainException(String expDescription)
```

```
{
```

```
        super(expDescription);
```

```
}
```

```
}
```

```
class DotException extends Exception
```

```
{
```

```
String expDescription;  
DotException(String expDescription)
```

```
{
```

```
    super(expDescription);
```

```
}
```

```
}
```

```
class AtTheRateException extends Exception
```

```
{
```

```
String expDescription;  
AtTheRateException(String expDescription)
```

```
{
```

```
    super(expDescription);
```

```
}
```

```
}
```

```
class EmailValidationMain
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        Scanner myObj = new Scanner(System.in);
```

```
        String email = myObj.next();
```

```
        boolean checkEndDot = false;
```

```
        checkEndDot = email.endsWith(".");
```

```
        int indexOfAt = email.indexOf('@');
```

```
        int lastIndexOfAt = email.lastIndexOf('.');
```

```
        int countOfAt = 0;
```

```
        for (int i = 0; i < email.length(); i++)
```

```
        {
```

```
            if(email.charAt(i)=='@')
```

```
                countOfAt++;
```

```
        }
```

```
        String buffering = email.substring(email.indexOf('@')+1, email.length());
```

```
        int len = buffering.length();
```

```
        int countOfDotAfterAt = 0;
```

```
        for (int i=0; i < len; i++)
```

```
        {
```

```
        if(buffering.charAt(i)=='.')
            countOfDotAfterAt++;
    }
    String userName = email.substring(0, email.indexOf('@'));
    String domainName = email.substring(email.indexOf('.')+1, email.length());
    int domainCheck=0;
    if((domainName.equals("in")) || (domainName.equals("com")) ||
(domainName.equals("net")) || (domainName.equals("biz")))
        domainCheck=1;
```

```
    try
    {

        if((checkEndDot) || (countOfDotAfterAt!=1))
        {
```

```
            throw new DotException("Invalid Dot usage");
        }
```

```
        if(countOfAt!=1)
        {
```

```
            throw new AtTheRateException("Invalid @ usage");
```



```
}
```

```
    if(domainCheck!=1)
```

```
{
```

```
        throw new DomainException("Invalid Domain");
```

```
}
```

```
    }catch(DotException e)
```

```
{
```

```
        System.out.println(e);
```

```
    }catch(AtTheRateException e)
```

```
{
```

```
        System.out.println(e);
```

```
    }catch(DomainException e)
```

```
{
```

```
        System.out.println(e);
    }

    if ((countOfAt==1) && (userName.endsWith(".")==false) &&
        (domainCheck==1) && (countOfDotAfterAt ==1) &&((indexOfAt+3) <=
        (lastIndexOfAt) && !checkEndDot))
    {

        System.out.println("Valid email address");

    }

    else

    {

        System.out.println("Invalid email address");

    }

    myObj.close();

}

}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN

Email: 240701805@rajalakshmi.edu.in

Roll no: 240701805

Phone: 6379580366

Branch: REC

Department: CSE - Section 6

Batch: 2028

Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 8_Q2

Attempt : 1

Total Mark : 10

Marks Obtained : 0

Section 1 : Coding

1. Problem Statement

Elsa, a busy professional, is using a scheduling application to plan her meetings efficiently. The application requires users to input meeting durations in minutes, ensuring that the duration is a positive integer and does not exceed 240 minutes (4 hours). Elsa needs a program to assist her in scheduling meetings securely with proper exception handling.

Create a Java class named ElsaMeetingScheduler. Implement a custom exception: InvalidDurationException for invalid meeting duration entries. Implement the main method to interactively take user input for a meeting duration. Implement the validateMeetingDuration method to validate the meeting duration based on the specified rules and throw a custom exception if the validation fails. Print appropriate success or error messages based on the meeting duration.

Implement a custom exception, `InvalidDurationException`, to handle cases where the entered meeting duration does not meet the specified criteria.

Input Format

The input consists of an integer value 'n', representing the meeting duration.

Output Format

The output is displayed in the following format:

If the entered meeting duration meets the specified criteria, the program outputs

"Meeting scheduled successfully!"

If the entered meeting duration is invalid, the program outputs an error message indicating the issue.

"Error: Invalid meeting duration. Please enter a positive integer not exceeding 240 minutes (4 hours)."

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 120

Output: Meeting scheduled successfully!

Answer

-

Status : Skipped

Marks : 0/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 8_Q3

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

In a user registration system, there is a requirement to implement a username validation module. Users attempting to register must adhere to specific criteria for their usernames to be considered valid.

Your task is to develop a program that takes user input for a desired username and validates it according to the following rules:

The username must not contain any spaces. The username must be at least 5 characters long.

Implement a custom exception, `InvalidUsernameException`, to handle cases where the entered username does not meet the specified criteria.

Input Format

The input consists of a string S, representing the desired username.

Output Format

If the username is valid, print "Username is valid: [S]".

If the username is invalid:

1. If the username is short, print "Invalid Username: Username must be at least 5 characters long"
2. If the username contains spaces, print "Invalid Username: Username cannot contain spaces"

Refer to the sample output for formatting specifications.

Sample Test Case

Input: John

Output: Invalid Username: Username must be at least 5 characters long

Answer

```
import java.util.Scanner;  
class InvalidUsernameException extends Exception
```

```
{
```

```
    public InvalidUsernameException(String message)
```

```
{
```

```
    super(message);
```

```
}  
}
```

```
class UsernameValidator
```

```
{
```

```
    public static void validateUsername(String username) throws  
        InvalidUsernameException
```

```
{
```

```
    if (username.contains(" "))
```

```
{
```

```
        throw new InvalidUsernameException("Username cannot contain  
        spaces");
```

```
}
```

```
    if (username.length() < 5)
```

```
{
```

```
        throw new InvalidUsernameException("Username must be at least 5  
        characters long");
```

```
}
```

```
}
```

```
}
```

```
public class Main
```

```
{
```

```
    public static void main(String[] args)
```

```
{
```

```
    Scanner scanner = new Scanner(System.in);
```

```
    String username = scanner.nextLine();
```

```
    try
```

```
{
```

```
        UsernameValidator.validateUsername(username); // Method call from  
        UsernameValidator
```

```
        System.out.println("Username is valid: " + username);
```

```
    } catch (InvalidUsernameException e)
```

```
{
```



```
System.out.println("Invalid Username: " + e.getMessage());
```

```
}
```

```
}
```

```
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 8_Q4

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

A local municipality is implementing an online voting system for a community event and wants to ensure that only eligible voters (those aged 18 or older) can participate.

Your task is to develop a program that validates the age of individuals attempting to vote online. If the user's age is below 18, the program should throw a custom exception, `InvalidAgeException`, preventing them from casting their vote. If the input is invalid, catch the appropriate `InputMismatchException` and print the in-built exception message.

Input Format

The input consists of an integer representing the age.

Output Format

If the age is 18 or older, print "Eligible to vote"

If the age is below 18, print "Exception occurred: InvalidAgeException: Age is not valid to vote"

If there is any other type of exception, print "An error occurred: " followed by the in-built exception message.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 20

Output: Eligible to vote

Answer

```
import java.util.InputMismatchException;  
import java.util.Scanner;
```

```
class InvalidAgeException extends Exception
```

```
{
```

```
    public InvalidAgeException(String message)
```

```
{
```

```
    super(message);
```

```
}
```

```
}
```

```
public class Main
```

```
{
```

```
    public static void validateAge(int age) throws InvalidAgeException
```

```
    {
```

```
        if (age < 18)
```

```
        {
```

```
            throw new InvalidAgeException("Age is not valid to vote");
```

```
        }
```

```
    }
```

```
    public static void main(String[] args)
```

```
    {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        try
```

```
{  
  
    int age = sc.nextInt();  
    validateAge(age);  
    System.out.println("Eligible to vote");  
  
}
```

```
    catch (InvalidAgeException e)
```

```
{
```

```
    System.out.println("Exception occurred: InvalidAgeException: " +  
e.getMessage());
```

```
}
```

```
    catch (InputMismatchException e)
```

```
{
```

```
    System.out.println("An error occurred: " + e);
```

```
}
```

```
    catch (Exception e)
```

```
{
```

```
        System.out.println("An error occurred: " + e.getMessage());
    }
    finally
    {
```

```
        sc.close();
```

```
    }
}
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 8_Q5

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

In a file management system, users are required to provide a valid file name when creating new files. The system enforces specific rules for file names to maintain consistency and avoid potential issues. Your task is to implement a Java program named FileNameValidator that takes user input for a file name and validates it according to the specified rules.

Rules for Valid File Name:

The file name must consist of alphanumeric characters (letters and digits) only. The file name must have a minimum length of 3 characters.

Implement a custom exception, FileNameValidator, to handle cases where the entered filename does not meet the specified criteria.

Input Format

The input consists of a string S, representing the desired filename.

Output Format

The output is displayed in the following format:

If the entered file name meets the specified criteria, the program outputs

"Valid file name"

If the entered file name does not meet the criteria and triggers the `InvalidFileNameException`, the program outputs

"Error: Invalid file name. It must be alphanumeric and have a minimum length of 3 characters."

Refer to the sample output for formatting specifications.

Sample Test Case

Input: myfile123

Output: Valid file name

Answer

```
import java.util.Scanner;
```

```
class InvalidFileNameException extends Exception
```

```
{
```

```
    public InvalidFileNameException(String message)
```

```
{
```



```
super(message);
```

```
}
```

```
}
```

```
class FileNameValidator
```

```
{
```

```
    public static void validateFileName(String fileName) throws  
        InvalidFileNameException
```

```
{
```

```
    if (fileName.length() < 3 || !fileName.matches("[A-Za-z0-9]+"))
```

```
{
```

```
        throw new InvalidFileNameException(  
            "Error: Invalid file name. It must be alphanumeric and have a minimum  
length of 3 characters."  
        );
```

```
}
```

```
}
```

```
public static void main(String[] args)
```

```
{
```

```
    Scanner sc = new Scanner(System.in);  
    String fileName = sc.nextLine();  
    sc.close();
```

```
    try
```

```
    {
```

```
        validateFileName(fileName);  
        System.out.println("Valid file name");
```

```
    } catch (InvalidFileNameException e)
```

```
    {
```

```
        System.out.println(e.getMessage());
```

```
    }
```

```
}
```

```
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

REC_2028_OOPS using Java_Week 8_PAH

Attempt : 1
Total Mark : 40
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Enigma is developing a simple web application that takes a user-input URL, validates it, and throws a custom exception `InvalidURLException` if the URL does not start with "http://" or "https://".

The main method prompts the user for input, validates the URL, and prints whether it is valid or not.

Input Format

The input consists of a string, representing the URL entered by the user.

Output Format

The output displays one of the following results:

If the entered URL is valid according to the specified format, the program prints:

"[URL] is a valid URL"

If the entered URL is not valid according to the specified format, the program prints:

"Invalid URL format: [URL]"

Refer to the sample output for formatting specifications.

Sample Test Case

Input: http://www.example.com

Output: http://www.example.com is a valid URL

Answer

```
import java.util.Scanner;  
class WebApplication
```

```
{
```

```
    public static void main(String[] args)
```

```
{
```

```
    String userInputURL = getUserInputURL();  
    try
```

```
validateURL(userInputURL);  
System.out.println(userInputURL+" is a valid URL");
```

```
} catch (InvalidURLException e)
```

```
{
```

```
System.out.println(e.getMessage()+userInputURL);
```

```
}
```

```
}
```

```
private static String getUserInputURL()
```

```
{
```

```
Scanner scanner = new Scanner(System.in);  
return scanner.nextLine();
```

```
}
```

```
private static void validateURL(String url) throws InvalidURLException
```

```
{
```

```
        if (!(url.startsWith("http://") || url.startsWith("https://")))
        {

            throw new InvalidURLExceptionFormatException("Invalid URL format: ");

        }
    }
}
class InvalidURLExceptionFormatException extends Exception
{
```

```
    public InvalidURLExceptionFormatException(String message)
    {

        super(message);

    }
}
```

Status : Correct

Marks : 10/10

2. Problem Statement

You are tasked to create a program that defines a custom exception `GradeException`. The program should include a `Student` class with fields for the student's name, age, and grade. Implement a method in the `Student` class that checks the grade, and if the grade is below 40, it should throw a `GradeException`. Otherwise, it should display the student's details.

Input Format

The input consists of three parameters in separate lines:

1. A string representing the student's name.
2. An integer representing the student's age.
3. An integer representing the student's grade.

Output Format

The output will display the student's details if the grade is valid.

If the grade is below 40, the program will display an error message "Grade is below 40".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: Alice

20

85

Output: Name: Alice

Age: 20

Grade: 85

Answer

-

Status : -

Marks : 0/10

3. Problem Statement

An HR software system is being developed to process employee payrolls. During payroll processing, the system must ensure that no employee has a negative salary and that no employee's salary exceeds 2,00,000. If either condition occurs, the system should throw a custom exception.

Create a custom exception `InvalidSalaryException` and a class `Employee` that processes salary according to the following rules:

If $\text{salary} < 0$, throw `InvalidSalaryException` with the message: "Salary cannot be negative". If $\text{salary} > 200000$, throw `InvalidSalaryException` with the message: "Salary exceeds threshold limit". Otherwise, display: "Salary processed successfully for <empName>: <salary>".

The payroll processing should always display: "Payroll process completed" at the end, regardless of whether an exception occurs.

Input Format

The first line of input contains an integer representing the employee ID.

The second line contains a string representing the employee's name.

The third line contains a floating-point number representing the salary of the employee.

Output Format

If the salary is valid: "Salary processed successfully for <empName>: <salary>"

"Payroll process completed"

If the salary is invalid: "<Exception Message>"

"Payroll process completed"

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 101

Rahul
150000.0

Output: Salary processed successfully for Rahul: 150000.0
Payroll process completed

Answer

-

Status : -

Marks : 0/10

4. Problem Statement

Daniel is developing a program to verify the age of users. He wants to ensure that the entered age is within a valid range. Write a program to help Daniel implement this age-checking feature using custom exceptions.

Daniel needs a program that takes an integer input representing a person's age. If the age is between 0 and 150 (inclusive), the program should print "Age is valid!". If the age is less than 0 or greater than 150, the program should throw a custom exception (InvalidAgeException) with the message "Invalid age. Please enter an age between 0 and 150."

Implement a custom exception, InvalidAgeException, to handle cases where the entered age does not meet the specified criteria.

Input Format

The input consists of an integer value 'n', representing the age.

Output Format

The output is displayed in the following format:

If the age is valid (between 0 and 150, inclusive), print

"Age is valid!".

If the age is invalid, print

"Error: Invalid age. Please enter an age between 0 and 150."

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 45

Output: Age is valid!

Answer

-

Status : -

Marks : 0/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

REC_2028_OOPS using Java_Week 8_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Faustus is managing his bank account and wants to create a program to update his account balance based on certain conditions. However, he needs to handle specific scenarios related to invalid inputs and insufficient balances. Faustus wants to update his account balance. He inputs the current balance and the amount to be updated.

The initial account balance should be positive. If Faustus enters a negative initial balance, the program should throw an `InvalidAmountException` with the message "Invalid amount. Please enter a positive initial balance." If the amount to be updated is negative, the program should check if the subtraction results in a negative balance. If so, it should throw an `InsufficientBalanceException` with the message "Insufficient balance." If the amount to be updated is positive, it should be added to the current balance, and the new balance should be printed.

Implement a custom exception, InvalidAmountException, and InsufficientBalanceException, to manage his bank account.

Input Format

The first line of input consists of a double value 'd', representing the initial account balance.

The second line of input consists of a double value 'd1', representing the amount to be updated.

Output Format

The output is displayed in the following format:

If the validation passes, print

"Account balance updated successfully! New balance: {new_balance}"

where {new_balance} is the updated account balance.

If the initial bank amount is negative it displays

"Error: Invalid amount. Please enter a positive initial balance."

If the updated amount exceeds the initial account balance in withdrawal it displays

"Error: Insufficient balance."

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1000
500

Output: Account balance updated successfully! New balance: 1500.0

Answer

```
import java.util.Scanner;
```

```
class InvalidAmountException extends Exception
```

```
{
```

```
    public InvalidAmountException(String message)
```

```
    {
```

```
        super(message);
```

```
    }
```

```
}
```

```
class InsufficientBalanceException extends Exception
```

```
{
```

```
    public InsufficientBalanceException(String message)
```

```
    {
```

```
        super(message);
```

```
    }
```

```
}
```

```
class Test
```

```
{
```

```
    public static double updateBalance(double currentBalance, double  
    updateAmount)
```

throws InvalidAmountException, InsufficientBalanceException

{

if (currentBalance < 0)

{

throw new InvalidAmountException("Invalid amount. Please enter a positive initial balance.");

}

double newBalance;

if (updateAmount < 0)

{

newBalance = currentBalance + updateAmount;

if (newBalance < 0)

{

throw new InsufficientBalanceException("Insufficient balance.");

}

} else

{

newBalance = currentBalance + updateAmount;

```
}
```

```
    return newBalance;
```

```
}
```

```
public static void main(String[] args)
```

```
{
```

```
    Scanner scanner = new Scanner(System.in);
```

```
    double initialBalance = scanner.nextDouble();
```

```
    double updateAmount = scanner.nextDouble();
```

```
    scanner.close();
```

```
    try
```

```
{
```

```
        double finalBalance = updateBalance(initialBalance, updateAmount);
```

```
        System.out.printf("Account balance updated successfully! New balance:  
%.1f\n", finalBalance);
```

```
    } catch (InvalidAmountException e)
```

```
{
```

```
        System.out.println("Error: " + e.getMessage());
```

```
    } catch (InsufficientBalanceException e)
```

```
{  
    System.out.println("Error: " + e.getMessage());  
  
}  
  
}  
  
}
```

Status : Correct

Marks : 10/10

2. Problem Statement

In an online shopping cart system, users can apply coupon codes during checkout to avail of discounts. However, to ensure the validity and security of coupon codes, the system enforces specific rules for their format. Your task is to implement a Java program named `CouponCodeValidator` that takes user input for a coupon code and validates it according to the specified rules.

Rules for Valid Coupon Code:

The coupon code must consist of exactly 10 characters. The coupon code must contain at least one alphabet (uppercase or lowercase) and at least one digit (0-9). Special characters are not allowed in the coupon code.

Implement a custom exception, `InvalidCouponException`, to handle cases where the entered coupon code does not meet the specified criteria.

Input Format

The input consists of a string `s`, representing the coupon code.

Output Format

The output is displayed in the following format:

If the entered coupon code meets the specified criteria, the program outputs

"Coupon code applied successfully!"

If the entered coupon code has less than or more than 10 characters it outputs

"Error: Invalid coupon code length. It must be exactly 10 characters."

If the entered coupon code contains only numeric or only alphabets it outputs

"Error: Invalid coupon code format. It must contain at least one alphabet and one digit."

If the entered coupon code contains special characters it outputs

"Error: Coupon code should not contain special characters."

Refer to the sample output for formatting specifications.

Sample Test Case

Input: ABCD123456

Output: Coupon code applied successfully!

Answer

-

Status : -

Marks : 0/10

3. Problem Statement

Hemanth is designing a banking system for XYZ Bank. The system should allow customers to perform deposit, withdrawal, and balance inquiry operations. Implement exception handling for scenarios involving invalid transaction amounts or insufficient funds.

Create two custom exception classes, InvalidAmountException and InsufficientFundsException, both extending the Exception class. Throw an

InvalidAmountException with a message if the deposit amount is less than or equal to zero. Throw an InsufficientFundsException if the withdrawal amount is greater than the available balance. Deduct the withdrawal amount from the balance if the withdrawal is successful.

Assist Hemanth in designing the program.

Input Format

The first line of input consists of a double value B, representing the initial balance.

The second line consists of a double value D, representing the deposit amount.

The third line consists of a double value W, representing the withdrawal amount.

Output Format

If the withdrawal is successful, print the amount withdrawn and the current balance, rounded off to one decimal place.

If an InvalidAmountException occurs, print "Error: [D] is not valid".

If an InsufficientFundsException occurs, print "Error: Insufficient funds".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1050.1

270.2

150.3

Output: Amount Withdrawn: 150.3

Current Balance: 1170.0

Answer

-

Status : -

Marks : 0/10

4. Problem Statement

Theo is trying to update his payment information on a subscription-based streaming service. To proceed, the system requires Theo to provide a valid credit card number consisting of 16 digits. However, Theo wants to make sure that the credit card number he enters meets the specified criteria with proper exception handling.

The credit card number must consist of exactly 16 digits. If the entered credit card number does not meet the specified criteria, the program should throw a custom exception, `InvalidCreditCardException`, and provide Theo with specific error messages: If the length of the credit card number is not 16 digits, the exception message should be: "Invalid credit card number length." If the credit card number contains non-numeric characters, the exception message should be: "Invalid credit card number format."

Implement a custom exception, `InvalidCreditCardException`, to fulfill Theo's requirements and keep his payment information secure.

Input Format

The input consists of a string value 's', consisting of the 16-digit credit card number.

Output Format

The output is displayed in the following format:

If the entered credit card number is valid, the program should output a success message:

"Payment information updated successfully!"

If the entered credit card has more than 16 digits or less than 16 digits it displays

"Error: Invalid credit card number length."

If the entered 16-digit credit card has non-integers it displays

"Error: Invalid credit card number format."

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1234567890123456

Output: Payment information updated successfully!

Answer

-

Status : -

Marks : 0/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

REC_2028_OOPS using Java_Week 9_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 14

Section 1 : MCQ

1. Which of the following methods removes and returns the last element from a LinkedList?

Answer

removeLast()

Status : Correct

Marks : 1/1

2. What will be the output of the following code?

```
import java.util.*;
class Main {
    public static void main(String[] args) {
        ArrayList<Integer> list = new ArrayList<>();
        list.add(1);
```

```
list.add(2);  
list.add(3);  
list.add(4);  
list.add(5);  
System.out.println(list.get(3));  
}  
}
```

Answer

4

Status : Correct

Marks : 1/1

3. What is Collection in Java?

Answer

A group of interfaces

Status : Wrong

Marks : 0/1

4. What will be the output of the following code?

```
import java.util.*;  
class Main {  
    public static void main(String[] args) {  
        ArrayList<Integer> list = new ArrayList<>();  
        list.add(1);  
        list.add(2);  
        list.add(3);  
        list.add(4);  
        list.set(2, 10);  
        System.out.println(list);  
    }  
}
```

Answer

[1, 2, 10, 4]

Status : Correct

Marks : 1/1

5. Which method is used to add an element to the top of the stack?

Answer

push()

Status : Correct

Marks : 1/1

6. What will be the output of the following code?

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        Stack<Integer> stack = new Stack<>();
        for (int i = 1; i <= 3; i++)
            stack.push(i * 2);
        stack.pop();
        stack.push(10);
        System.out.println(stack.peek());
    }
}
```

Answer

10

Status : Correct

Marks : 1/1

7. What will be the output of the following code?

```
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
        ArrayList<String> list = new ArrayList<>();
        list.add("Apple");
        list.add("Banana");
        list.remove("Apple");
        System.out.println(list);
    }
}
```

Answer

[Banana]

Status : Correct

Marks : 1/1

8. What will be the output of the following code?

```
import java.util.*;
class Main {
    public static void main(String[] args) {
        ArrayList<String> list = new ArrayList<>();
        list.add("Java");
        list.add("Python");
        list.add("Java");
        list.add("C++");
        System.out.println(list.indexOf("Java"));
    }
}
```

Answer

0

Status : Correct

Marks : 1/1

9. What is the correct way to create an ArrayList in Java?

Answer

`ArrayList<String> list = new ArrayList<>();`

Status : Correct

Marks : 1/1

10. What will be the output of the following code?

```
import java.util.ArrayList;
```



```
public class Main {  
    public static void main(String[] args) {  
        ArrayList<Integer> list = new ArrayList<>();  
        list.add(10);  
        list.add(20);  
        list.add(30);  
        System.out.println("Size of the list: " + list.size());  
    }  
}
```

Answer

Size of the list: 3

Status : Correct

Marks : 1/1

11. What will be the output of the following code?

```
import java.util.*;  
public class Main {  
    public static void main(String[] args) {  
        Stack<Integer> s = new Stack<>();  
        s.push(10);  
        s.push(20);  
        s.push(30);  
        System.out.println(s.peek());  
    }  
}
```

Answer

30

Status : Correct

Marks : 1/1

12. What does the addFirst() method of LinkedList do?

Answer

Adds an element to the beginning of the list

Status : Correct

Marks : 1/1

13. What will be the output of the following code?

```
import java.util.*;
class Main {
    public static void main(String[] args) {
        ArrayList<String> list = new ArrayList<>();
        list.add("apple");
        list.add("banana");
        list.add("cherry");
        list.add("banana");
        System.out.println(list.lastIndexOf("banana"));
    }
}
```

Answer

3

Status : Correct

Marks : 1/1

14. How can you access the first element of an ArrayList named as list?

Answer

list.get(0);

Status : Correct

Marks : 1/1

15. What will be the output of the following code?

```
import java.util.*;
class Main {
    public static void main(String[] args) {
        ArrayList<Integer> list = new ArrayList<>();
        list.add(10);
        list.add(20);
        list.add(30);
        list.remove(1);
        System.out.println(list);
    }
}
```

}

Answer

[10, 30]

Status : Correct

Marks : 1/1

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 9_Q1

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Bobby is tasked with processing a sequence of numbers from a monitoring system. He needs to extract a strictly increasing subsequence using an ArrayList. The program should dynamically add numbers to the ArrayList only if they are greater than the last number currently stored in the list. Bobby aims to efficiently utilize the dynamic resizing and indexing features of the ArrayList to solve this problem.

Help Bobby implement this solution.

Input Format

The first line of input consists of an integer N, representing the number of elements.

The second line consists of N space-separated integers, representing the elements.

Output Format

The output prints the list of integers in increasing sequence, ignoring out-of-order elements.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 7

3 5 9 1 11 7 13

Output: [3, 5, 9, 11, 13]

Answer

```
import java.util.*;
```

```
public class Main
```

```
{
```

```
    public static void main(String[] args)
```

```
{
```

```
    Scanner sc = new Scanner(System.in);
```

```
    int N = sc.nextInt();
```

```
    ArrayList<Integer> list = new ArrayList<>();
```

```
for (int i = 0; i < N; i++)  
{  
  
    int num = sc.nextInt();  
  
    if (list.isEmpty() || num > list.get(list.size() - 1))  
    {  
  
        list.add(num);  
  
    }  
  
}  
  
System.out.println(list);  
sc.close();  
  
}  
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 9_Q2

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Vikram loves listening to music and wants to create a simple playlist manager using Java Collections. The playlist supports the following operations:

"ADD <song>" Adds the song to the end of the playlist. "REMOVE <song>" Removes the first occurrence of the song from the playlist. If the song is not found, do nothing. "SHOW" Displays all songs in the playlist in order. If the playlist is empty, print "EMPTY". "NEXT" Moves to the next song in the playlist and prints its name. If the playlist is empty, print "EMPTY".

The playlist maintains a "current song" position that starts at the first song when it's added. The NEXT command moves to the next song and prints it, wrapping around to the first song after reaching the last song. When removing songs, the current position adjusts accordingly to maintain

proper navigation.

Help Vikram implement this playlist manager.

Input Format

The first line of the input consists of an integer n , the number of operations.

The next n lines, each containing a command:

- "ADD <song>"
- "REMOVE <song>"
- "SHOW"
- "NEXT"

Output Format

For each "SHOW" command, print the songs in order, separated by spaces.

For each "NEXT" command, print the next song in the playlist.

If no song exists, print "EMPTY".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 7

ADD song1

ADD song2

SHOW

NEXT

REMOVE song2

SHOW

NEXT

Output: song1 song2

song2

song1

song1

Answer


```
import java.util.*;
```

```
public class Main
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        Scanner sc = new Scanner(System.in);  
        int n = Integer.parseInt(sc.nextLine());
```

```
        LinkedList<String> playlist = new LinkedList<>();  
        int currentIndex = 0;
```

```
        for (int i = 0; i < n; i++)
```

```
        {
```

```
            String commandLine = sc.nextLine();  
            String[] parts = commandLine.split(" ", 2);  
            String command = parts[0];
```

```
            switch (command)
```

```
            {
```

```
case "ADD":
```

```
    String songToAdd = parts[1];  
    playlist.add(songToAdd);
```

```
    break;
```

```
case "REMOVE":
```

```
    String songToRemove = parts[1];  
    int removeIndex = playlist.indexOf(songToRemove);  
    if (removeIndex != -1)
```

```
{
```

```
    playlist.remove(removeIndex);
```

```
    if (playlist.isEmpty())
```

```
{
```

```
        currentIndex = 0;
```

```
    } else if (removeIndex < currentIndex)
```

```
{
```

```
        currentIndex--;
```

```
    } else if (currentIndex >= playlist.size())
```

```
{
```

```
currentIndex = 0;
```

```
}
```

```
}
```

```
break;
```

```
case "SHOW":
```

```
if (playlist.isEmpty())
```

```
{
```

```
System.out.println("EMPTY");
```

```
} else
```

```
{
```

```
for (String song : playlist)
```

```
{
```

```
System.out.print(song + " ");
```

```
}
```

```
System.out.println();
```

```
}
```

```
break;
```

```
case "NEXT":
```

```
if (playlist.isEmpty())
```

```
{
```

```
System.out.println("EMPTY");
```

```
} else
```

```
{
```

```
currentIndex = (currentIndex + 1) % playlist.size();  
System.out.println(playlist.get(currentIndex));
```

```
}
```

```
break;
```

```
}
```

```
}
```

```
sc.close();
```

```
}
```

```
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 9_Q3

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Assist Pranitha in developing a program that takes an integer N as input, representing the number of names to be read. Then read N names and store them in an ArrayList. Finally, input a search string and output the frequency of that string in the list of names.

Note: Some parts of the code are provided as snippets, and you need to complete the remaining sections by writing the necessary code.

Input Format

The first line of input consists of an integer N, representing the number of names to be read.

The following N lines consist of N names, as a string.

The last line consists of a string, representing the name to be searched.

Output Format

The output prints a single integer, representing the frequency of the specified name in the given list.

If the specified name is not found, print 0.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5

Alice

Bob

Ankit

Alice

Pranitha

Alice

Output: 2

Answer

```
import java.util.*;
```

```
public class Main
```

```
{
```

```
    public static void main(String[] args)
```

```
{
```

```
Scanner sc = new Scanner(System.in);
int N = sc.nextInt();
sc.nextLine();
ArrayList<String> names = new ArrayList<>();
for (int i = 0; i < N; i++)
```

```
{
```

```
    names.add(sc.nextLine());
```

```
}
```

```
String search = sc.nextLine();
int count = 0;
for (String name : names)
```

```
{
```

```
    if (name.equals(search))
```

```
{
```

```
        count++;
```

```
}
```

```
}
```

```
System.out.println(count);
```

```
}
```


}

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

REC_2028_OOPS using Java_Week 9_PAH

Attempt : 1
Total Mark : 30
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Rekha is a teacher who wants to calculate the average of marks scored by her students in a test. She needs to store all the marks dynamically because the number of students may vary each time. Using an ArrayList allows her to easily add any number of marks without worrying about the initial size.

Help her implement the task.

Input Format

The first line of input is an integer n, representing the number of students..

The second line of input consists of n double values, representing the marks of each student, separated by a space.

Output Format

The output prints: "Average of the list: " followed by the average value formatted to two decimal places.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5

1.0 2.0 3.0 4.0 5.0

Output: Average of the list: 3.00

Answer

```
import java.util.*;
```

```
public class Main
```

```
{
```

```
    public static void main(String[] args)
```

```
{
```

```
    Scanner sc = new Scanner(System.in);
```

```
    int n = sc.nextInt();
```

```
    ArrayList<Double> marks = new ArrayList<>();
```

```
    for (int i = 0; i < n; i++)
```

```
{
```

```
marks.add(sc.nextDouble());

}
double sum = 0;
for (double m : marks)
{
    sum += m;
}
double avg = sum / n;
System.out.printf("Average of the list: %.2f", avg);

}

}
```

Status : Correct

Marks : 10/10

2. Problem Statement

Arun is building a task manager to keep track of tasks using a LinkedList. The task manager supports the following operations:

"ADD <task>" Adds the given task to the end of the list."REMOVE" Removes the first task from the list."SHOW" Displays all tasks in the list in order. If the list is empty, print "EMPTY".

Help Arun implement this functionality using a LinkedList.

Input Format

The first line of the input consists of an integer n, the number of operations.

The next n lines, each containing a command:

- "ADD <task>"
- "REMOVE"
- "SHOW"

Output Format

For each "SHOW" command, the output prints the tasks in order, separated by spaces.

If no tasks exist, print "EMPTY".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5

ADD homework

ADD project

SHOW

REMOVE

SHOW

Output: homework project
project

Answer

-

Status : -

Marks : 0/10

3. Problem Statement

Aditi is analyzing stock market trends and wants to find the Next Greater Element (NGE) for each stock price in a list. The Next Greater Element for an element x in an array is the first element to the right that is greater than x. If no greater element exists, return -1 for that position.

Your task is to help Aditi by efficiently computing the Next Greater Element for each element in the given array using a Stack.

Example:

Input:

6

4 5 2 10 8 6

Output:

5 10 10 -1 -1 -1

Explanation:

For each element:

4 5 (next greater element) 5 10 2 10 10 -1 (No greater element) 8 -16 -1

Input Format

The first line contains an integer n , representing the number of elements.

The second line contains n space-separated integers $\text{arr}[i]$, where $\text{arr}[i]$ is the stock price on the i -th day.

Output Format

The output prints n space-separated integers representing the Next Greater Element for each element in the array.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6

4 5 2 10 8 6

Output: 5 10 10 -1 -1 -1

Answer

-

Status : -

Marks : 0/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

REC_2028_OOPS using Java_Week 9_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

A teacher is filtering a list of words provided by students. Some words contain too many vowels, making them difficult for a spelling competition. The teacher decides to remove all words that contain more than two vowels.

Help the teacher to implement it using ArrayList.

Input Format

The first line contains an integer N, representing the number of words in the list.

The next N lines contain a string representing the words (one per line).

Output Format

The output consists of words that contain two or less than two vowels, printed in the same order they appeared in the input. Each word is printed on a new line.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 1

sri

Output: sri

Answer

```
import java.util.ArrayList;  
import java.util.Scanner;
```

```
import java.util.ArrayList;  
import java.util.Scanner;
```

```
class VowelFilter
```

```
{
```

```
    private static int countVowels(String word)
```

```
    {
```

```
        int count = 0;
```

```
        for (char c : word.toCharArray())
```

```
        {
```

```
            if (c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u')
```

```
            {
```

```
count++;
```

```
}
```

```
}
```

```
return count;
```

```
}
```

```
public static void filterWords(int N, Scanner scanner)
```

```
{
```

```
    ArrayList<String> filteredWords = new ArrayList<>();
```

```
    for (int i = 0; i < N; i++)
```

```
    {
```

```
        String word = scanner.nextLine();  
        int vowelCount = countVowels(word);
```

```
        if (vowelCount <= 2)
```

```
        {
```

```
            filteredWords.add(word);
```

```
        }
```

```
    }
```

```
    for (String word : filteredWords)
```

```
{  
  
    System.out.println(word);  
  
}  
  
}  
  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        int n = sc.nextInt();  
        sc.nextLine();  
        VowelFilter.filterWords(n, sc);  
        sc.close();  
    }  
}
```

Status : Correct

Marks : 10/10

2. Problem Statement

Aarav is developing a music playlist application where users can manage their favorite songs. He wants to implement a feature that allows users to reorder the playlist by moving a song from one position to another.

You need to implement a function that performs the following operations using a LinkedList:

Add songs to the playlist in the given order. Move a song from a specified position to another position in the playlist. Print the final playlist after all operations.

Input Format

The first line of the input consists of an integer n representing the number of songs.

The next n lines, each containing a string representing a song name.

After the songs are given the next line contains an integer m, the number of move operations.

The next m lines, each containing two integers x and y representing the move operation where the song at position x (0-based index) should be moved to position y.

Output Format

The output prints the final playlist, each song on a new line.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5

SongA

SongB

SongC

SongD

SongE

2

2 4

0 3

Output: SongB

SongD

SongE

SongA

SongC

Answer

-

Status : -

Marks : 0/10

3. Problem Statement

Mesa, a store manager, needs a program to manage inventory items. Define a class ItemType with private attributes for name, deposit, and cost per day. Create an ArrayList in the Main class to store ItemType objects, allowing input and display.

Note: Use "%-20s%-20s%-20s" for formatting output in tabular format, display double values with 1 decimal place.

Input Format

The first line of input consists of an integer n, representing the number of items.

For each of the n items, there are three lines:

1. The name of the item (a string)
2. The deposit amount (a double value)
3. The cost per day (a double value)

Output Format

The output prints a formatted table with columns for name, deposit and cost per day.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3
Laptop
10000.0
250.0
Light
1000.0
50.0
Fan
1000.0
100.0

Output: Name	Deposit	Cost Per Day
Laptop	10000.0	250.0
Light	1000.0	50.0

Fan 1000.0 100.0

Answer

-

Status : -

Marks : 0/10

4. Problem Statement

Raman, a computer science teacher, is responsible for registering students for his programming class. To streamline the registration process, he wants to develop a program that stores students' names and allows him to retrieve a student's name based on their index in the list.

Raman has decided to use an ArrayList to store the names of students, as it provides efficient dynamic resizing and indexing.

Write a program that enables Raman to input the names of students and fetch a student's name using the specified index. If the entered index is invalid, the program should return an appropriate message.

Input Format

The first line of input consists of an integer n , representing the number of students to register.

The next n lines of input consist of the names of each student, one by one.

The last line of input is an integer, representing the index (0-indexed) of the element to retrieve.

Output Format

If the index is valid (within the bounds of the ArrayList), print "Element at index [index]: " followed by the element (student name as string).

If the index is invalid, print "Invalid index".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5

Alice

Bob

Ankit

Alice

Prajit

2

Output: Element at index 2: Ankit

Answer

-

Status : -

Marks : 0/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

REC_2028_OOPS using Java_Week 10_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 15

Section 1 : MCQ

1. Which method retrieves the lowest key in a TreeMap?

Answer

firstKey()

Status : Correct

Marks : 1/1

2. How does HashSet check for duplicate elements?

Answer

Using equals() and hashCode()

Status : Correct

Marks : 1/1

3. Which statement is true about HashSet and TreeSet?

Answer

TreeSet provides sorted elements

Status : Correct

Marks : 1/1

4. Which method removes all elements from a Set?

Answer

clear()

Status : Correct

Marks : 1/1

5. What will be the output of the following code?

```
import java.util.*;
class Main {
    public static void main(String[] args) {
        HashMap<String, Integer> map = new HashMap<>();
        map.put("X", 10);
        map.put("Y", 20);
        map.put("Z", 30);
        map.remove("Y");
        System.out.println(map);
    }
}
```

Answer

{X=10, Z=30}

Status : Correct

Marks : 1/1

6. What happens if two keys have the same hash code in a HashMap?

Answer

A linked list is used to store values with the same hash

Status : Correct

Marks : 1/1

7. What is the time complexity of retrieving an element from a HashSet?

Answer

O(1)

Status : Correct

Marks : 1/1

8. What will happen if you add a null element to a TreeSet?

Answer

An exception occurs

Status : Correct

Marks : 1/1

9. What will be the output of the following code?

```
import java.util.*;
class Main {
    public static void main(String[] args) {
        HashMap<String, String> map = new HashMap<>();
        map.put("A", "Apple");
        map.put("B", "Banana");
        map.put("C", "Cherry");
        map.replace("B", "Blueberry");
        System.out.println(map);
    }
}
```

Answer

{A=Apple, B=Blueberry, C=Cherry}

Status : Correct

Marks : 1/1

10. Which of the following is true about HashMap?

Answer

It is not synchronized

Status : Correct

Marks : 1/1

11. Which of the following is true about TreeMap?

Answer

It maintains natural ordering

Status : Correct

Marks : 1/1

12. Which of the following allows null keys in Java?

Answer

HashMap

Status : Correct

Marks : 1/1

13. What will happen if you add elements in descending order in a TreeSet?

Answer

They are sorted in ascending order

Status : Correct

Marks : 1/1

14. What will be the output of the following code?

```
import java.util.*;
class Main {
    public static void main(String[] args) {
        HashMap<String, Integer> map = new HashMap<>();
        map.put("A", 1);
        map.put("B", 2);
        map.put("C", 3);
    }
}
```

```
        System.out.println(map.containsKey("B"));  
    }  
}
```

Answer

true

Status : Correct

Marks : 1/1

15. What happens when you add duplicate elements to a HashSet?

Answer

The duplicate is ignored

Status : Correct

Marks : 1/1

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 10_Q1

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : COD

1. Problem Statement

A city traffic management system needs to track vehicles entering a toll booth. Each vehicle is uniquely identified by its registration number. The system should allow adding vehicles to a record, ensuring that no duplicate registration numbers exist. The vehicles should be stored in a HashSet, which does not guarantee any specific order.

Your task is to implement a program using a HashSet that allows adding vehicle details and displaying the records.

Input Format

The first line of input contains an integer N - the number of vehicles.

The next N lines contain details of each vehicle in the format: "RegNumber

OwnerName VehicleType"

1. RegNumber (String) - A unique registration number (Alphanumeric).
2. OwnerName (String) - The name of the vehicle owner.
3. VehicleType (String, Car, Bike, or Truck) - The type of vehicle.

If a vehicle with the same registration number is already present, ignore the duplicate entry.

Output Format

The output prints the unique vehicle records in any order (since HashSet does not maintain order).

Output format: "RegNumber OwnerName VehicleType"

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5

KA01AB1234 John Car
MH02CD5678 Alice Bike
DL03EF9012 Bob Truck
TN04GH3456 Mike Car
KA01AB1234 John Car

Output: TN04GH3456 Mike Car
KA01AB1234 John Car
MH02CD5678 Alice Bike
DL03EF9012 Bob Truck

Answer

```
import java.util.HashSet;  
import java.util.Scanner;  
import java.util.Objects;
```

```
class Vehicle
```

```
{
```

```
private final String regNumber;  
private final String record;
```

```
public Vehicle(String regNumber, String ownerName, String vehicleType)
```

```
{
```

```
    this.regNumber = regNumber;  
    this.record = regNumber + " " + ownerName + " " + vehicleType;
```

```
}
```

```
@Override  
public boolean equals(Object o)
```

```
{
```

```
    if (this == o) return true;  
    if (o == null || getClass() != o.getClass()) return false;  
    Vehicle vehicle = (Vehicle) o;  
    return Objects.equals(regNumber, vehicle.regNumber);
```

```
}
```

```
@Override  
public int hashCode()
```

```
{
```

```
return Objects.hash(regNumber);
```

```
}
```

```
@Override
```

```
public String toString()
```

```
{
```

```
return record;
```

```
}
```

```
}
```

```
public class Main
```

```
{
```

```
public static void main(String[] args)
```

```
{
```

```
Scanner scanner = new Scanner(System.in);
```

```
int N = scanner.nextInt();
```

```
scanner.nextLine();
```

```
HashSet<Vehicle> vehicleRecords = new HashSet<>();
```



```
for (int i = 0; i < N; i++)  
{
```

```
String line = scanner.nextLine();
```

```
String[] parts = line.split(" ");
```

```
if (parts.length == 3)
```

```
{
```

```
String regNumber = parts[0];  
String ownerName = parts[1];  
String vehicleType = parts[2];
```

```
vehicleRecords.add(new Vehicle(regNumber, ownerName,  
vehicleType));
```

```
}
```

```
}
```

```
for (Vehicle vehicle : vehicleRecords)
```

```
{
```

```
System.out.println(vehicle);
```

```
}
```

```
    scanner.close();
```

```
}
```

```
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 10_Q2

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : COD

1. Problem Statement

John is organizing a fruit festival, and the quantities of various fruits are stored in a HashMap where fruit names are keys and quantities are values.

Help him develop a program to find the total quantity of fruits for the festival by summing up the values in the HashMap.

Input Format

The input consists of fruit quantities in the format 'fruitName:quantity', where fruitName is the name of the fruit(a string), and quantity is a double value representing the quantity.

The input is terminated by entering "done".

Output Format

The output prints a double value, representing the sum of values in the HashMap, rounded off to two decimal places.

If the value is not numeric, print "Invalid input".

If any special characters other than ':' are entered, print "Invalid format".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: Banana:15.2

Orange:56.3

Mango:47.3

done

Output: 118.80

Answer

```
import java.util.*;
```

```
import java.text.DecimalFormat;
```

```
public class Main
```

```
{
```

```
    public static void main(String[] args)
```

```
{
```

```
    Scanner sc = new Scanner(System.in);
```

```
    HashMap<String, Double> fruitMap = new HashMap<>();
```

```
    boolean invalidFormat = false;
```

```
boolean invalidInput = false;
while (true)
{

    String input = sc.nextLine().trim();
    if (input.equalsIgnoreCase("done"))
    {

        break;

    }
```

```
    if (!input.contains(":") || input.split(":").length != 2)
    {

        invalidFormat = true;
        break;

    }
```

```
String[] parts = input.split(":");
String fruit = parts[0];
String qtyStr = parts[1];
```

```
if (!fruit.matches("[A-Za-z]+"))
```

```
{
```

```
    invalidFormat = true;  
    break;
```

```
}
```

```
    double quantity = 0.0;  
    try
```

```
{
```

```
        quantity = Double.parseDouble(qtyStr);
```

```
    } catch (NumberFormatException e)
```

```
{
```

```
        invalidInput = true;  
        break;
```

```
}
```

```
    fruitMap.put(fruit, quantity);
```

```
}
```

```
if (invalidFormat)
```

```
{
```

```
    System.out.println("Invalid format");
```

```
} else if (invalidInput)
```

```
{
```

```
    System.out.println("Invalid input");
```

```
} else
```

```
{
```

```
    double total = 0.0;  
    for (double value : fruitMap.values())
```

```
{
```

```
        total += value;
```

```
}
```

```
    DecimalFormat df = new DecimalFormat("0.00");  
    System.out.println(df.format(total));
```

```
}
```

```
sc.close();
```

```
}
```

```
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 10_Q3

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : COD

1. Problem Statement

Priya is analyzing encrypted messages in a research project. She wants to analyze the frequency of each character in a given paragraph. The characters should be stored in a TreeMap so that the output is sorted in ascending order of characters automatically.

You are required to build a Java program that:

Uses a `TreeMap<Character, Integer>` to count how many times each character appears in the message. Ignores spaces and considers only alphabets (case-sensitive). Outputs the frequencies of characters in sorted order.

You must use a TreeMap in the class named MessageAnalyzer.

Input Format

The first line of input contains an integer n, the number of lines in the message.

The next n lines each contain a string (the encrypted message line).

Output Format

The first line of output prints: "Character Frequency:"

Then print each character and its frequency in the format: "<character>: <count>"

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 2
Hello World
Java

Output: Character Frequency:

H: 1

J: 1

W: 1

a: 2

d: 1

e: 1

l: 3

o: 2

r: 1

v: 1

Answer

```
import java.util.Scanner;  
import java.util.TreeMap;  
import java.util.Map;
```

```
class MessageAnalyzer
```

```
{
```

```
private TreeMap<Character, Integer> frequencyMap;
```

```
public MessageAnalyzer()
```

```
{
```

```
    this.frequencyMap = new TreeMap<>();
```

```
}
```

```
public void analyzeMessage(String message)
```

```
{
```

```
    for (char ch : message.toCharArray())
```

```
{
```

```
        if (Character.isLetter(ch))
```

```
{
```

```
            int count = frequencyMap.getOrDefault(ch, 0);  
            frequencyMap.put(ch, count + 1);
```

```
        }
```

```
}
```

```
}
```

```
public void displayFrequencies()
```

```
{
```

```
    System.out.println("Character Frequency:");
```

```
    for (Map.Entry<Character, Integer> entry : frequencyMap.entrySet())
```

```
{
```

```
        System.out.println(entry.getKey() + ": " + entry.getValue());
```

```
}
```

```
}
```

```
}
```

```
public class Main
```

```
{
```

```
    public static void main(String[] args)
```

```
{  
  
    Scanner scanner = new Scanner(System.in);  
  
    int n = scanner.nextInt();  
    scanner.nextLine();  
  
    MessageAnalyzer analyzer = new MessageAnalyzer();  
  
    for (int i = 0; i < n; i++)  
    {  
  
        String line = scanner.nextLine();  
        analyzer.analyzeMessage(line);  
  
    }  
  
    analyzer.displayFrequencies();  
    scanner.close();  
}  
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 10_Q4

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : COD

1. Problem Statement

In a ticket reservation system, you store the available seat numbers in a TreeSet. Users input their desired seat number, and the program checks whether the chosen seat is available.

Using a TreeSet ensures quick and efficient verification of seat availability, ensuring a smooth and organized ticket booking process.

Input Format

The first line of input contains a single integer n , representing the number of available seats.

The second line contains n space-separated integers, representing the available seat numbers.

The third line contains an integer m , representing the seat number that needs to be searched.

Output Format

The output displays "[m] is present!" if the given seat is available. Otherwise, it displays "[m] is not present!"

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

2 4 5 6

5

Output: 5 is present!

Answer

```
import java.util.Scanner;  
import java.util.TreeSet;
```

```
public class Main
```

```
{
```

```
    public static void main(String[] args)
```

```
{
```

```
    Scanner scanner = new Scanner(System.in);
```

```
    int n = scanner.nextInt();  
    scanner.nextLine();
```

```
String seatsLine = scanner.nextLine();  
String[] seatStrings = seatsLine.split(" ");
```

```
TreeSet<Integer> availableSeats = new TreeSet<>();
```

```
for (String seatStr : seatStrings)
```

```
{
```

```
    String trimmedStr = seatStr.trim();  
    if (!trimmedStr.isEmpty())
```

```
{
```

```
        try
```

```
{
```

```
            availableSeats.add(Integer.parseInt(trimmedStr));
```

```
        } catch (NumberFormatException ignored)
```

```
{
```

```
}
```



```
}  
}  
  
    int m = scanner.nextInt();  
    if (availableSeats.contains(m))  
    {  
  
        System.out.println(m + " is present!");  
  
    } else  
    {  
  
        System.out.println(m + " is not present!");  
    }  
  
    scanner.close();  
  
}  
  
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

REC_2028_OOPS using Java_Week 10_PAH

Attempt : 1
Total Mark : 30
Marks Obtained : 30

Section 1 : Coding

1. Problem Statement

A university maintains a list of student records and wants to store them in a sorted manner based on their GPA. If two students have the same GPA, they should be further sorted by their name in lexicographical order. Implement a program that uses a TreeSet to store student records and ensures unique student IDs.

Input Format

The first line contains an integer N - the number of students.

The next N lines contain details of each student in the format: "StudentID Name GPA"

- StudentID (Integer) - A unique identifier.
- Name (String) - The student's name (can contain spaces).

- GPA (Double) - The Grade Point Average.

Output Format

The output prints the list of students in ascending order of GPA.

If two students have the same GPA, sort them by name.

Print details in the format: "StudentID Name GPA" in the output, GPA is rounded to two decimal places.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5

101 John 8.5

102 Alice 9.1

103 Bob 8.5

104 Zoe 7.3

105 Charlie 9.1

Output: 104 Zoe 7.30

103 Bob 8.50

101 John 8.50

102 Alice 9.10

105 Charlie 9.10

Answer

```
import java.util.Scanner;  
import java.util.TreeSet;  
import java.util.Locale;  
import java.util.Comparator;  
class Student implements Comparable<Student>
```

```
{
```

```
private int studentID;  
private String name;  
private double gpa;
```

```
public Student(int studentID, String name, double gpa)
```

```
{
```

```
    this.studentID = studentID;  
    this.name = name;  
    this.gpa = gpa;
```

```
}
```

```
@Override
```

```
public int compareTo(Student other)
```

```
{
```

```
    if (this.gpa != other.gpa)
```

```
{
```

```
        return Double.compare(this.gpa, other.gpa);
```

```
}
```

```
    int nameComparison = this.name.compareTo(other.name);  
    if (nameComparison != 0)
```

```
{
```

```
        return nameComparison;
```

```
    }
```

```
        return Integer.compare(this.studentID, other.studentID);
```

```
    }
```

```
    @Override
```

```
    public String toString()
```

```
    {
```

```
        return String.format(Locale.US, "%d %s %.2f", studentID, name, gpa);
```

```
    }
```

```
    }
```

```
public class Main
```

```
{
```

```
    public static void main(String[] args)
```

```
{
```

```
Scanner scanner = new Scanner(System.in).useLocale(Locale.US);

int N = scanner.nextInt();
scanner.nextLine();
TreeSet<Student> studentRecords = new TreeSet<>();

for (int i = 0; i < N; i++)
{

    String line = scanner.nextLine();
    Scanner lineScanner = new Scanner(line).useLocale(Locale.US);

    int id = lineScanner.nextInt();
    StringBuilder nameBuilder = new StringBuilder();

    String token = "";
    double gpa = 0.0;

    while(lineScanner.hasNext())
    {

        token = lineScanner.next();

        try
        {

            gpa = Double.parseDouble(token);
            break;
```

```
} catch (NumberFormatException e)
```

```
{
```

```
    if (nameBuilder.length() > 0)
```

```
    {
```

```
        nameBuilder.append(" ");
```

```
    }
```

```
    nameBuilder.append(token);
```

```
}
```

```
}
```

```
    lineScanner.close();
```

```
    String name = nameBuilder.toString().trim();
```

```
    studentRecords.add(new Student(id, name, gpa));
```

```
}
```

```
    for (Student student : studentRecords)
```

```
    {
```

```
        System.out.println(student);
```

```
}
```

```
    scanner.close();
```

```
}
```

```
}
```

Status : Correct

Marks : 10/10

2. Problem Statement

Riya is building a calendar event scheduler where each event is stored in chronological order using a TreeMap. The key represents the event time in 24-hour format (HH:MM), and the value is the event description.

She wants the system to:

Automatically sort events by time. Avoid duplicate time entries — if a duplicate time is entered, ignore the new entry. Print all scheduled events in order.

Implement this logic using a class named EventManager.

Input Format

The first line of the input contains an integer n , representing the number of events.

The next n lines each contain a string in the format: "HH:MM Description"

(Example: 09:00 TeamMeeting).

Output Format

The first line of the output prints "Scheduled Events:"

The next k lines print each event in the format: "HH:MM - Description"

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5

09:00 TeamMeeting

13:30 LunchBreak

11:00 ProjectUpdate

09:00 Standup

15:00 ClientCall

Output: Scheduled Events:

09:00 - TeamMeeting

11:00 - ProjectUpdate

13:30 - LunchBreak

15:00 - ClientCall

Answer

```
import java.util.Scanner;  
import java.util.TreeMap;  
import java.util.Map;
```

```
class EventManager
```

```
{
```

```
    private TreeMap<String, String> events;
```

```
    public EventManager()
```

```
{
```

```
this.events = new TreeMap<>();
```

```
}
```

```
public void addEvent(String time, String description)
```

```
{
```

```
    events.putIfAbsent(time, description);
```

```
}
```

```
public void displayEvents()
```

```
{
```

```
    System.out.println("Scheduled Events:");
```

```
    for (Map.Entry<String, String> entry : events.entrySet())
```

```
{
```

```
        String time = entry.getKey();
```

```
        String description = entry.getValue();
```

```
        System.out.println(time + " - " + description);
```

```
    }
```

```
}
```

```
}
```

```
public class Main
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        Scanner scanner = new Scanner(System.in);
```

```
        int n = scanner.nextInt();  
        scanner.nextLine();
```

```
        EventManager manager = new EventManager();
```

```
        for (int i = 0; i < n; i++)
```

```
        {
```

```
            String line = scanner.nextLine();
```

```
            String[] parts = line.split(" ", 2);
```

```
            if (parts.length == 2)
```

```
            {
```

```
String time = parts[0];  
String description = parts[1];  
manager.addEvent(time, description);
```

```
}
```

```
}
```

```
manager.displayEvents();
```

```
scanner.close();
```

```
}
```

```
}
```

Status : Correct

Marks : 10/10

3. Problem Statement

Sarah is working on a spam detection system that analyzes incoming messages for unique patterns. Spammers often use repetitive character sequences, making it important to identify the first non-repeating character in a message.

Given a string, Sarah needs to determine the first character that appears only once. If all characters repeat, the system should return -1.

She decides to use a HashMap to efficiently track character frequencies and find the solution.

Input Format

The first line contains an integer N representing , the length of the string.

The second line contains a string of N lowercase English letters (a-z).

Output Format

The output prints a character representing the first non-repeating character. If none exist, print -1.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 10
abacabadac

Output: d

Answer

```
import java.util.HashMap;
import java.util.Scanner;

public class Main
{

    public static char findFirstNonRepeating(String s)
    {

        HashMap<Character, Integer> charCounts = new HashMap<>();
        for (char c : s.toCharArray())

        {

            charCounts.put(c, charCounts.getOrDefault(c, 0) + 1);

        }

    }

}
```

```
        for (char c : s.toCharArray())
        {

            if (charCounts.get(c) == 1)

            {

                return c;

            }

        }

        return '-';

    }
```

```
    public static void main(String[] args)
```

```
    {

        Scanner scanner = new Scanner(System.in);

        int N = scanner.nextInt();
        scanner.nextLine(); // Consume the newline

        String inputString = scanner.nextLine();

        scanner.close();

        char result = findFirstNonRepeating(inputString);

        if (result == '-')
        {
```

```
System.out.println("-1");
```

```
} else
```

```
{
```

```
System.out.println(result);
```

```
}
```

```
}
```

```
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

REC_2028_OOPS using Java_Week 10_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : COD

1. Problem Statement

A college professor wants to keep track of students who attend classes. Each student has a unique roll number and their attendance count increases every time they attend a class. The system should allow adding a student, marking their attendance, and displaying all students with their total attendance.

Your task is to implement a Java program using TreeSet to maintain students in sorted order of roll numbers and track their attendance count.

Operations:

A roll_no name Add a student with roll number and name (if not already added).M roll_no Mark attendance for the student with the given roll number (increase their count by 1).D Display all students in ascending order of roll number along with their attendance count.

Input Format

The first line contains an integer N - the number of students.

The next N lines contain one of the following commands:

A roll_no name

M roll_no

D

- A (Add) Adds a new student with a unique roll number and name.
- M (Mark) Increases attendance count for the given roll number.
- D (Display) Prints all students in ascending order of roll number.

Output Format

For D, output prints each student's roll number, name, and attendance count in ascending order of roll number.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5

A 101 Alice

A 102 Bob

M 101

M 101

D

Output: 101 Alice 2

102 Bob 0

Answer

```
import java.util.*;
```

```
class Student implements Comparable<Student>
```

```
{
```

```
    int rollNo;  
    String name;  
    int attendance;
```

```
    Student(int rollNo, String name)
```

```
{
```

```
    this.rollNo = rollNo;  
    this.name = name;  
    this.attendance = 0;
```

```
}
```

```
    @Override  
    public int compareTo(Student other)
```

```
{
```

```
        return Integer.compare(this.rollNo, other.rollNo);
```

```
}
```

```
    @Override  
    public boolean equals(Object obj)
```

```
{
```

```
if (this == obj) return true;
if (!(obj instanceof Student)) return false;
Student s = (Student) obj;
return this.rollNo == s.rollNo;
```

```
}
```

```
@Override
public int hashCode()
```

```
{
```

```
return Objects.hash(rollNo);
```

```
}
```

```
}
```

```
public class Main
```

```
{
```

```
public static void main(String[] args)
```

```
{
```

```
Scanner sc = new Scanner(System.in);
int n = Integer.parseInt(sc.nextLine());
TreeSet<Student> students = new TreeSet<>();
```

```
for (int i = 0; i < n; i++)  
{
```

```
String[] parts = sc.nextLine().split(" ");  
String command = parts[0];
```

```
if (command.equals("A"))
```

```
{
```

```
int roll = Integer.parseInt(parts[1]);  
String name = parts[2];  
students.add(new Student(roll, name));
```

```
}
```

```
else if (command.equals("M"))
```

```
{
```

```
int roll = Integer.parseInt(parts[1]);  
for (Student s : students)
```

```
{
```

```
if (s.rollNo == roll)
```

```
{
```

```
s.attendance++;  
break;
```

```
}
```

```
}
```

```
}
```

```
else if (command.equals("D"))
```

```
{
```

```
    for (Student s : students)
```

```
    {
```

```
        System.out.println(s.rollNo + " " + s.name + " " + s.attendance);
```

```
    }
```

```
}
```

```
}
```

```
}
```

}
Status : Correct

Marks : 10/10

2. Problem Statement

The city library maintains a record of books available for lending. Each book is uniquely identified by its ISBN number, along with its title and author. The librarian wants to efficiently store and manage these records, ensuring books can be listed in the order they were added.

Your task is to implement a Library Management System using HashSet where:

The librarian adds books with ISBN, title, and author. The librarian can remove books by providing an ISBN. Finally, the librarian displays the available books in the order they were added.

Implement a class Library that will handle these operations. The main function should manage user input and interact with the Library class accordingly.

Input Format

The first line contains an integer n – the number of books to be added.

The next n lines contain three values: ISBN (integer), Title (string without spaces), and Author (string without spaces).

1. An integer employee_id
2. A string title
3. A string author name

The next line contains an integer m – the number of books to be removed.

The next m lines follow, each contains an ISBN number to remove.

Output Format

The output prints a list of books available in the library after performing all operations in the format:

"ISBN: <isbn>, Title: <title>, Author: <author>"

If no books remain, print: "No books available"

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3

1234 JavaCompleteGuide JohnDoe

5678 PythonBasics JaneDoe

9012 DataStructures AliceSmith

1

5679

Output: ISBN: 1234, Title: JavaCompleteGuide, Author: JohnDoe

ISBN: 9012, Title: DataStructures, Author: AliceSmith

ISBN: 5678, Title: PythonBasics, Author: JaneDoe

Answer

```
import java.util.*;
```

```
import java.util.LinkedHashSet;
```

```
import java.util.Objects;
```

```
import java.util.Scanner;
```

```
class Book
```

```
{
```

```
    private int isbn;
```

```
    private String title;
```

```
    private String author;
```

```
    public Book(int isbn, String title, String author)
```

```
{
```

```
this.isbn = isbn;  
this.title = title;  
this.author = author;
```

```
}
```

```
@Override  
public boolean equals(Object o)
```

```
{
```

```
    if (this == o) return true;  
    if (o == null || getClass() != o.getClass()) return false;  
    Book book = (Book) o;  
    return isbn == book.isbn;
```

```
}
```

```
@Override  
public int hashCode()
```

```
{
```

```
    return Objects.hash(isbn);
```

```
}
```

```
@Override  
public String toString()
```

```
{
```

```
    return String.format("ISBN: %d, Title: %s, Author: %s", isbn, title, author);
```



```
}
```

```
public int getIsbn()
```

```
{
```

```
    return isbn;
```

```
}
```

```
}
```

```
class Library
```

```
{
```

```
    private LinkedHashSet<Book> books;
```

```
    public Library()
```

```
{
```

```
        this.books = new LinkedHashSet<>();
```

```
}
```

```
    public void addBook(int isbn, String title, String author)
```

```
{
```

```
        books.add(new Book(isbn, title, author));
```

```
}
```

```
public void removeBook(int isbnToRemove)
```

```
{
```

```
    Book dummyBook = new Book(isbnToRemove, "", "");  
    books.remove(dummyBook);
```

```
}
```

```
public void displayBooks()
```

```
{
```

```
    if (books.isEmpty())
```

```
{
```

```
        System.out.println("No books available");
```

```
    } else
```

```
{
```

```
        for (Book book : books)
```

```
{
```

```
            System.out.println(book);
```

```
        }
```

```
}  
}  
  
}
```

```
class Main {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        Library library = new Library();  
        int n = sc.nextInt();  
        for (int i = 0; i < n; i++) {  
            int isbn = sc.nextInt();  
            String title = sc.next();  
            String author = sc.next();  
            library.addBook(isbn, title, author);  
        }  
        int m = sc.nextInt();  
        for (int i = 0; i < m; i++) {  
            int isbn = sc.nextInt();  
            library.removeBook(isbn);  
        }  
        library.displayBooks();  
        sc.close();  
    }  
}
```

Status : Correct

Marks : 10/10

3. Problem Statement

Arjun is working on a program that checks if one set of numbers is a subset of another. If Set B is a subset of Set A, the program should print "YES" followed by the sorted elements of Set B. If Set B is not a subset of Set A, the program should print "NO" followed by the average of all elements from both sets combined, rounded to two decimal places.

Implement a class Solution with the required method to perform the subset check using TreeSet in Java.

Input Format

The first line contains an integer n - the number of elements in Set A.

The second line contains n space-separated integers - the elements of Set A.

The third line contains an integer m - the number of elements in Set B.

The fourth line contains m space-separated integers - the elements of Set B.

Output Format

If Set B is a subset of Set A, print "YES" followed by the sorted values of Set B.

Otherwise, print "NO" followed by the average of all numbers in both sets (rounded to two decimal places).

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5

1 2 3 4 5

3

2 3 5

Output: YES 2 3 5

Answer

```
import java.util.*;
```

```
import java.util.Scanner;
```

```
import java.util.TreeSet;
```

```
import java.util.Locale;
```

```
class Solution
```

```
{
```

```
    public static void checkSubset(TreeSet<Integer> setA, TreeSet<Integer> setB,
```

```
int totalElements, long combinedSum)
```

```
{
```

```
    if (setA.containsAll(setB))
```

```
    {
```

```
        System.out.print("YES");  
        for (int element : setB)
```

```
        {
```

```
            System.out.print(" " + element);
```

```
        }
```

```
        System.out.println();
```

```
    } else
```

```
    {
```

```
        double average = (double) combinedSum / totalElements;  
        System.out.printf(Locale.US, "NO %.2f\n", average);
```

```
    }
```

```
}
```

```
}
```

```
class Main {
```

```
    public static void main(String[] args) {
```

```

Scanner sc = new Scanner(System.in);
int n = sc.nextInt();
TreeSet<Integer> setA = new TreeSet<>();
long sum = 0;
for (int i = 0; i < n; i++) {
    int num = sc.nextInt();
    setA.add(num);
    sum += num;
}
int m = sc.nextInt();
TreeSet<Integer> setB = new TreeSet<>();
for (int i = 0; i < m; i++) {
    int num = sc.nextInt();
    setB.add(num);
    sum += num;
}
Solution.checkSubset(setA, setB, n + m, sum);
sc.close();
}
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Bob wants to develop a score-tracking application for a gaming tournament. Each player's score is stored in a HashMap with the player's name as the key and the score as the value.

Write a program to assist Bob that takes user input to enter player scores, calculates the maximum score from the HashMap, and prints the player with the highest score.

Input Format

The input consists of strings representing player details in the format "playerName:score".

The input is terminated by entering "done".

Output Format

The output displays a string, representing the player's name who scored the maximum.

If the value is not numeric, print "Invalid input".

If any special characters other than ':' are given, print "Invalid format".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: Alice:15

Bob:56

done

Output: Bob

Answer

```
import java.util.*;

import java.util.HashMap;
import java.util.Map;
import java.util.Scanner;

class ScoreTracker
{

    public HashMap<String, Integer> scoreMap;

    public ScoreTracker()
    {

        this.scoreMap = new HashMap<>();

    }

}
```

```
public boolean processInput(String line)
```

```
{
```

```
    String trimmedLine = line.trim();
```

```
    if (!trimmedLine.matches("[a-zA-Z0-9:]+$"))
```

```
    {
```

```
        System.out.println("Invalid format");
```

```
        return false;
```

```
    }
```

```
    int firstColon = trimmedLine.indexOf(':');
```

```
    int lastColon = trimmedLine.lastIndexOf(':');
```

```
    if (firstColon != lastColon || firstColon == -1 || firstColon == 0 || firstColon ==  
trimmedLine.length() - 1)
```

```
    {
```

```
        System.out.println("Invalid format");
```

```
        return false;
```

```
    }
```

```
    String[] parts = trimmedLine.split(":", 2);
```

```
    String playerName = parts[0];
```

```
    String scoreStr = parts[1];
```

```
    try
```

```
    {
```



```
int score = Integer.parseInt(scoreStr);
scoreMap.put(playerName, score);
```

```
} catch (NumberFormatException e)
```

```
{
```

```
    System.out.println("Invalid input");
    return false;
```

```
}
```

```
    return true;
```

```
}
```

```
public String findTopPlayer()
```

```
{
```

```
    String maxScorePlayer = null;
    int maxScore = Integer.MIN_VALUE;
```

```
    for (Map.Entry<String, Integer> entry : scoreMap.entrySet())
```

```
{
```

```
        if (entry.getValue() > maxScore)
```

```
{
```

```
            maxScore = entry.getValue();
            maxScorePlayer = entry.getKey();
```

```
}  
}  
    return maxScorePlayer;  
  
}  
  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Scanner scanner = new Scanner(System.in);  
        ScoreTracker tracker = new ScoreTracker();  
        boolean validInput = true;  
  
        while (true) {  
            String input = scanner.nextLine();  
  
            if (input.toLowerCase().equals("done")) {  
                break;  
            }  
  
            if (!tracker.processInput(input)) {  
                validInput = false;  
                break;  
            }  
        }  
  
        if (validInput && !tracker.scoreMap.isEmpty()) {  
            System.out.println(tracker.findTopPlayer());  
        }  
  
        scanner.close();  
    }  
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

REC_2028_OOPS using Java_Week 11

Attempt : 1
Total Mark : 20
Marks Obtained : 20

Section 1 : Project

1. Problem Statement

In Café Central, the menu is cataloged and stored in a database.

To efficiently manage the restaurant's menu using Java and JDBC, you must build a Restaurant Management System that supports:

Adding new menu items

Updating menu item prices

Viewing details of a menu item

Displaying all menu items in sorted order

You are given two files:

File 1: MenuItem.java (POJO Class)

This class represents the MenuItem entity.

A MenuItem contains the following details:

Field	Description
itemId	Unique Menu Item ID (Integer)
name	Item Name (String)
category	Item Category (String)
price	Item Price (Double)

Students must write code in the marked area:

```
class MenuItem {  
    private int itemId;  
    private String name;  
    private String category;  
    private double price;  
  
    public MenuItem() {}  
  
    public MenuItem(int itemId, String name, String category, double price) {  
        // write your code here  
    }  
  
    // Include getters and setters  
}
```

Expected in this part:

Assign parameter values to instance variables inside the constructor.

Add getters and setters for all attributes.

File 2: MenuItemDAO.java (Data Access Layer)

This class handles all database operations using JDBC.

Students must complete the missing JDBC logic in the following methods:

```
class MenuItemDAO {

    public void addItem(Connection conn, MenuItem menuItem)
    throws SQLException {

        // write your code here

    }

    public void updateItemPrice(Connection conn, int itemId, double
    newPrice) throws SQLException {

        // write your code here

    }

    public void deleteMenuItem(Connection conn, int itemId) throws
    SQLException {

        // write your code here

    }

    public MenuItem viewItemDetails(Connection conn, int itemId) throws
    SQLException {

        // write your code here

    }

    public List<MenuItem> displayAllMenuItems(Connection conn) throws
    SQLException {

        // write your code here

    }

    private MenuItem mapToMenuItem(ResultSet rs) throws SQLException {
        return new MenuItem(
```

```
        // write your code here
    );
}
}
```

Expected in this part:

Write SQL queries for INSERT, UPDATE, DELETE, SELECT.

Execute queries using PreparedStatement or Statement.

Map ResultSet rows to MenuItem objects using mapToMenuItem().

Return a List<MenuItem> where required.

The system should connect to a MySQL database using the following default credentials:

DB URL: jdbc:mysql://localhost/ri_db

USER: test

PWD: test123

The menu table has already been created with the following structure:

Table Name: menu

Input Format

The first line of input consists of an integer choice, representing the operation to be performed (1 for Add Item, 2 for Restock item, 3 for reduce item, 4 for Display, 5 for Exit).

For choice 1 (Add Menu Item):

- The second line consists of an integer item_id.
- The third line consists of a string name.
- The fourth line consists of a string category.
- The fifth line consists of a double price.

For choice 2 (Update Item Price):

- The second line consists of an integer item_id.
- The third line consists of a double new_price.

For choice 3 (View Item Details):

- The second line consists of an integer item_id.

For choice 4 (Display All Menu Items):

- No additional inputs are required.

For choice 5 (Exit):

- No additional inputs are required.

Output Format

For choice 1 (Add Menu Item):

- Print "Menu item added successfully" if the item was added.
- Print "Failed to add item." if the insertion failed.

For choice 2 (Update Item Price):

- Print "Item price updated successfully" if the price update was successful.
- Print "Item not found." if the specified item ID does not exist.

For choice 3 (View Item Details):

- Display the item details in the format:
- ID: [item_id] | Name: [name] | Category: [category] | Price: [price]
- Print "Item not found." if the specified item ID does not exist.

For choice 4 (Display All Menu Items):

- Display each item on a new line in the format:
- ID | Name | Category | Price
- If no items are available, print nothing (or handle with an appropriate message if desired).

For choice 5 (Exit):

- Print "Exiting Restaurant Management System."

For invalid input:

- Print "Invalid choice. Please try again."

Sample Test Case

Input: 1

11

Margherita Pizza

Main Course

12.99

4

5

Output: Menu item added successfully

ID | Name | Category | Price

11 | Margherita Pizza | Main Course | 12.99

Exiting Restaurant Management System.

Answer

```
import java.sql.*;
```

```
import java.util.Scanner;
```

```
class RestaurantManagementSystem {
```

```
    public static void main(String[] args) {
```

```
        try (Connection conn = DriverManager.getConnection("jdbc:mysql://localhost/ri_db", "test", "test123"));
```

```
            Scanner scanner = new Scanner(System.in)) {
```

```
                boolean running = true;
```

```
                while (running) {
```

```
                    int choice = scanner.nextInt();
```

```
                    switch (choice) {
```

```
                        case 1:
```

```
                            addMenuItem(conn, scanner);
```

```
                            break;
```

```
                        case 2:
```

```
                            updateItemPrice(conn, scanner);
```

```
                            break;
```



```

        case 3:
            viewItemDetails(conn, scanner);
            break;
        case 4:
            displayAllMenuItems(conn);
            break;
        case 5:
            System.out.println("Exiting Restaurant Management System.");
            running = false;
            break;
        default:
            System.out.println("Invalid choice. Please try again.");
    }
}
} catch (SQLException e) {
    e.printStackTrace();
}
}
}

```

```

public static void addItem(Connection conn, Scanner scanner){

```

```

    int itemId = scanner.nextInt();
    scanner.nextLine(); // Consume newline

```

```

    String name = scanner.nextLine();
    String category = scanner.nextLine();
    double price = scanner.nextDouble();

```

```

    MenuItem menuItem = new MenuItem(itemId, name, category, price); //
    Using POJO

```

```

    String insertQuery = "INSERT INTO menu (item_id, name, category, price)
VALUES (?, ?, ?, ?)";

```

```

    try (PreparedStatement stmt = conn.prepareStatement(insertQuery))

```

```

    {

```

```

        stmt.setInt(1, menuItem.getItemId());
        stmt.setString(2, menuItem.getName());

```

```
stmt.setString(3, menuItem.getCategory());
stmt.setDouble(4, menuItem.getPrice());

int rowsInserted = stmt.executeUpdate();
System.out.println(rowsInserted > 0 ? "Menu item added successfully" :
"Failed to add item.");
```

```
} catch (SQLException e)
```

```
{
```

```
System.out.println("Error adding item: " + e.getMessage());
```

```
}
```

```
}
```

```
public static void updateItemPrice(Connection conn, Scanner scanner)
```

```
{
```

```
int itemId = scanner.nextInt();
double newPrice = scanner.nextDouble();
```

```
String updateQuery = "UPDATE menu SET price = ? WHERE item_id = ?";
try (PreparedStatement stmt = conn.prepareStatement(updateQuery))
```

```
{
```

```
stmt.setDouble(1, newPrice);
stmt.setInt(2, itemId);
```

```
int rowsUpdated = stmt.executeUpdate();
System.out.println(rowsUpdated > 0 ? "Item price updated successfully" :
"Item not found.");
```

```
} catch (SQLException e)
```

```
{
```

```
    System.out.println("Error updating price: " + e.getMessage());
```

```
}
```

```
}
```

```
public static void viewItemDetails(Connection conn, Scanner scanner)
```

```
{
```

```
    int itemId = scanner.nextInt();
```

```
    String selectQuery = "SELECT * FROM menu WHERE item_id = ?";
```

```
    try (PreparedStatement stmt = conn.prepareStatement(selectQuery))
```

```
{
```

```
        stmt.setInt(1, itemId);
```

```
        ResultSet rs = stmt.executeQuery();
```

```
        if (rs.next())
```

```
{
```

```
            MenuItem menuItem = new MenuItem(
```

```
                rs.getInt("item_id"),
```

```
                rs.getString("name"),
```

```
                rs.getString("category"),
```

```
                rs.getDouble("price")
```

```
            );
```

```
System.out.printf("ID: %d | Name: %s | Category: %s | Price: %.2f%n",
    menuItem.getItemId(),
    menuItem.getName(),
    menuItem.getCategory(),
    menuItem.getPrice());
```

```
} else
```

```
{
```

```
    System.out.println("Item not found.");
```

```
}
```

```
} catch (SQLException e)
```

```
{
```

```
    System.out.println("Error retrieving item details: " + e.getMessage());
```

```
}
```

```
}
```

```
public static void displayAllMenuItems(Connection conn)
```

```
{
```

```
    String displayQuery = "SELECT * FROM menu ORDER BY item_id";
```

```
    try (Statement stmt = conn.createStatement();
```

```
        ResultSet rs = stmt.executeQuery(displayQuery))
```

```
{
```

```
System.out.println("ID | Name | Category | Price");  
while (rs.next())
```

```
{
```

```
    MenuItem menuItem = new MenuItem(  
        rs.getInt("item_id"),  
        rs.getString("name"),  
        rs.getString("category"),  
        rs.getDouble("price")  
    );
```

```
    System.out.printf("%d | %s | %s | %.2f%n",  
        menuItem.getItemId(),  
        menuItem.getName(),  
        menuItem.getCategory(),  
        menuItem.getPrice());
```

```
}
```

```
} catch (SQLException e)
```

```
{
```

```
    System.out.println("Error displaying menu items: " + e.getMessage());
```

```
}
```

```
}
```

```
}
```

```
class MenuItem
```

```
{  
    private int itemId;  
    private String name;  
    private String category;  
    private double price;  
  
    // Constructor  
    public MenuItem(int itemId, String name, String category, double price)  
  
    {  
        this.itemId = itemId;  
        this.name = name;  
        this.category = category;  
        this.price = price;  
  
    }  
  
    // Getters and Setters  
    public int getItemId()  
  
    {  
        return itemId;  
    }  
    public void setItemId(int itemId)  
  
    {  
        this.itemId = itemId;  
    }  
  
    public String getName()
```

```
return name;
```

```
}
```

```
public void setName(String name)
```

```
{
```

```
    this.name = name;
```

```
}
```

```
public String getCategory()
```

```
{
```

```
    return category;
```

```
}
```

```
public void setCategory(String category)
```

```
{
```

```
    this.category = category;
```

```
}
```

```
public double getPrice()
```

```
{
```

```
    return price;
```

```
}
```

```
public void setPrice(double price)
```

```
{
```

```
    this.price = price;
```

```
}
```

```
}
```

//

Status : Correct

Marks : 10/10

2. Problem Statement

Create a JDBC-based Inventory Management System that handles runtime input to manage items in an inventory. The system should allow users to:

Add a new item (item ID, name, quantity, price).

Restock an item by increasing its quantity.

Reduce the stock of an item, ensuring sufficient quantity.

Display all items in the inventory in a sorted order by item ID.

Exit the application.

Half of the code is given here; Only the remaining part should be completed.

The system should connect to a MySQL database using the following default credentials:

DB URL: jdbc:mysql://localhost/ri_db

USER: test

PWD: test123

The items table has already been created with the following structure:

Table Name: items

Input Format

The first line of input consists of an integer choice, representing the operation to be performed (1 for Add Item, 2 for Restock item, 3 for reduce item, 4 for Display, 5 for Exit).

For choice 1 (Add Item):

- The second line consists of an integer item_id.
- The third line consists of a string name.
- The fourth line consists of an integer quantity.
- The fifth line consists of a double price.

For choice 2 (Restock Item):

- The second line consists of an integer item_id.
- The third line consists of an integer quantity_to_add (must be positive).

For choice 3 (Reduce Stock):

- The second line consists of an integer item_id.
- The third line consists of an integer quantity_to_remove (must be positive).

For choice 4 (Display Inventory):

- No additional inputs are required.

For choice 5 (Exit):

- No additional inputs are required.

Output Format

For choice 1 (Add Item):

- Print "Item added successfully" if the item was added.
- Print "Failed to add item." if the insertion failed.

For choice 2 (Restock Item):

- Print "Item restocked successfully" if the restock was successful.
- Print "Item not found." if the specified item ID does not exist.

For choice 3 (Reduce Stock):

- Print "Stock reduced successfully" if the stock reduction was successful.
- Print "Not enough stock to remove." if there is insufficient quantity.

- Print "Item not found." if the specified item ID does not exist.

For choice 4 (Display Inventory):

- Display each item on a new line in the format:
- ID | Name | Quantity | Price
- If no items are available, print nothing (or handle with an appropriate message if desired).

For choice 5 (Exit):

- Print "Exiting Inventory Management System."

For invalid input:

- Print "Invalid choice. Please try again."

Sample Test Case

Input: 1

101

Laptop

50

1200.00

4

5

Output: Item added successfully

ID | Name | Quantity | Price

101 | Laptop | 50 | 1200.00

Exiting Inventory Management System.

Answer

```
import java.sql.*;
```

```
import java.util.Scanner;
```

```
class InventoryManagementSystem {
```

```
    public static void main(String[] args) {
```

```
        try (Connection conn = DriverManager.getConnection("jdbc:mysql://localhost/ri_db", "test", "test123");
```

```
            Scanner scanner = new Scanner(System.in)) {
```

```
                boolean running = true;
```

```

while (running) {

    int choice = scanner.nextInt();

    switch (choice) {
        case 1:
            addItem(conn, scanner);
            break;
        case 2:
            restockItem(conn, scanner);
            break;
        case 3:
            reduceStock(conn, scanner);
            break;
        case 4:
            displayInventory(conn);
            break;
        case 5:
            System.out.println("Exiting Inventory Management System.");
            running = false;
            break;
        default:
            System.out.println("Invalid choice. Please try again.");
    }
}
} catch (SQLException e) {
    e.printStackTrace();
}
}

private static void addItem(Connection conn, Scanner scanner){

```

```

try
{

```

```

    int itemId = scanner.nextInt();
    scanner.nextLine();
    String name = scanner.nextLine();

```

```
int quantity = scanner.nextInt();
double price = scanner.nextDouble();

String sql = "INSERT INTO items (item_id, name, quantity, price) VALUES
(?, ?, ?, ?)";
PreparedStatement ps = conn.prepareStatement(sql);
ps.setInt(1, itemId);
ps.setString(2, name);
ps.setInt(3, quantity);
ps.setDouble(4, price);

int rows = ps.executeUpdate();
if (rows > 0)
    System.out.println("Item added successfully");
else
    System.out.println("Failed to add item.");

} catch (SQLException e)

{

    System.out.println("Failed to add item.");

}

}

private static void restockItem(Connection conn, Scanner scanner)

{

    try

    {

        int itemId = scanner.nextInt();
        int quantityToAdd = scanner.nextInt();
```

```
String checkSql = "SELECT quantity FROM items WHERE item_id = ?";
PreparedStatement checkStmt = conn.prepareStatement(checkSql);
checkStmt.setInt(1, itemId);
ResultSet rs = checkStmt.executeQuery();
```

```
if (!rs.next())
```

```
{
```

```
    System.out.println("Item not found.");
    return;
```

```
}
```

```
int newQuantity = rs.getInt("quantity") + quantityToAdd;
```

```
String updateSql = "UPDATE items SET quantity = ? WHERE item_id = ?";
PreparedStatement updateStmt = conn.prepareStatement(updateSql);
updateStmt.setInt(1, newQuantity);
updateStmt.setInt(2, itemId);
updateStmt.executeUpdate();
```

```
System.out.println("Item restocked successfully");
```

```
} catch (SQLException e)
```

```
{
```

```
    System.out.println("Item not found.");
```

```
}
```

```
}
```

```
private static void reduceStock(Connection conn, Scanner scanner)
```

```
{  
    try  
    {  
  
        int itemId = scanner.nextInt();  
        int qtyToRemove = scanner.nextInt();  
  
        String checkSql = "SELECT quantity FROM items WHERE item_id = ?";  
        PreparedStatement checkStmt = conn.prepareStatement(checkSql);  
        checkStmt.setInt(1, itemId);  
        ResultSet rs = checkStmt.executeQuery();  
  
        if (!rs.next())  
        {  
  
            System.out.println("Item not found.");  
            return;  
  
        }  
  
        int currentQty = rs.getInt("quantity");  
  
        if (currentQty < qtyToRemove)  
        {  
  
            System.out.println("Not enough stock to remove.");  
            return;  
  
        }  
  
        int updatedQty = currentQty - qtyToRemove;
```

```
String updateSql = "UPDATE items SET quantity = ? WHERE item_id = ?";
PreparedStatement updateStmt = conn.prepareStatement(updateSql);
updateStmt.setInt(1, updatedQty);
updateStmt.setInt(2, itemId);
updateStmt.executeUpdate();
```

```
System.out.println("Stock reduced successfully");
```

```
} catch (SQLException e)
```

```
{
```

```
    System.out.println("Item not found.");
```

```
}
```

```
}
```

```
private static void displayInventory(Connection conn)
```

```
{
```

```
    try
```

```
    {
```

```
        String sql = "SELECT * FROM items ORDER BY item_id";
        PreparedStatement ps = conn.prepareStatement(sql);
        ResultSet rs = ps.executeQuery();
```

```
        System.out.println("ID | Name | Quantity | Price"); // REQUIRED HEADER
```

```
        while (rs.next())
```

```
        {
```

```
System.out.println(  
    rs.getInt("item_id") + " | " +  
    rs.getString("name") + " | " +  
    rs.getInt("quantity") + " | " +  
    String.format("%.2f", rs.getDouble("price"))  
);
```

```
}
```

```
} catch (SQLException e)
```

```
{
```

```
    System.out.println("Error displaying inventory.");
```

```
}
```

```
}
```

```
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

REC_Week 12_Java_Lamba Expressions_MCQ

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : MCQ

1. Can a lambda expression in Java have a body with multiple statements?

Answer

Yes, if the statements are enclosed in curly braces

Status : Correct

Marks : 1/1

2. Can a lambda expression in Java have a body with multiple statements?

Answer

Yes, if the statements are enclosed in curly braces

Status : Correct

Marks : 1/1

3. Which of the following is a valid lambda expression in Java?

Answer

All of the mentioned options

Status : Correct

Marks : 1/1

4. Which functional interface in Java takes two arguments and returns a result?

Answer

BiFunction

Status : Correct

Marks : 1/1

5. What is the syntax for a basic lambda expression in Java?

Answer

(parameters) -> expression

Status : Correct

Marks : 1/1

6. What is a lambda expression in Java?

Answer

A way to define anonymous methods

Status : Correct

Marks : 1/1

7. Can a lambda expression have more than one parameter?

Answer

Yes, it can have multiple parameters

Status : Correct

Marks : 1/1

8. Which functional interface is commonly used with lambda expressions in Java?

Answer

Runnable

Status : Correct

Marks : 1/1

9. Which of the following interfaces is NOT a functional interface in Java?

Answer

Iterable

Status : Correct

Marks : 1/1

10. What is the return type of a lambda expression in Java?

Answer

The return type is inferred from the context

Status : Correct

Marks : 1/1

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 12_Q1

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Sabrina is working on a project that involves analyzing a set of numbers. In her exploration, she encounters scenarios where extracting even numbers and finding their sum is essential.

Create a program that calculates the sum of even numbers from a given array of integers using a lambda expression.

Input Format

The first line of input consists of an integer N, representing the size of the array.

The second line consists of N space-separated integers, representing the elements of the array.

Output Format

The output prints the sum of the even integers from the array.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3

29 37 45

Output: 0

Answer

```
import java.util.*;
import java.util.stream.*;

class Main

{

    public static void main(String[] args)
    {

        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];
        for(int i=0;i<n;i++)
            arr[i]=sc.nextInt();
        int sum = Arrays.stream(arr)
            .filter(x -> x % 2 == 0)
            .sum();
        System.out.println(sum);
    }
}
```

}
}

240701805

240701805

240701805

Status : Correct

Marks : 10/10

240701805

240701805

240701805

240701805

240701805

240701805

240701805

240701805

240701805

240701805

240701805

240701805

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 12_Q2

Attempt : 1
Total Mark : 10
Marks Obtained : 0

Section 1 : Coding

1. Problem Statement

Alex is learning about Java's functional interfaces and lambda expressions.

He wants to write a simple program that prints the square of each number in an array using a predefined functional interface.

Help Alex complete this task using the Consumer functional interface.

Input Format

- The first line contains an integer N, the number of elements in the array.
- The second line contains N space-separated integers.

Output Format

- Print the squares of all elements in the array, separated by a space.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4

1 2 3 4

Output: 1 4 9 16

Answer

-

Status : Skipped

Marks : 0/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 12_Q3

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

In the mystical realm of programming, there exists a magical incantation to reveal hidden words.

Elara, the skilled enchantress, wishes to summon a word using her spell and then reverse its characters to uncover its enchanted reflection.

Write a program that uses the predefined functional interface `Supplier<String>` and a lambda expression to:

Supply (generate) a string, and

Display its reversed form.

Input Format

No input is required from the user.

The string must be supplied internally using a Supplier<String>.

Output Format

Print the reversed version of the supplied string.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: Wizard!!

Output: !!draziW

Answer

```
import java.util.function.Supplier;
import java.util.Scanner;
```

```
public class Main
```

```
{
```

```
    public static void main(String[] args)
```

```
{
```

```
    Supplier<String> spell = () ->
```

```
{
```

```
Scanner sc = new Scanner(System.in);  
String s = sc.hasNextLine() ? sc.nextLine() : "";  
return s;
```

```
};
```

```
String word = spell.get();  
System.out.println(new StringBuilder(word).reverse().toString());
```

```
}
```

```
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 12_Q4

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Abi is working on a text analysis project where she needs to categorize words based on their length.

Words that have three or fewer characters are considered "Short", while words with more than three characters are classified as "Long."

Write a Java program that takes a sentence as input, analyzes each word, and prints a list showing whether each word is "Short" or "Long."

Use the predefined functional interface `Function<String, String>` along with a lambda expression for categorization.

Input Format

A single line containing a sentence (words separated by spaces).

Output Format

- A single line with each word categorized as "Short" or "Long", separated by spaces.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: I love my cat

Output: Short Long Short Short

Answer

```
import java.util.*;  
import java.util.function.Function;
```

```
public class Main
```

```
{
```

```
    public static void main(String[] args)
```

```
{
```

```
    Scanner sc = new Scanner(System.in);
```

```
    String sentence = sc.nextLine();
```

```
    String[] words = sentence.split(" ");
```

```
    Function<String, String> categorize = word -> word.length() <= 3 ? "Short" :
```

```
    "Long";
```

```
    for (String word : words)
```

```
{
```

```
System.out.print(categorize.apply(word) + " ");
```

```
}
```

```
}
```

```
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

REC_Week 12_Java_Lamba Expressions_PAH

Attempt : 1
Total Mark : 40
Marks Obtained : 10

Section 1 : COD

1. Problem Statement

Aditya is developing a reading app that recommends books to users based on a predefined list.

Each time a user opens the app, it should supply the next book title in the list, one at a time, using a lambda expression and the Supplier functional interface.

When all books have been recommended, the list should start again from the beginning.

Input Format

The first line contains an integer n — the total number of available book titles.

The next n lines each contain a book title (a string).

The next line contains an integer m — the number of times users open the app (i.e., the number of recommendations to be made).

Output Format

Print the supplied book title for each recommendation, one per line.

If $m > n$, repeat the list from the start.

Sample Test Case

Input: 3

The Alchemist

Atomic Habits

Ikigai

5

Output: The Alchemist

Atomic Habits

Ikigai

The Alchemist

Atomic Habits

Answer

```
import java.util.*;
```

```
import java.util.function.Supplier;
```

```
public class Main
```

```
{
```

```
    public static void main(String[] args)
```

```
{
```



```
Scanner sc = new Scanner(System.in);
int n = Integer.parseInt(sc.nextLine());
List<String> books = new ArrayList<>();
for (int i = 0; i < n; i++)

{

    books.add(sc.nextLine());

}
int m = Integer.parseInt(sc.nextLine());
int[] index =

{

0

};
Supplier<String> getNextBook = () ->

{

    String book = books.get(index[0]);
    index[0] = (index[0] + 1) % books.size();
    return book;

};
for (int i = 0; i < m; i++)

{
```

```
System.out.println(getNextBook.get());
```

```
}
```

```
}
```

```
}
```

Status : Correct

Marks : 10/10

2. Problem Statement

Emily, an analyst at a data processing firm, is tasked with cleaning up datasets to remove duplicate values from lists of integers.

Create a Java program that allows Emily to input a series of integers, with the program then utilizing a lambda expression to efficiently remove any duplicates.

Input Format

The first line of input consists of an integer N, representing the size of the array.

The second line consists of N space-separated integers, each denoting an array element.

Output Format

The output prints the array elements after removing the duplicates inside the square bracket separated by a comma and space.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 15

1 2 3 4 3 2 1 2 3 4 4 4 5 5 6

Output: [1, 2, 3, 4, 5, 6]

Answer

-

Status : -

Marks : 0/10

3. Problem Statement

Rishi is working as an HR analyst in a software company. He wants to filter a list of employees based on their salary using modern Java techniques. He has a list of employee names and salaries and wants to use lambda expressions to filter those who earn more than a specific threshold.

Implement a program using lambda expressions and functional interfaces to print the names of employees whose salary is greater than or equal to 50,000.

Input Format

The first line of input consists of an integer n , representing the number of employees.

The next n lines. Each line contains a String (employee name) and an int (salary).

Output Format

The output prints the names of employees whose salary is greater than or equal to 50000, each on a new line.

If no employee found with salary greater than 50000, print: No employee found with salary ≥ 50000

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4
Amit 45000

Sneha 50000
Ravi 60000
Priya 30000
Output: Sneha
Ravi

Answer

-

Status : -

Marks : 0/10

4. Problem Statement

Sneha is developing a feature for an e-commerce application that helps display product details after applying a seasonal discount.

She decides to use lambda expressions with the Consumer functional interface to print each product's name, original price, and discounted price neatly.

The program should:

Accept a list of product names and their prices. Apply a 15% discount on all products. Use a Consumer lambda expression to display the details in a formatted manner.

Input Format

The first line of input consists of an integer n , representing the number of products.

The next n lines each contain a String (product name) and a double (price) separated by a space.

Output Format

For each product, print the details in the format:

Product: <name>, Original Price: <price>, Discounted Price: <discounted price>

If there are no products, print:

No products available

Sample Test Case

Input: 1

Phone 60000

Output: Product: Phone, Original Price: 60000.0, Discounted Price: 51000.0

Answer

-

Status : -

Marks : 0/10

Rajalakshmi Engineering College

Name: VS THAMIZH SELVAN
Email: 240701805@rajalakshmi.edu.in
Roll no: 240701805
Phone: 6379580366
Branch: REC
Department: CSE - Section 6
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

REC_Week 12_Java_Lambda Expressions_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

A company named TechNova is collecting feedback from its customers. Each customer gives a feedback score (an integer between 1 and 10) along with their name.

The company wants to:

Display each customer's name along with their feedback in a formatted way using a lambda expression and a Consumer functional interface. After displaying all feedbacks, calculate and display the average feedback score. You need to implement this functionality using Java lambda expressions and streams, emphasizing the Consumer interface for displaying formatted output.

Input Format

The first line of input contains an integer n, representing the number of customers.

The next n lines each contain a String (customer name) followed by an int (feedback score).

Output Format

- Each line prints a customer's name and feedback in the format:
- Customer: <name>, Feedback Score: <score>

- After all customers are displayed, print the average feedback as:
- Average Feedback: <average_value>

(Average should be displayed up to two decimal places.)

Sample Test Case

Input: 3

Ravi 7

Ananya 9

Kiran 8

Output: Customer: Ravi, Feedback Score: 7

Customer: Ananya, Feedback Score: 9

Customer: Kiran, Feedback Score: 8

Average Feedback: 8.00

Answer

```
import java.util.*;  
import java.util.function.Consumer;  
import java.util.stream.Collectors;
```

```
public class Main
```

```
{
```

```
public static void main(String[] args)
{

    Scanner sc = new Scanner(System.in);
    int n = Integer.parseInt(sc.nextLine());
    List<String> names = new ArrayList<>();
    List<Integer> scores = new ArrayList<>();

    for (int i = 0; i < n; i++)
    {

        String[] input = sc.nextLine().split(" ");
        names.add(input[0]);
        scores.add(Integer.parseInt(input[1]));

    }

    Consumer<Integer> displayFeedback = i ->
        System.out.println("Customer: " + names.get(i) + ", Feedback Score: " +
        scores.get(i));

    for (int i = 0; i < n; i++)
    {

        displayFeedback.accept(i);
```



```
}  
  
    double average =  
scores.stream().collect(Collectors.averagingInt(Integer::intValue));  
    System.out.printf("Average Feedback: %.2f", average);  
  
}  
  
}
```

Status : Correct

Marks : 10/10

2. Problem Statement

Problem Statement

Sophia, a data analyst, is studying experimental results collected from various lab sensors. Each sensor provides a list of numeric readings, and Sophia wants to calculate the average of these readings to analyze consistency.

She decides to use lambda expressions and the Function functional interface to compute the average of all the recorded values efficiently.

Your Task

Write a Java program that:

Reads the total number of measurements. Reads all the measurement values as doubles. Uses a `Function<double[], Double>` lambda expression to calculate the average value. Displays the final average, formatted to two decimal places.

Input Format

The first line of input consists of an integer `N`, representing the number of measurements.

The second line contains `N` space-separated double values.

Output Format

Print the average of the entered values, rounded to two decimal places.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6

2.2 1.2 5.4 4.6 2.9 55.7

Output: 12.00

Answer

```
import java.util.*;  
import java.util.function.Function;
```

```
public class Main
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        int n = sc.nextInt();
```

```
        double[] readings = new double[n];
```

```
        for (int i = 0; i < n; i++)
```

```
    {
```

```
readings[i] = sc.nextDouble();
```

```
}
```

```
Function<double[], Double> average = arr ->
```

```
{
```

```
    double sum = 0;  
    for (double val : arr) sum += val;  
    return sum / arr.length;
```

```
};
```

```
double avg = average.apply(readings);  
System.out.printf("%.2f", avg);
```

```
}
```

```
}
```

Status : Correct

Marks : 10/10

3. Problem Statement

Riya is developing a college admission system that assigns unique roll numbers to each newly admitted student.

Each roll number should follow this fixed format:

<DEPT>-<YEAR>-<4-digit-sequence>

where:

<DEPT> is the department code (in uppercase, e.g., CSE, ECE, MECH). <YEAR> is the admission year (e.g., 2025). <4-digit-sequence> starts from a given number and increases sequentially for each student. Write a Java program using a Supplier<String> lambda to generate and print the roll numbers for n students.

Input Format

First line: integer n — number of roll numbers to generate

Second line: string DEPT — department code (uppercase letters only)

Third line: integer YEAR — admission year

Fourth line: integer start — starting sequence number ($0 \leq \text{start} \leq 9999$)

Output Format

Print n roll numbers, one per line, in the required format

Sequence must be zero-padded to 4 digits

If sequence exceeds 9999, wrap around to 0000

Sample Test Case

Input: 5

CSE

2025

98

Output: CSE-2025-0098

CSE-2025-0099

CSE-2025-0100

CSE-2025-0101

CSE-2025-0102

Answer

```
import java.util.*;  
import java.util.function.Supplier;
```

```
public class Main
```

```
{
```

```
    public static void main(String[] args)
```

```
{
```

```
    Scanner sc = new Scanner(System.in);  
    int n = Integer.parseInt(sc.nextLine());  
    String dept = sc.nextLine();  
    int year = Integer.parseInt(sc.nextLine());  
    int start = Integer.parseInt(sc.nextLine());
```

```
    int[] seq =
```

```
{
```

```
        start
```

```
    };
```

```
    Supplier<String> rollSupplier = () ->
```

```
{
```

```
    String roll = String.format("%s-%d-%04d", dept, year, seq[0]);  
    seq[0] = (seq[0] + 1) % 10000;  
    return roll;
```

```
};
```

```
for (int i = 0; i < n; i++)  
{  
  
    System.out.println(rollSupplier.get());  
  
}  
  
}  
}
```

Status : Correct

Marks : 10/10

4. Problem Statement

Nethra is a researcher working on a project that involves analyzing experimental data. As part of her analysis, she needs to determine whether a given word is a palindrome or not.

Create a Java program that allows Nethra to input a word, and then check and display whether the entered word is a palindrome. Use lambda expressions to perform the palindrome check.

Input Format

The first line of input consists of a word.

Output Format

The output prints whether the given word is a palindrome or not in the following format:

"<input> is palindrome" or "<input> is not palindrome".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: malayalam

Output: malayalam is palindrome

Answer

// You are using Java

import java.util.*;

import java.util.function.Predicate;

public class Main

{

public static void main(String[] args)

{

Scanner sc = new Scanner(System.in);
String word = sc.nextLine();

Predicate<String> isPalindrome = s ->

{

String rev = new StringBuilder(s).reverse().toString();
return s.equals(rev);

```
};
```

```
    if (isPalindrome.test(word))
```

```
    {
```

```
        System.out.println(word + " is palindrome");
```

```
    } else
```

```
    {
```

```
        System.out.println(word + " is not palindrome");
```

```
    }
```

```
}
```

```
}
```

Status : Correct

Marks : 10/10